

LLM Inference Optimization

Spring 2026

Lecturer: Yuedong (Steven) Xu

Fudan University

ydxu@fudan.edu.cn

Disclaimer

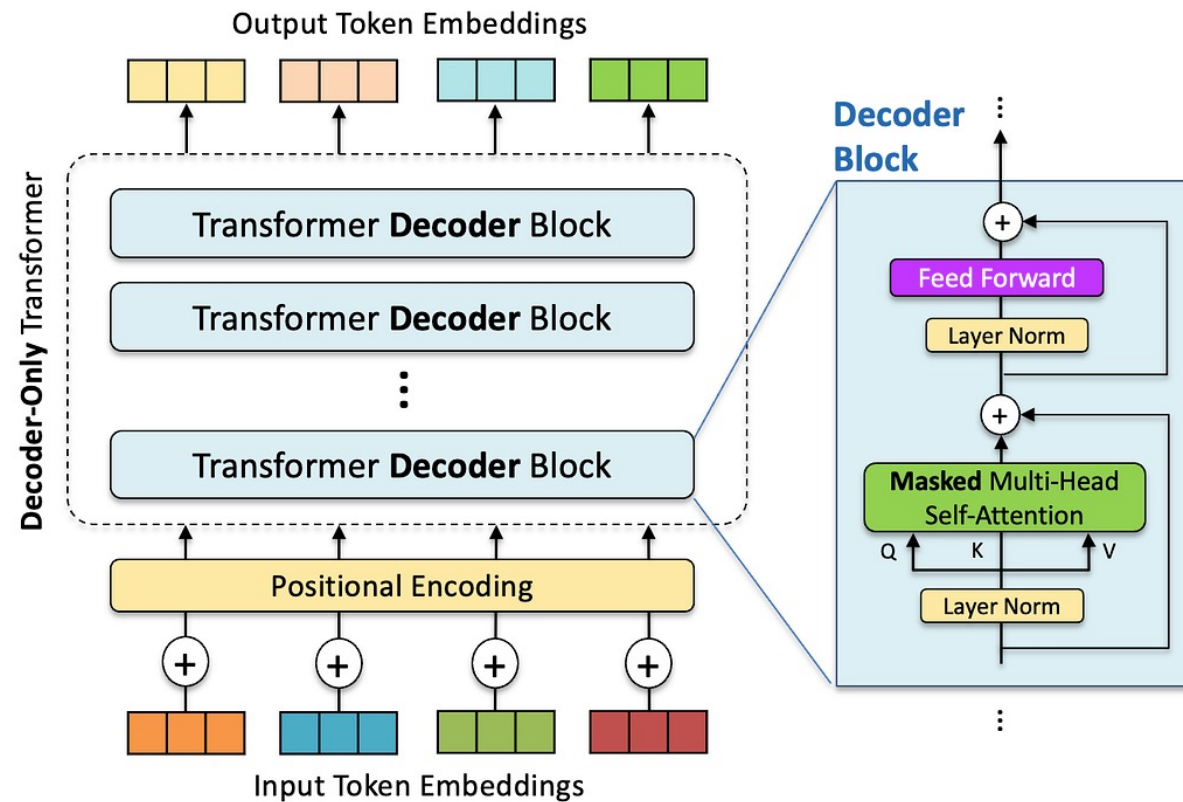
Machine learning systems is a broad and rapidly evolving field. The course material has been developed using a broad spectrum of resources, including research papers, lecture slides, blogposts, research talks, tutorial videos, and other materials shared by the research community. Sometimes external animations and exquisite pictures are heavily reused.

Inference Optimization: Outline

- Overview
- Attention Optimization
- Continuous Batching
- KV Cache Optimization
- Speculative Decoding
- Distributed Serving

Overview

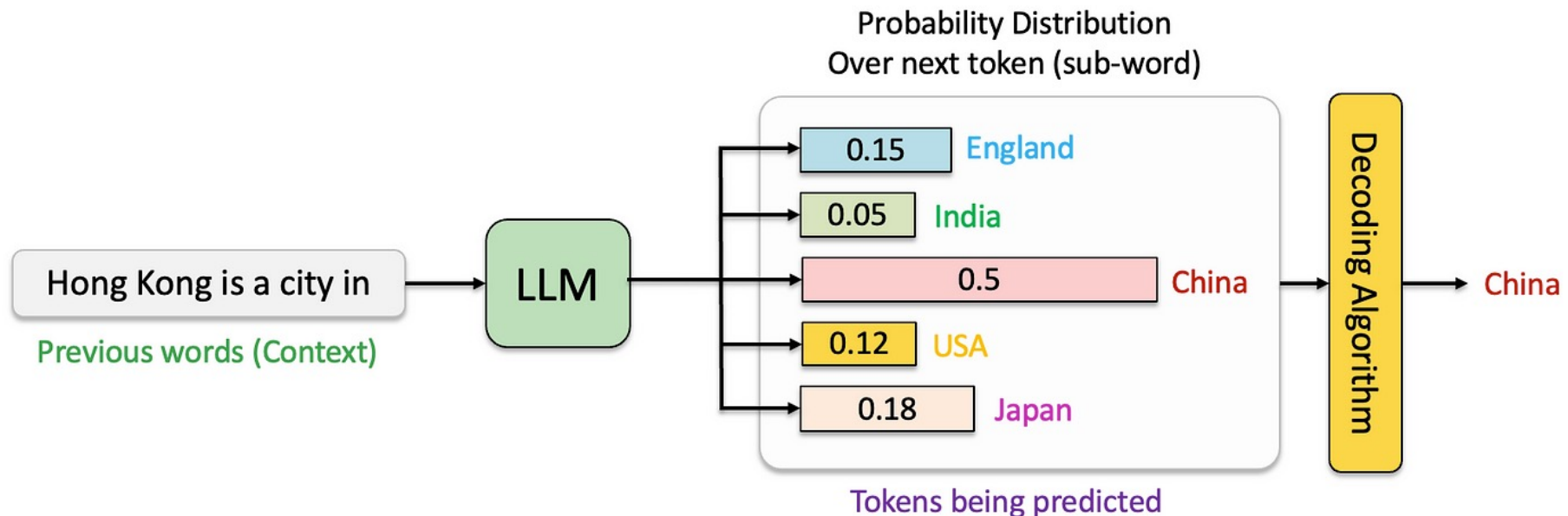
- Decoder-only Transformer



GPT (Generative Pre-trained Transformer) is the first decoder-only Transformer model

Overview

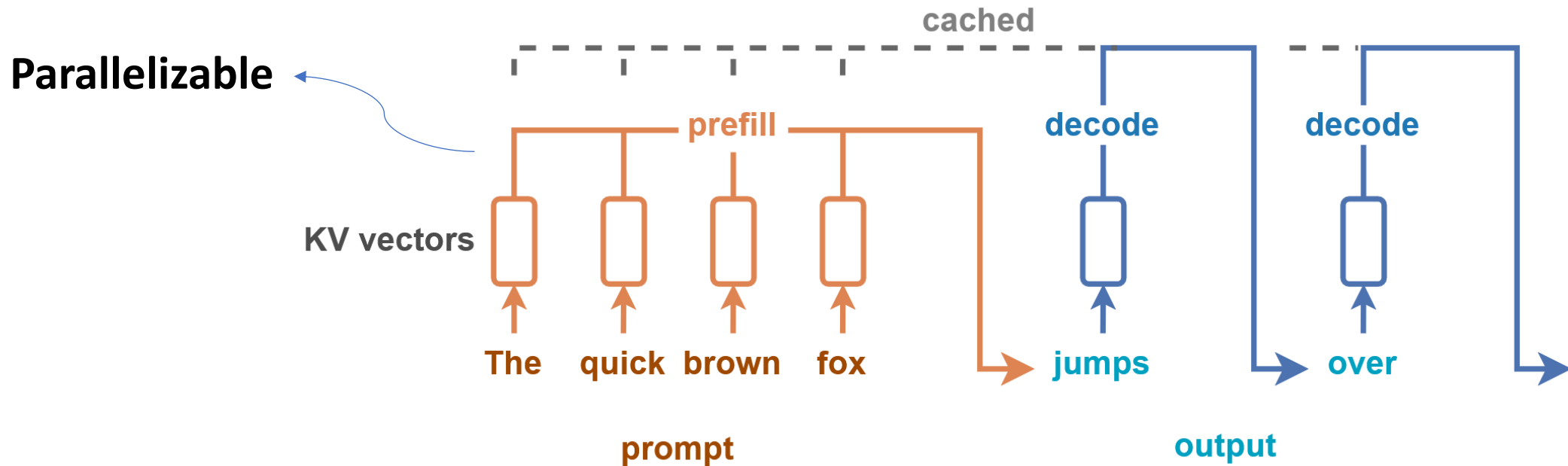
- Decoder-only Transformer
 - Generating a probabilistic distribution over possible next token, and a decoding algorithm is employed to select the actual output token



Next-token prediction, i.e. generating output tokens one by one

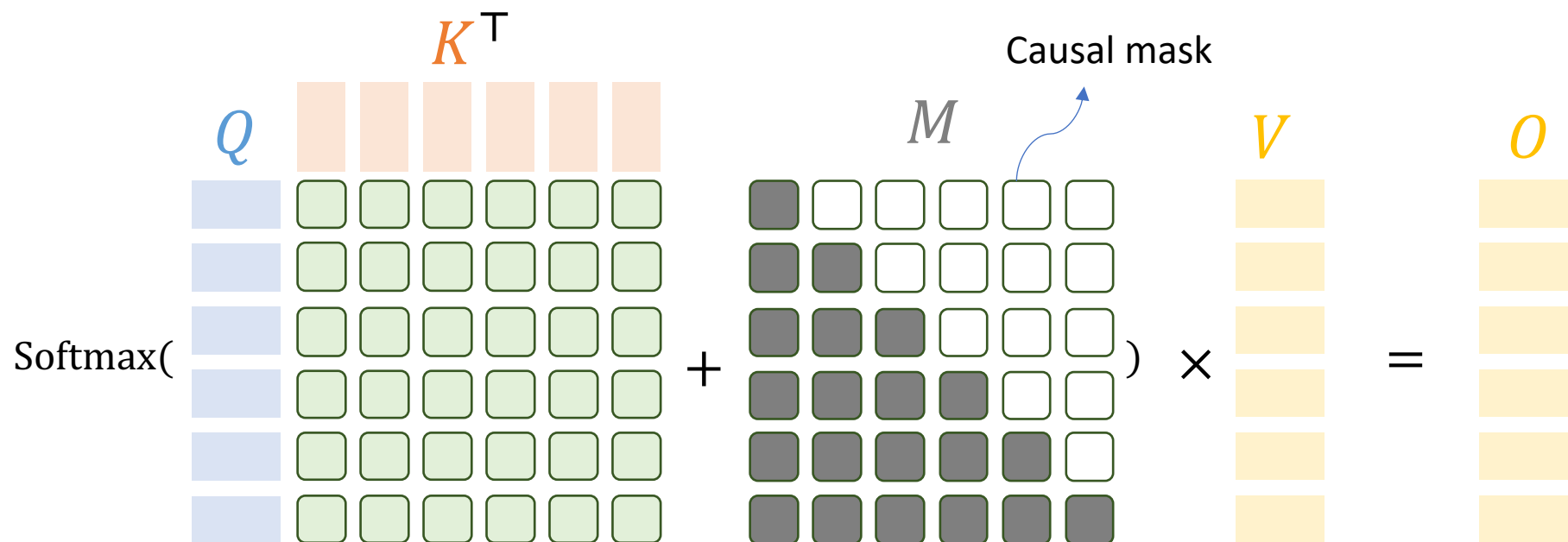
Overview

- Autoregressive Decoding
 - **“Prefill”** refers to the **initial parallel computation phase** where the model processes the entire input prompts for subsequent token-by-token generation



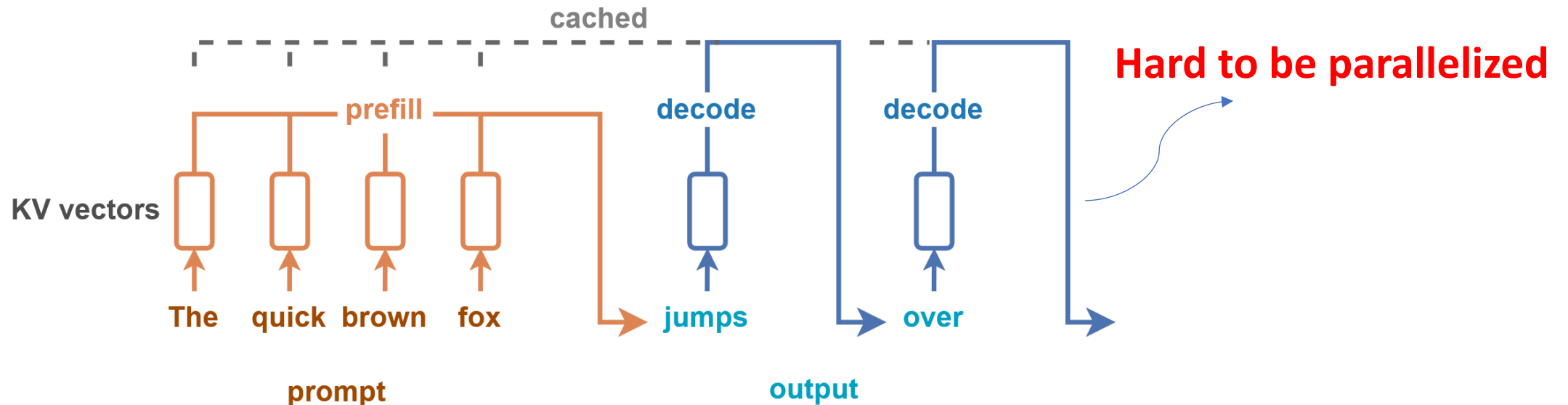
Overview

- Prefill: highly parallelizable
 - A small batch size can "saturate" GPU computation
 - Parallelization over batch size, header size, sequence length and thread-block tiling



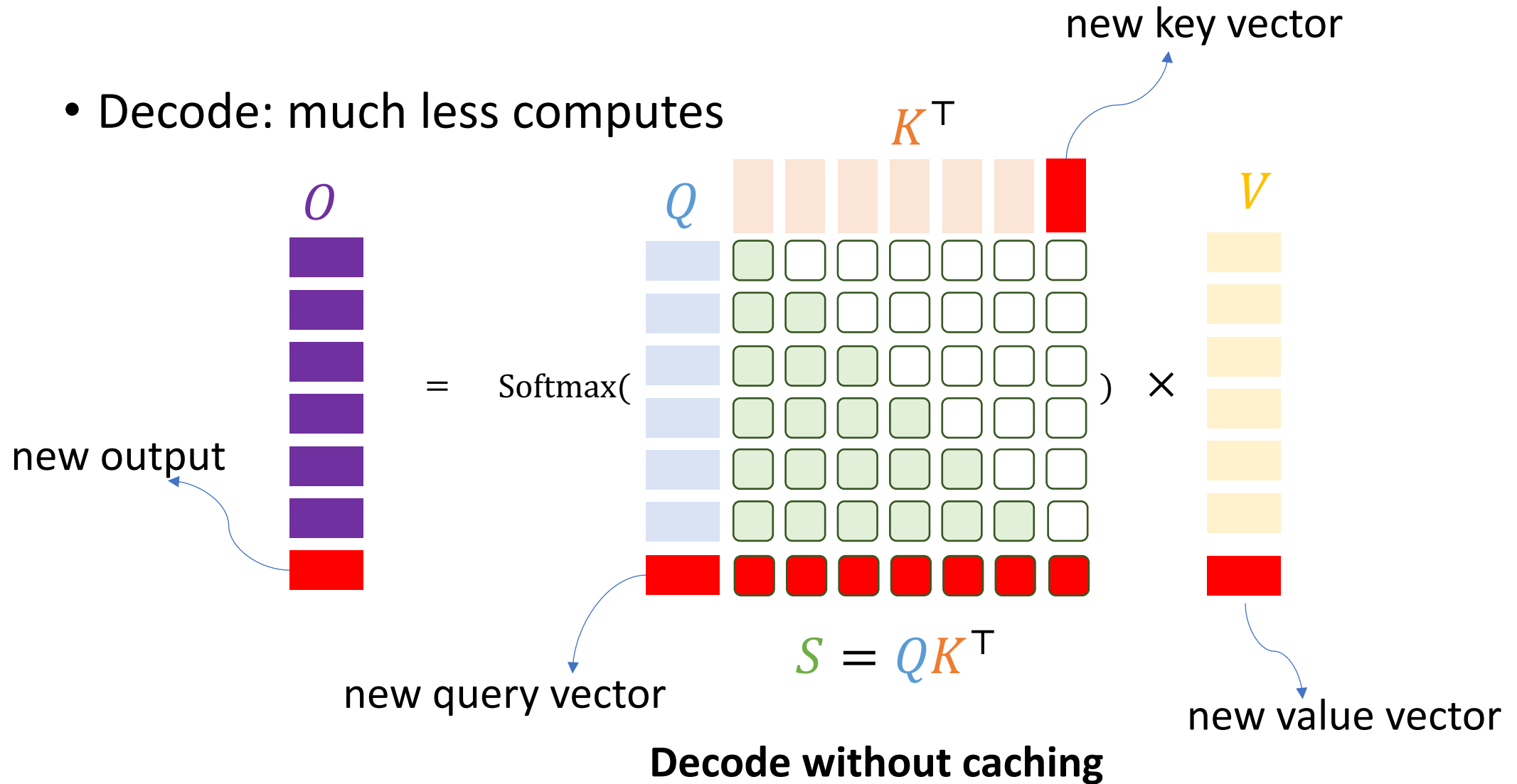
Overview

- Autoregressive Decoding
 - **“Decode”** refers to the **iterative process of generating output tokens** one at a time, where each new token is predicted based on the input prompt and all previously generated tokens

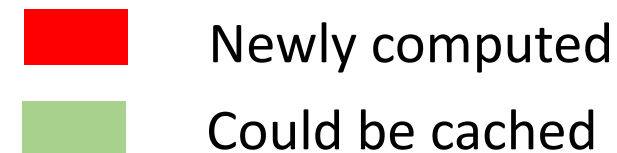


Overview

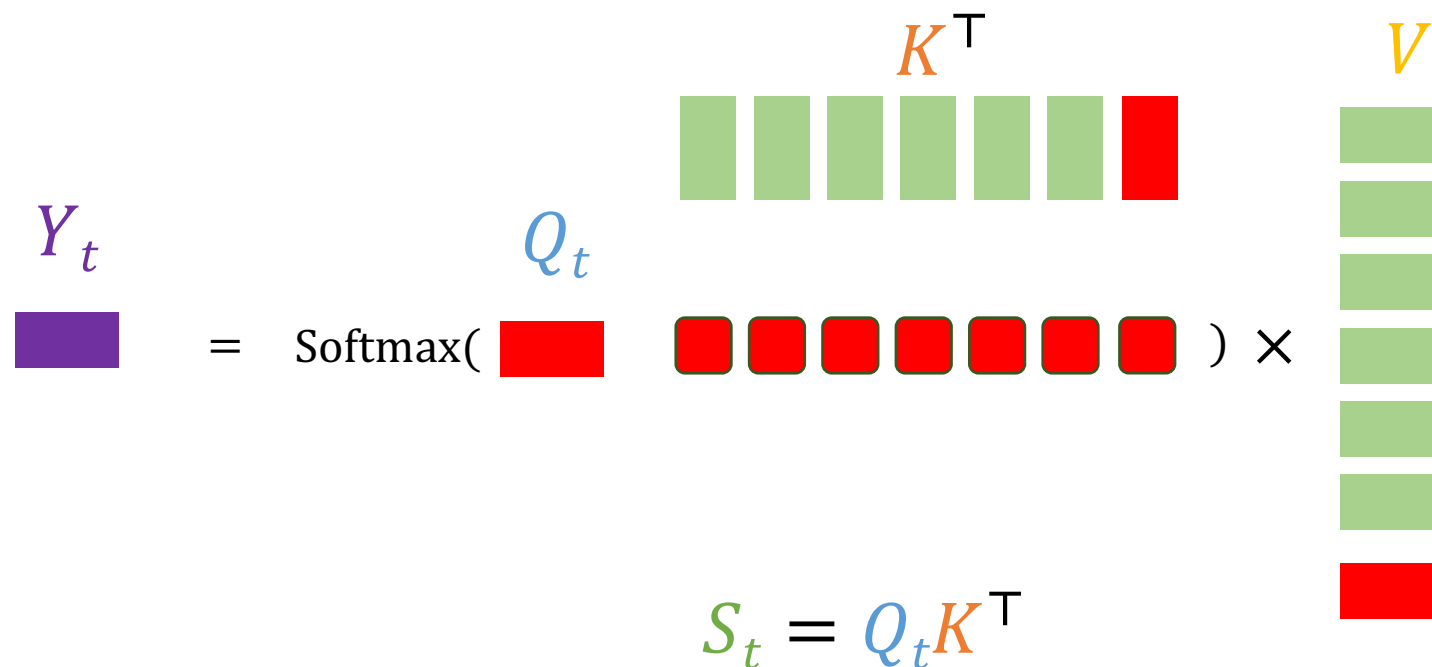
- Decode: much less computes



Overview



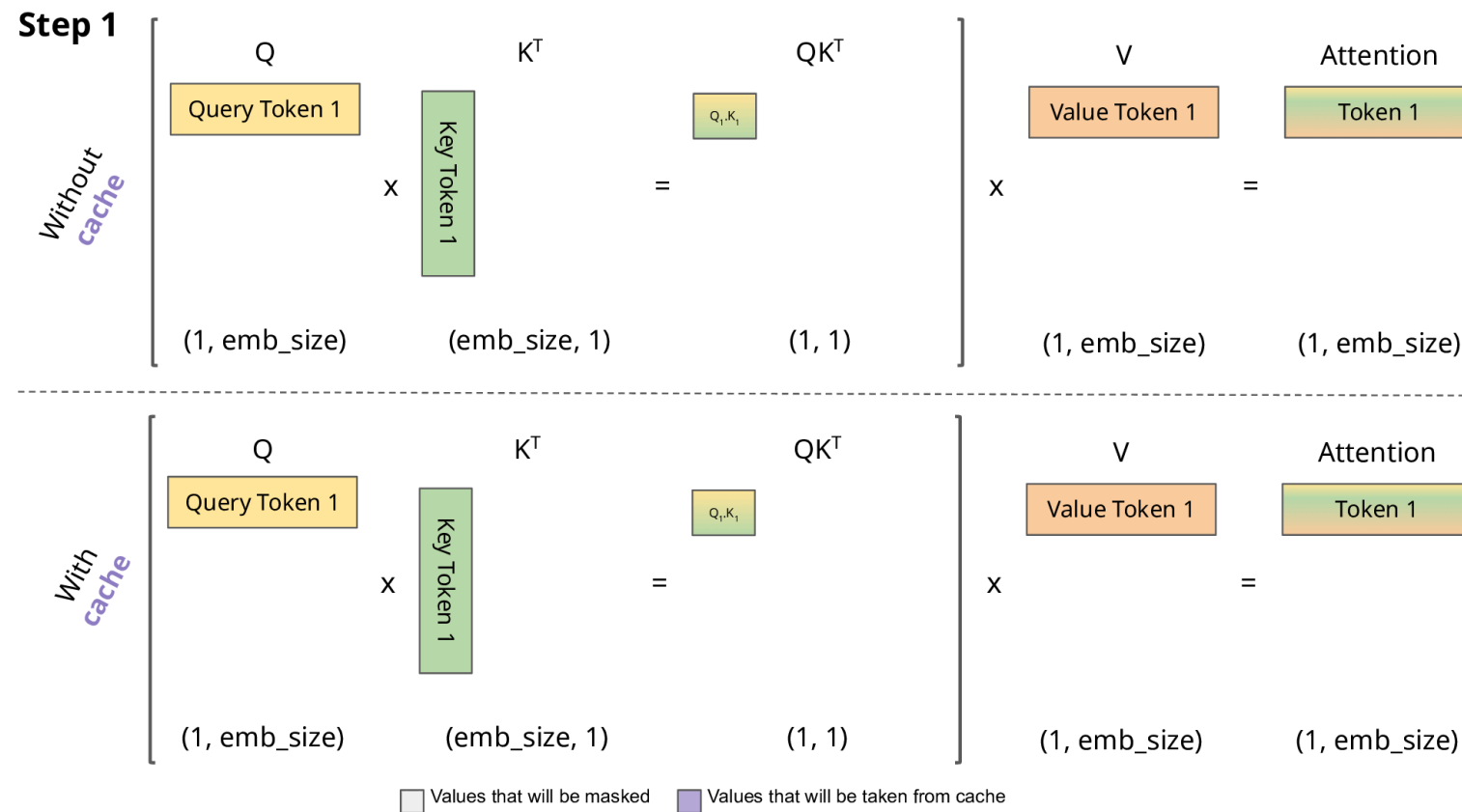
- Decode: much less computes
 - Why caching Key and Value, other than Query



Decode Compute output token one by one

Overview

- An animation



Overview

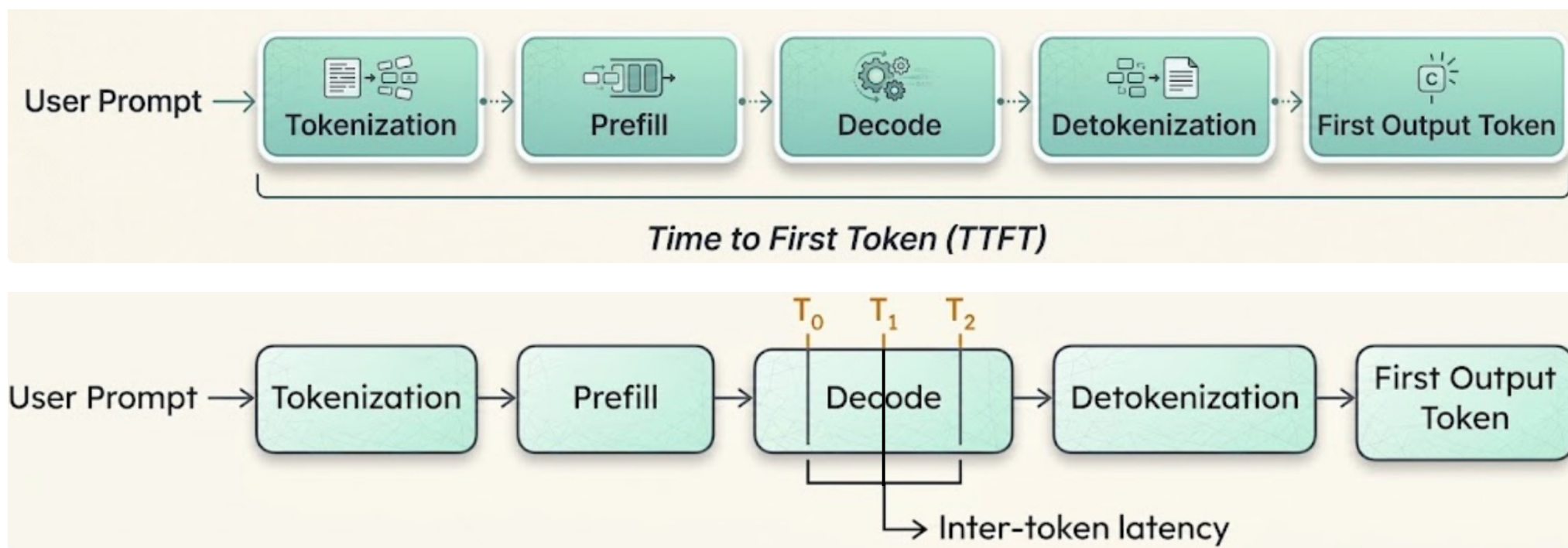
- Why Caching KV?
 - Reducing redundant computations for KEYS and VALUES
 - Increasing memory consumption

$$\begin{array}{ccccccc} 2 & \times & 2 & \times & 8096 & \times & 80 & \times & 64 & \times & 64 \\ \text{K/V} & & \text{Float16} & & \text{Sequence length} & & \text{\# of layers} & & \text{\# of heads} & & \text{dimension} \end{array}$$

~10.6 GB!

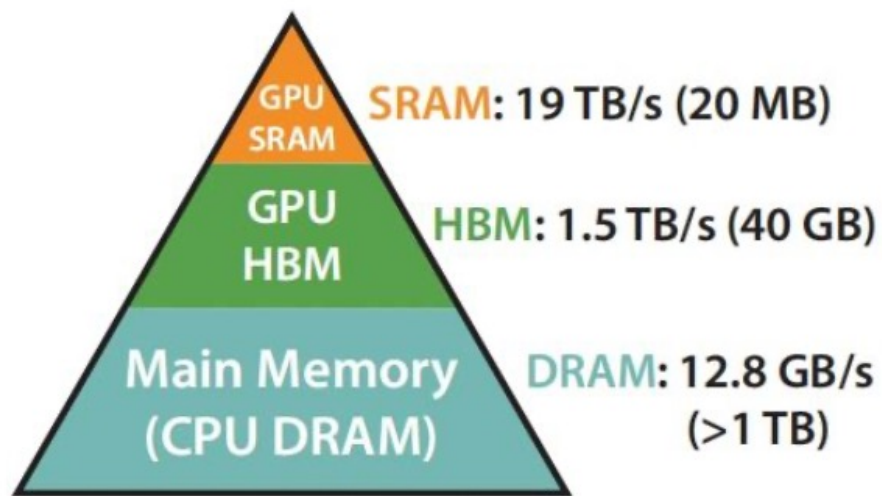
Overview

- Autoregressive Decoding
 - Token generation core metrics: **TTFT** and **TPOT** (time per output token)



Overview

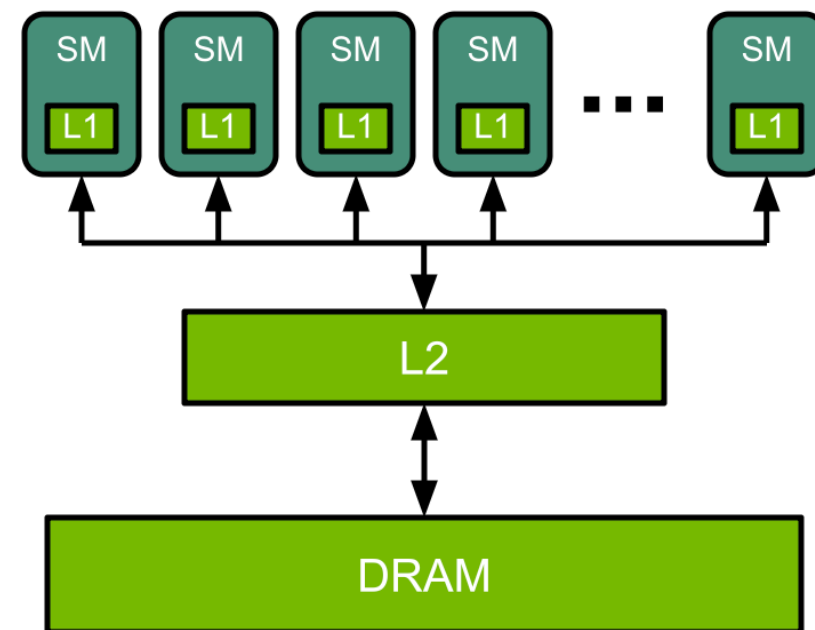
- Hierarchical GPU Memory
 - I/O throughput versus memory size



Memory Hierarchy with
Bandwidth & Memory Size

A macroscopic view of I/O tradeoff (V100)

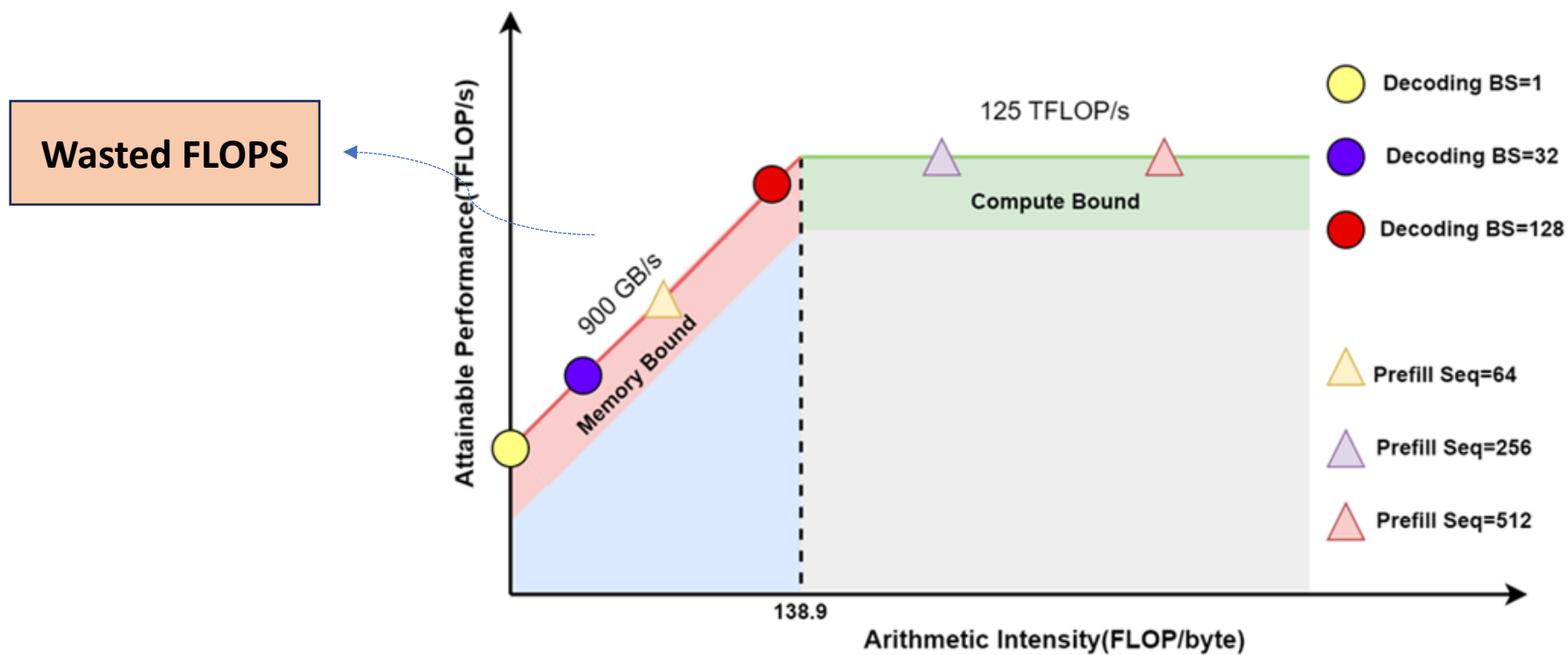
Insufficient to store data at
SRAM: load from HBM to
SRAM and write back to HBM



L1 instruction cache: 192KB per SM * 108 SM ~20MB
Data flow: DRAM/L2 Cache to/from L1 Cache, much
slower than computing

Overview

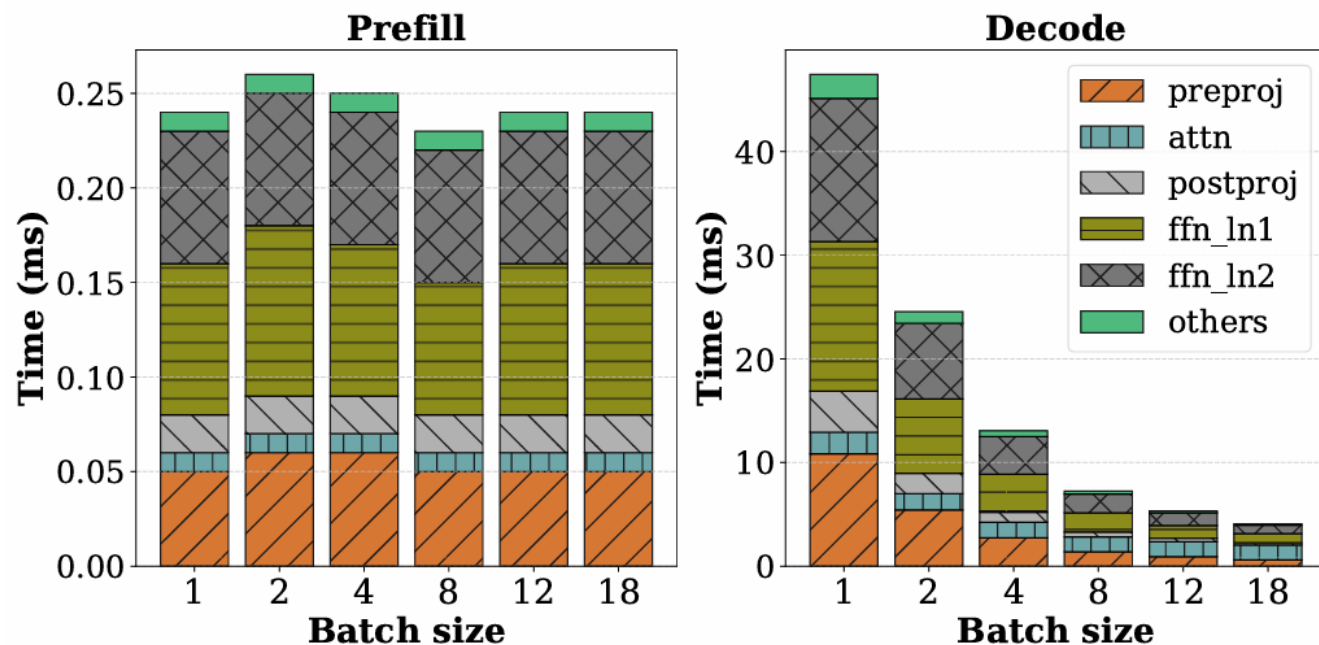
- Prefill and Decode
 - Prefill: Compute intensive with GEMM, Decode: memory I/O intensive with GEMV



Roofline model of NVIDIA V100 GPU

Overview

- Prefill and Decode
 - Prefill: Compute intensive, Decode: memory I/O intensive
 - Prefill saturates GPU compute even at batch size of 1
 - Decode under-utilizes GPU compute and costs as much as 200 times prefill for bs=1



Per-token prefill and decode time with different batch sizes
(sequence length = **1024**) for LLaMa-13B on A6000 GPU

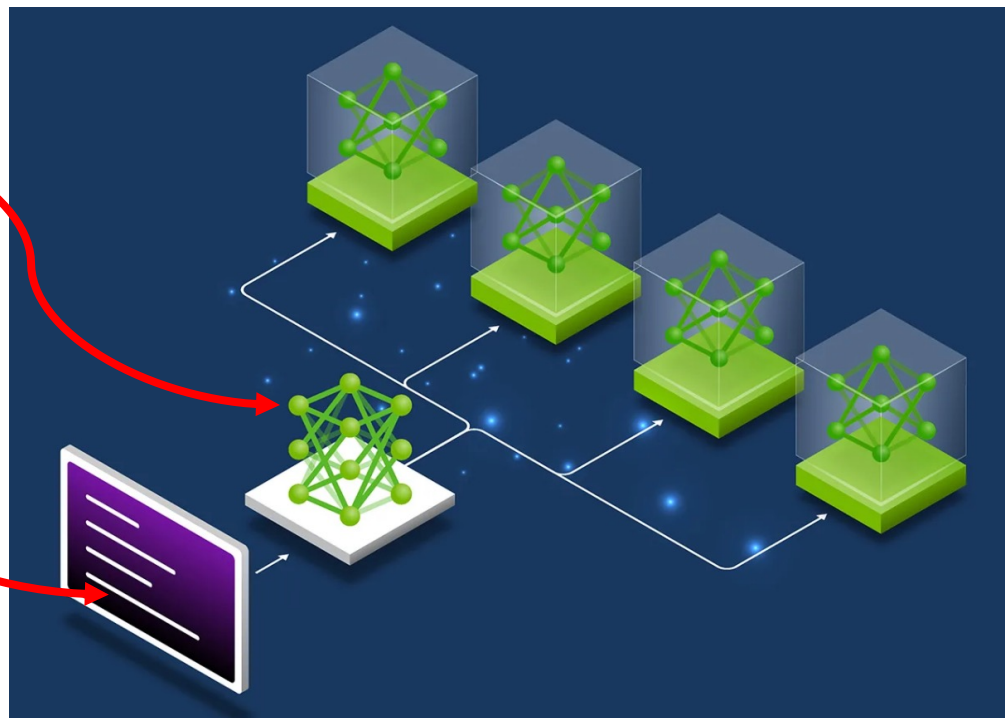
Overview

- Very Large-scale LLM Inference



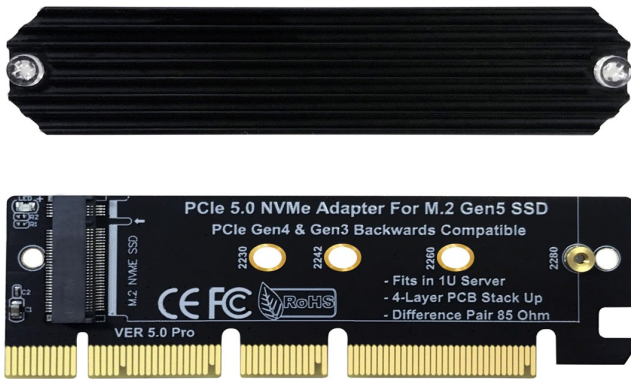
Full model size with 256 experts

Long context and high
request loads

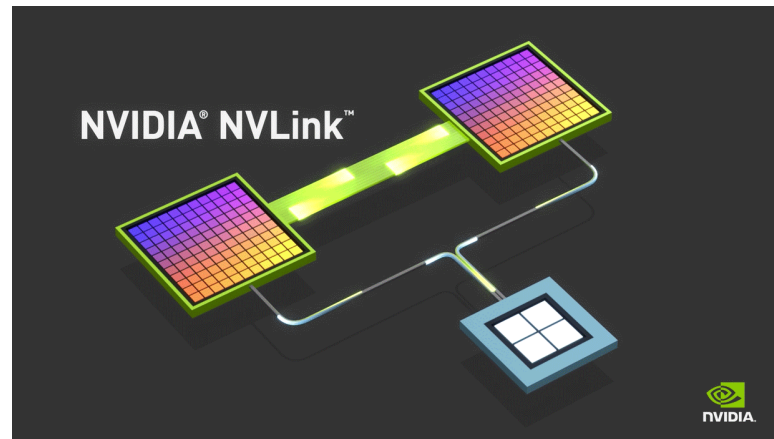


Overview

- Versatile communication medium
 - Transmission delay emerges



PCIe Link
e.g. CPU – GPU with
up to 128GB/s bandwidth



NVLink
e.g. GPU – GPU
in total 1.8TB/s bandwidth



RoCE
e.g. Machine – Machine
up to 800 Gbps

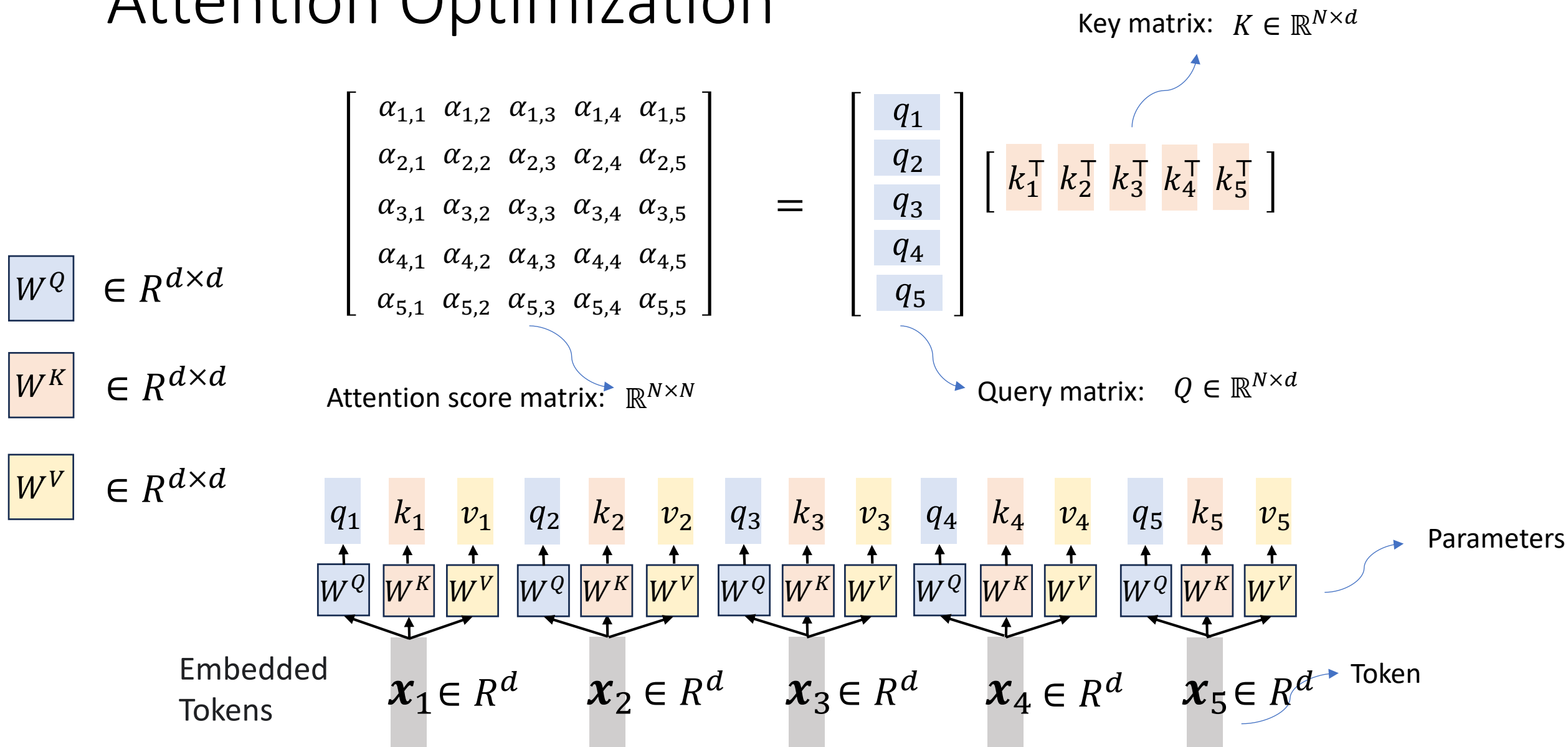
Overview

- Challenges of LLM Inference
 - New token generation paradigm
 - **Prefill** and **Decode**
 - Hierarchical memory
 - **Fast** I/O small size versus **slow** I/O large size
 - Heterogeneous bandwidth
 - **High** intra-machine bandwidth versus **low** inter-machine bandwidth
- To emphasize
 - many optimization methods, e.g. **attention optimization** can be employed for model **training** (e.g. sparse attention, linear attention, flash attention)

Inference Optimization: Outline

- Overview
- Attention Optimization
 - **Sparse Attention**
 - Linear Attention
 - Flash Attention
 - Continuous Batching
- Continuous Batching
- KV Cache Optimization
- Speculative Decoding
- Distributed Serving

Attention Optimization



Attention Optimization

- Real-world operations
 - Transferring both matrices from global memory to shared memory for computing, and write the result back to global memory progressively
 - Complexity (naïve method)
 - Computation: $O(N^2d)$
 - Considering multiplications and additions $\rightarrow$ computing time $T_C = \frac{2N^2d}{c}$
 - Communication: $O(N^2)$
 - Considering bidirectional I/O read/write $\rightarrow$ communication time $T_{I/O} = \frac{2*(N^2+2Nd)}{B}$
 - Global storage: $O(N^2)$
-
- (partially) overlapped**
- GPT3 with 10K tokens in FP16:
200MB per head per layer

Attention Optimization

- Real-world operations
 - Transferring both matrices from global memory to shared memory for

Computation, I/O and storage should all be optimized!

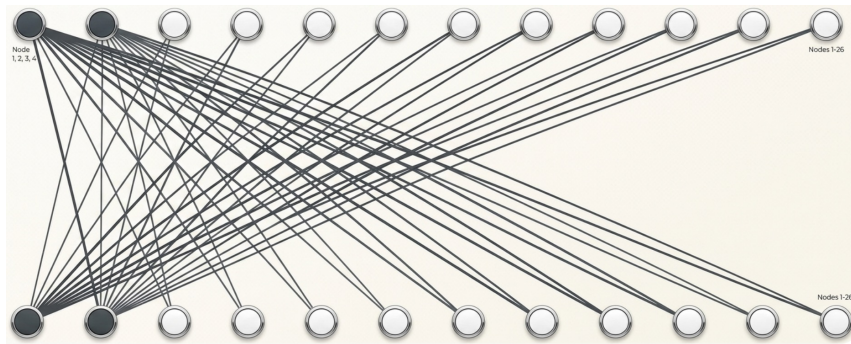
- Communication: $O(N^2)$
 - Considering bidirectional I/O read/write $\rightarrow$ communication time $T_{I/O} = \frac{2*(N^2+2Nd)}{B}$
- Global storage: $O(N^2)$

Sparse Attention

- Sparse attention
 - reduce computational and memory cost by only computing attention for a **subset of token pairs** instead of all pairs
 - evidence from machine learning community (e.g. Rewon Child, 2019)
- Sparse attention patterns
 - Position-based sparse attention: **rule-based or heuristic methods** to decide the positions where the interactions of these pair-wise tokens are important
 - Content-based sparse attention: tokens selectively attending only to other tokens that are **relevant based on their representations**

Position-based Sparse Attention

- Global attention
 - global nodes act as an information hub, allowing them to attend to every other node in the sequence



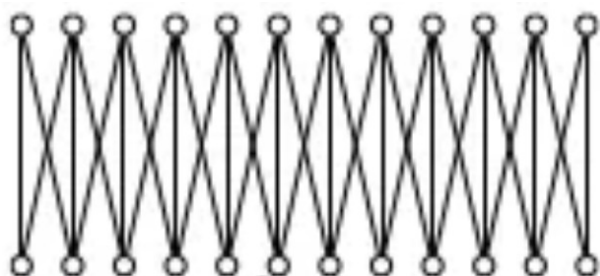
A bipartite graph illustration of QK^T computation

	Key							
	k_1	k_2	k_3	k_4	k_5	k_6	k_7	k_8
Query	q_1							
	q_2							
	q_3							
	q_4							
	q_5							
	q_6							
	q_7							
	q_8							

Neither computed nor stored in memory

Position-based Sparse Attention

- Band attention
 - often termed “local attention” or “sliding window attention”, in which a node’s attentions are confined to neighboring nodes in a local window

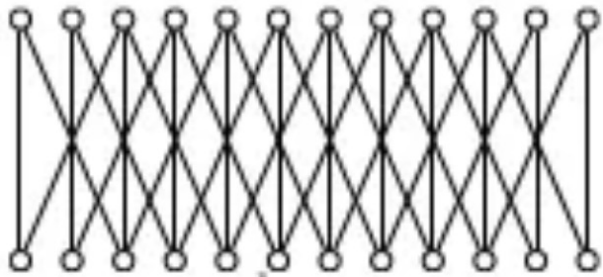


	Key							
	k_1	k_2	k_3	k_4	k_5	k_6	k_7	k_8
Query q_1								
q_2								
q_3								
q_4								
q_5								
q_6								
q_7								
q_8								

- Reducing complexity from $O(N^2)$ to $O(kN)$
- Limited receptive field (cannot capture long-range dependencies)
- Sacrificing context modeling ability: slow information propagation

Position-based Sparse Attention

- Dilated attention
 - using a dilated window with gaps of dilation equal to or greater than 1

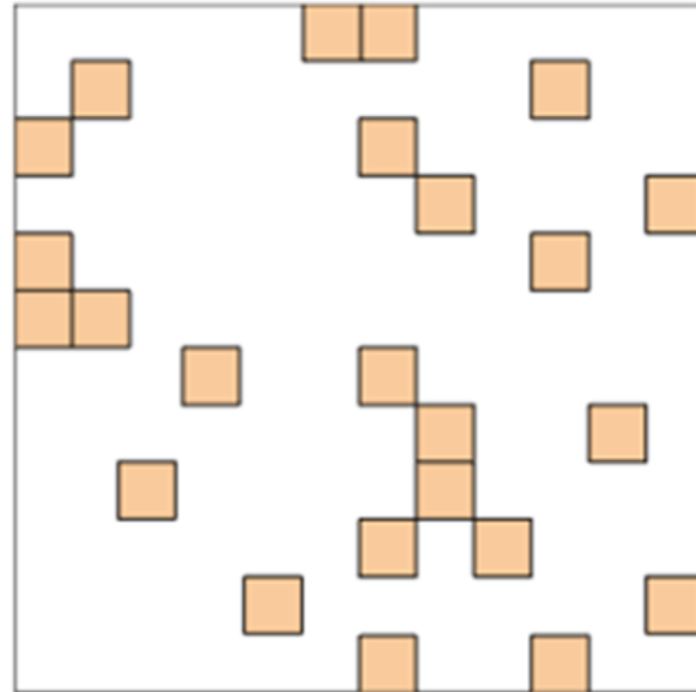
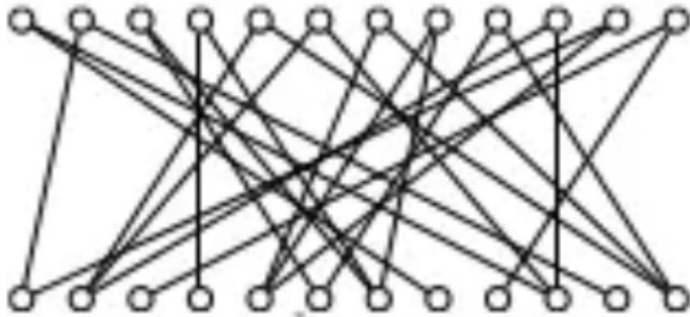


	Key							
	k_1	k_2	k_3	k_4	k_5	k_6	k_7	k_8
Query q_1								
q_2								
q_3								
q_4								
q_5								
q_6								
q_7								
q_8								

- Reducing complexity from $O(N^2)$ to $O(kN)$
- Missing fine-grained, short-range dependencies (failure to capture important neighbors in those gaps)

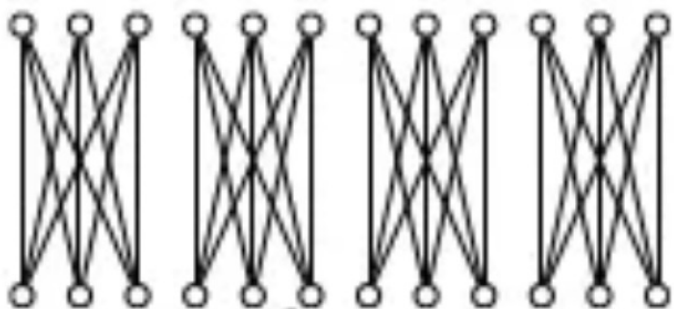
Position-based Sparse Attention

- Random attention
 - each query randomly samples a few keys for better capturing non-local interactions



Position-based Sparse Attention

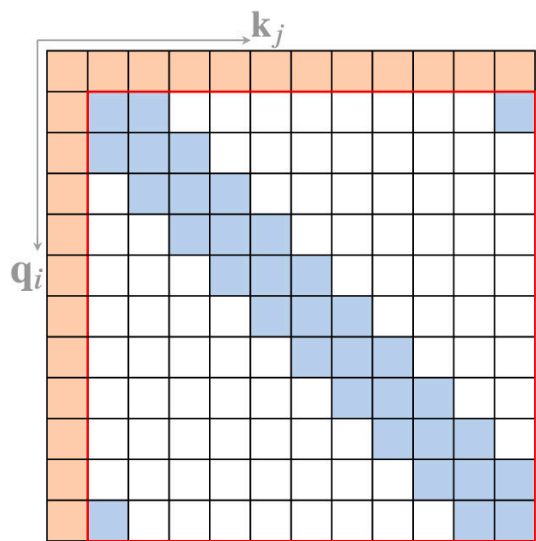
- Block attention
 - input sequence segmented into multiple non-intersection query blocks and each block assigned a local memory block



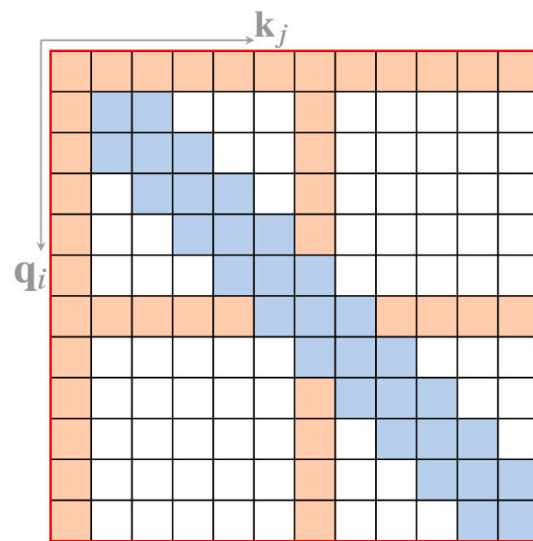
	Key							
	k_1	k_2	k_3	k_4	k_5	k_6	k_7	k_8
Query	q_1							
	q_2							
	q_3							
	q_4							
	q_5							
	q_6							
	q_7							
	q_8							

Position-based Sparse Attention

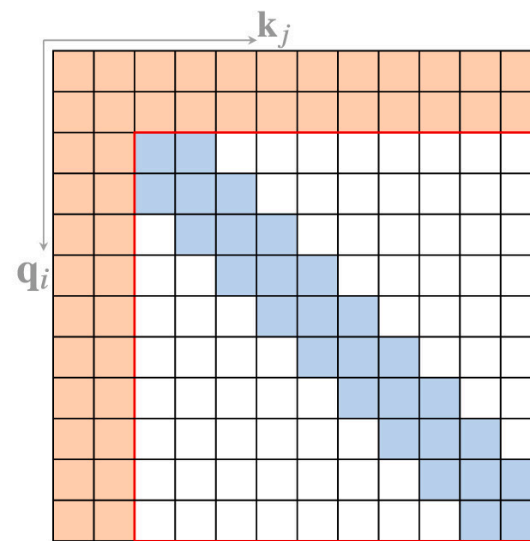
- COMPOUND attention



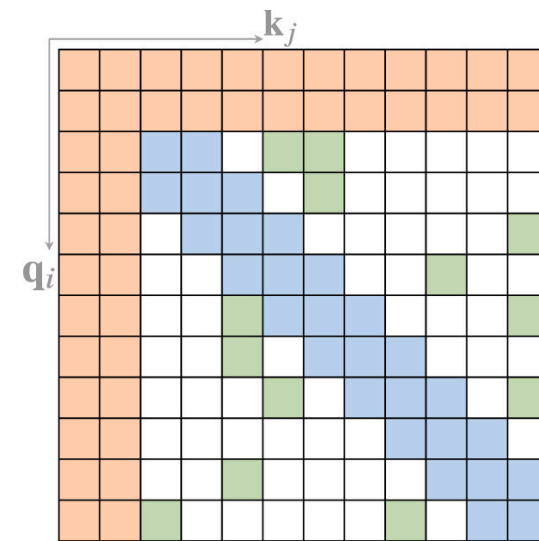
(a) Star-Transformer



(b) Longformer



(c) ETC

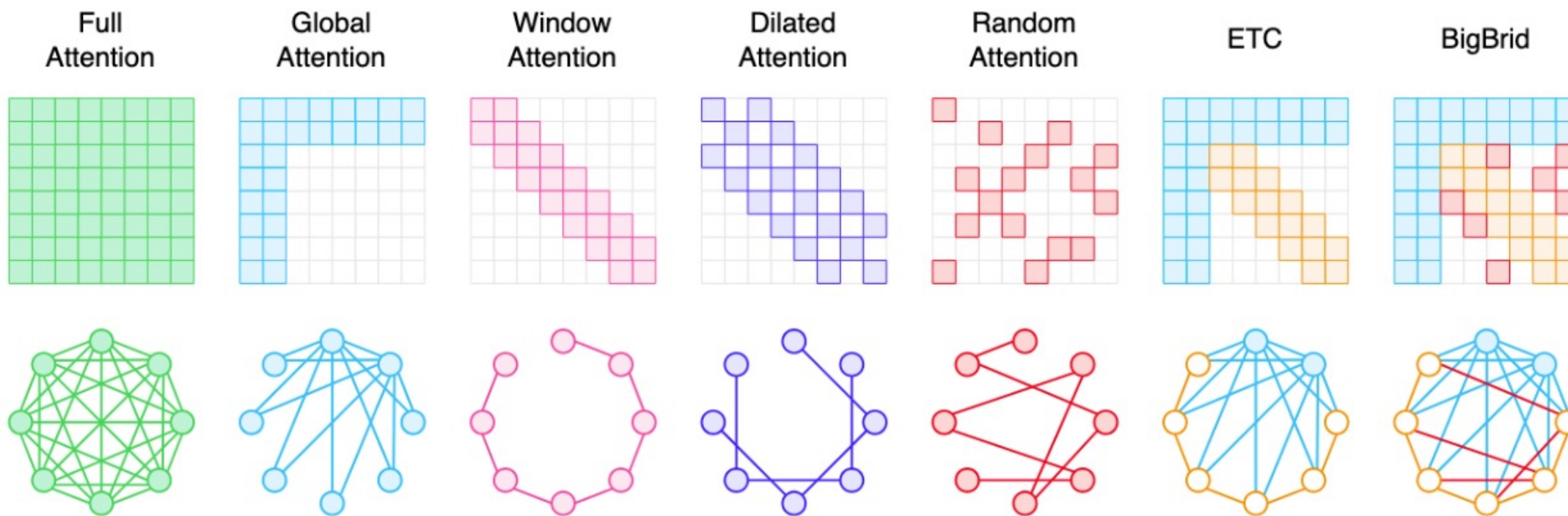


(d) BigBird

- BigBird : local + global + random yields dense-equivalent performance
- Increased implementation complexity, more hyperparameters to tune and potential redundancy

Position-based Sparse Attention

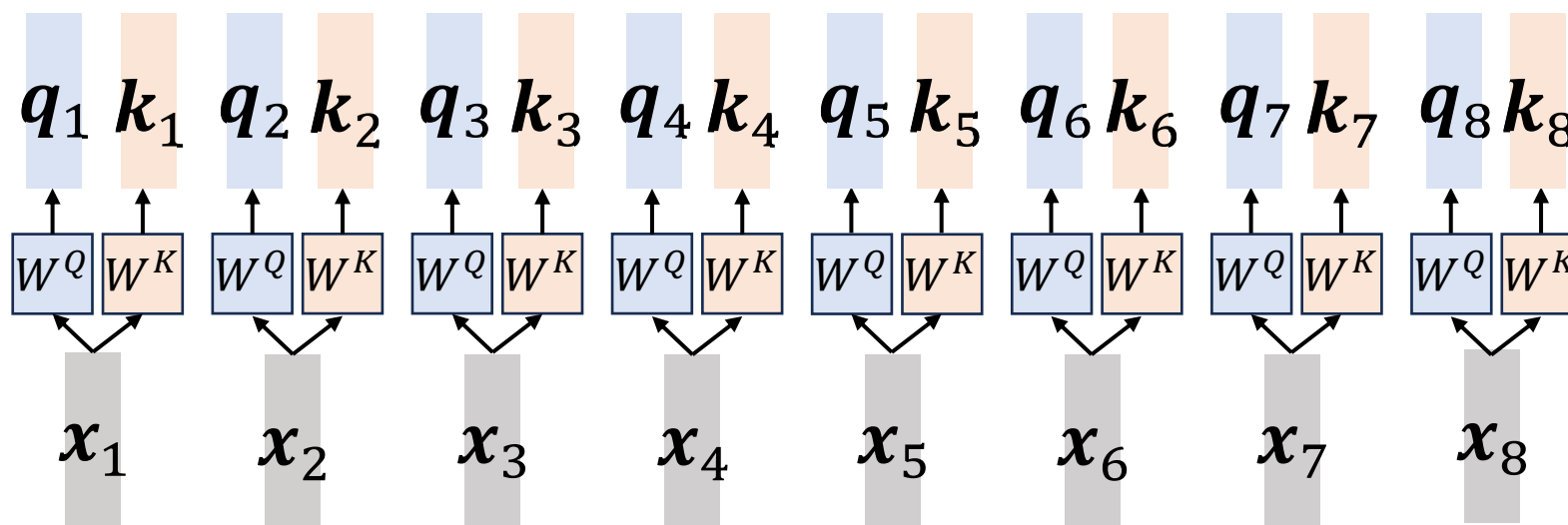
- Representative sparse attentions in a nutshell



Content-based Sparse Attention

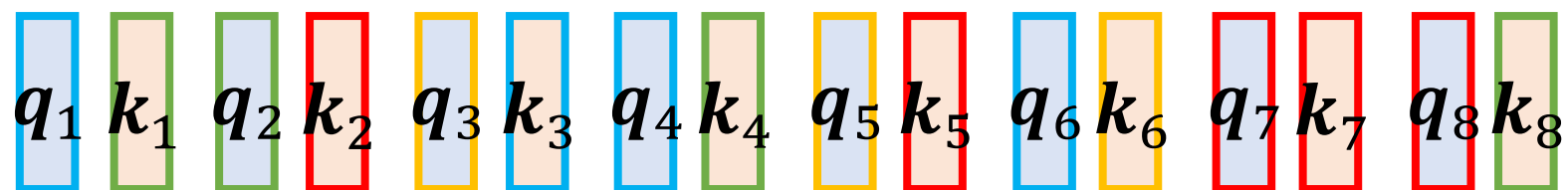
- Routing transformer
 - Key idea: a large inner product means that query and key are “similar”

Clustering (approximate & fast)

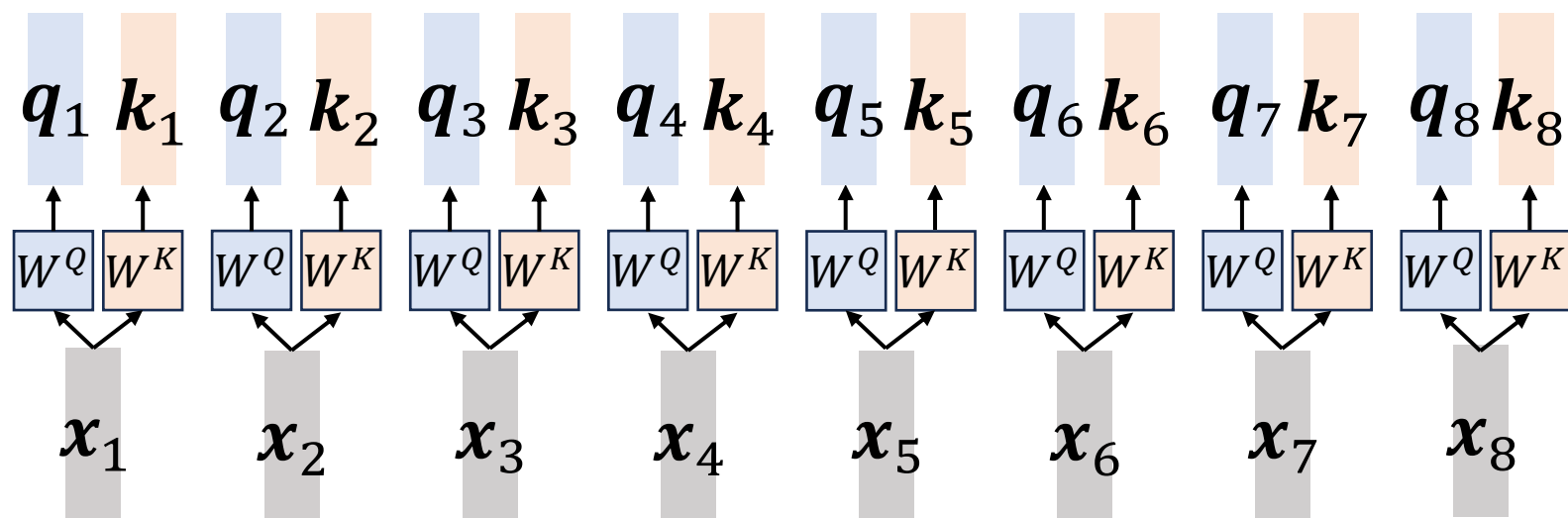


Content-based Sparse Attention

- Routing transformer

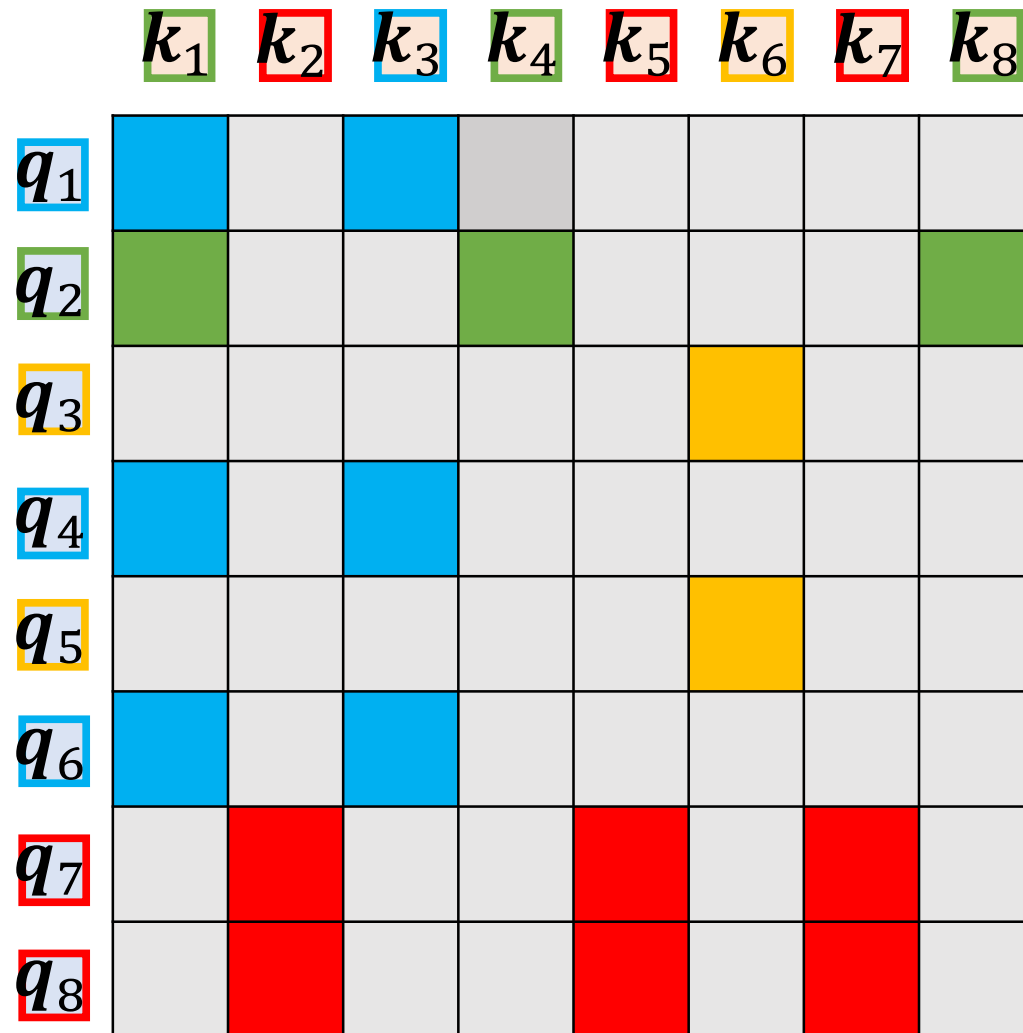


K-means Clustering (approximate & fast)



Content-based Sparse Attention

- Clusters
 - $\{q_1, q_4, q_6, k_1, k_3\}$
 - $\{q_2, k_1, k_4, k_8\}$
 - $\{q_3, q_5, k_6\}$
 - $\{q_7, q_8, k_2, k_5, k_7\}$
- Updating centroid
 - Exponentially weighted moving average (refer to the paper)

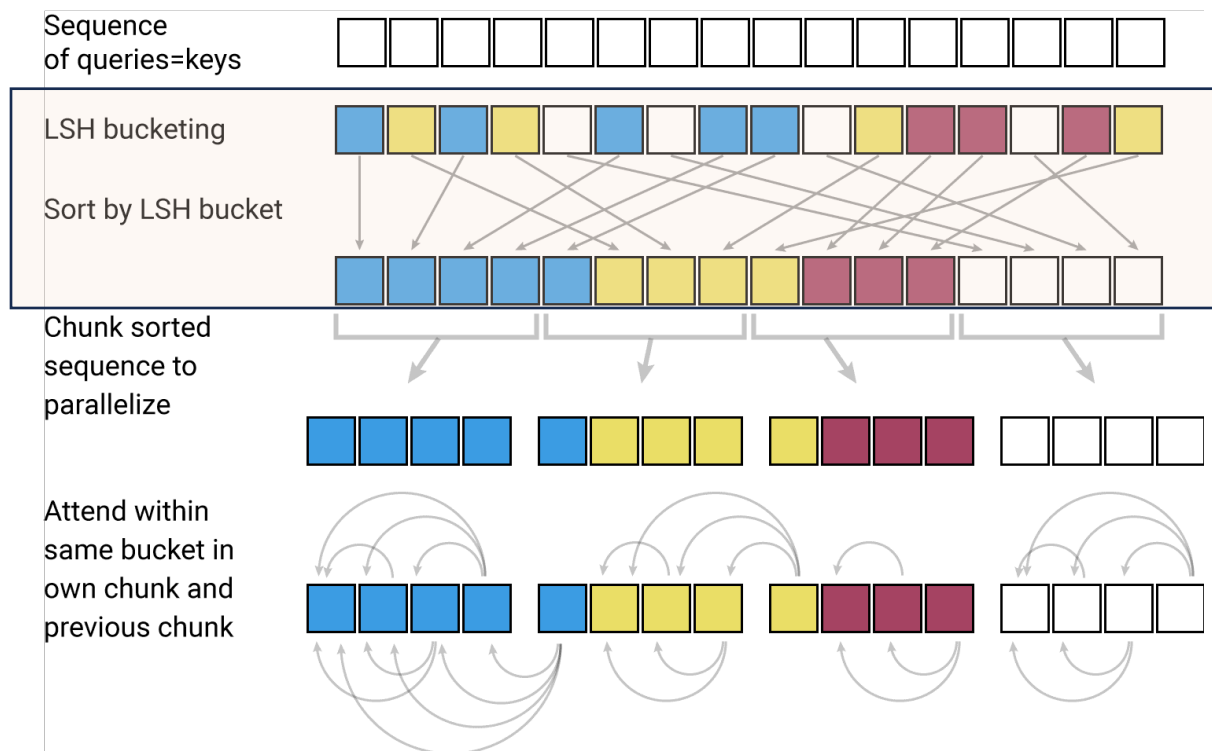


Content-based Sparse Attention

- Routing transformer: summary
 - attention scores in scaled dot-product rely on the similarity between Q and K
→ approximated by grouping them in a shared projection space
 - needs to be **double checked** by our students manually
 - each query only needs to attend to keys within its assigned cluster (typically $\sqrt{n}$ in size), reducing complexity to $O(n\sqrt{n})$ while preserving relevant long-range dependencies
 - cluster centroids are learned during training (e.g.), allowing the model to adaptively group based on content

Content-based Sparse Attention

- **Reformer**: bucketing keys and queries (using LSH), and only computing the attention scores for those inside the same bucket

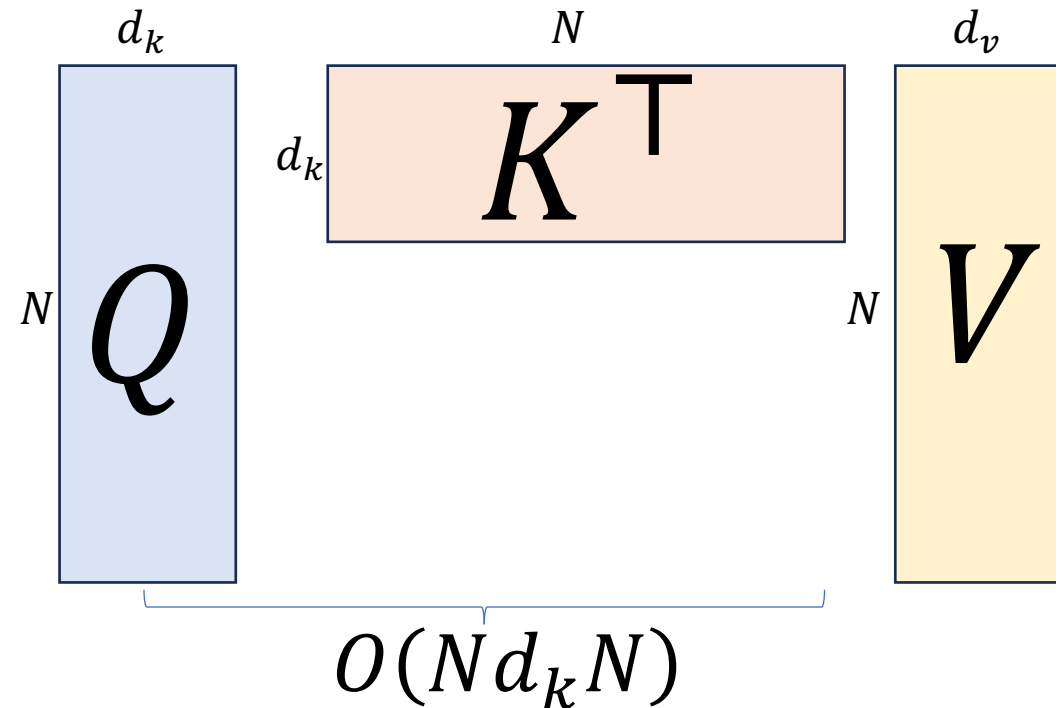


- Using **locality-sensitive hashing** (LSH) to select key-value pairs for each query
- Using Reversible Transformer to reduce memory usage during training
- Similar items (queries and keys) falling in the same bucket with high probability
- Processing sequences of length **64K tokens** or more efficiently
- Possible enhancement?

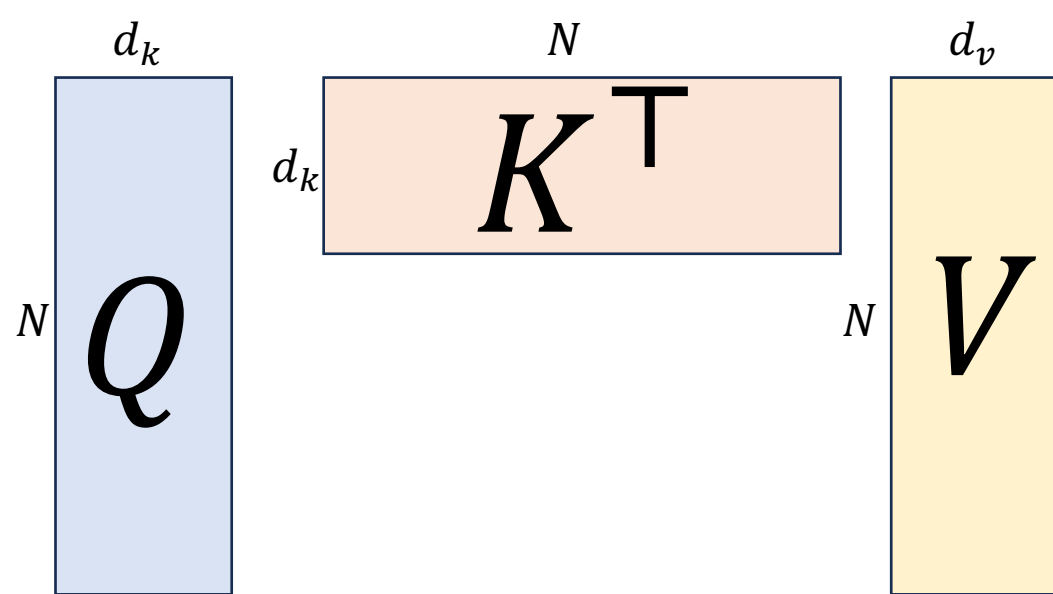
Inference Optimization: Outline

- Overview
- Attention Optimization
 - Sparse Attention
 - **Linear Attention**
 - Flash Attention
- Continuous Batching
- KV Cache Optimization
- Speculative Decoding
- Distributed Serving

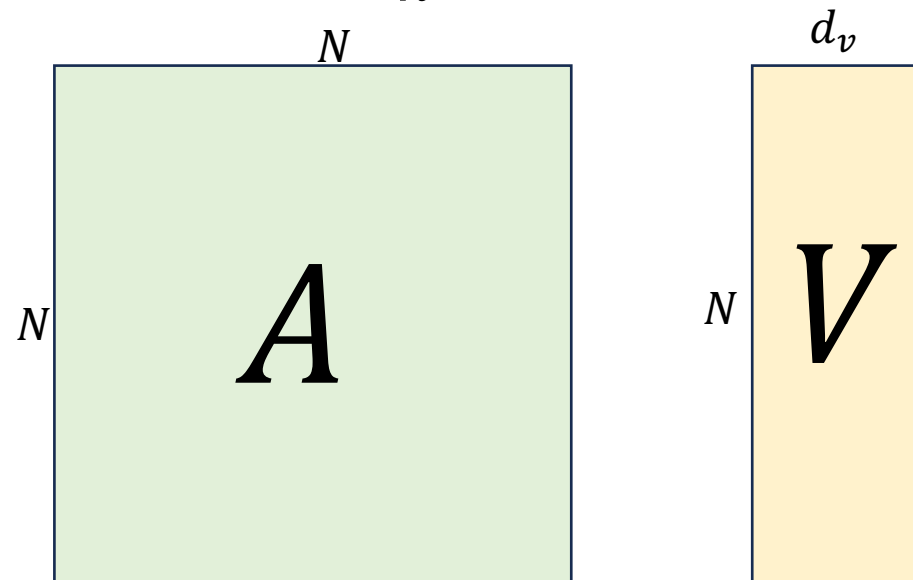
- Before softmax
 - Attention scores can be any real number - positive, negative, or zero, and their magnitudes vary greatly
- After softmax
 - Normalization transforms attention scores into a probability distribution
 - Exponential function has a powerful effect: it magnifies larger differences between the input scores



“optimal parenthesization” via dynamic programming
 (N stands for # of tokens, d_k stands for hidden dimension)



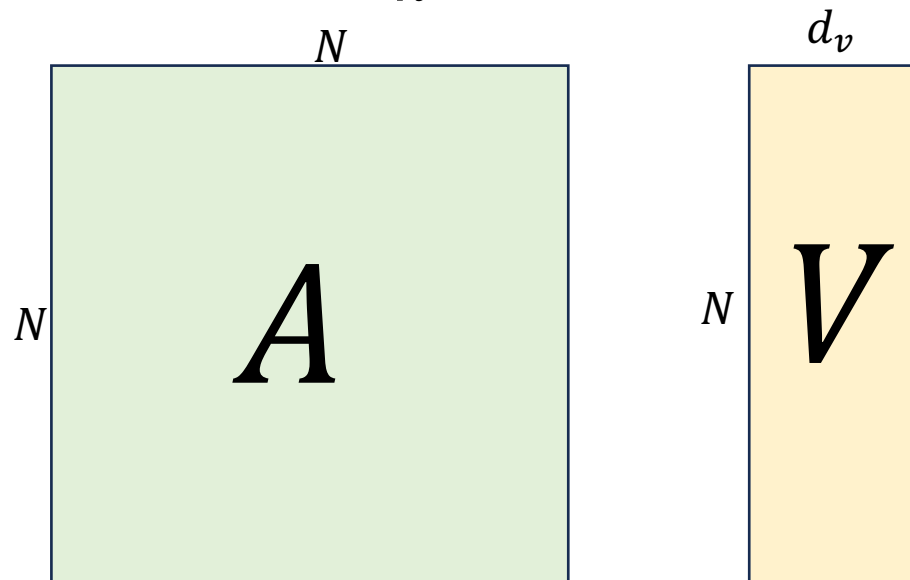
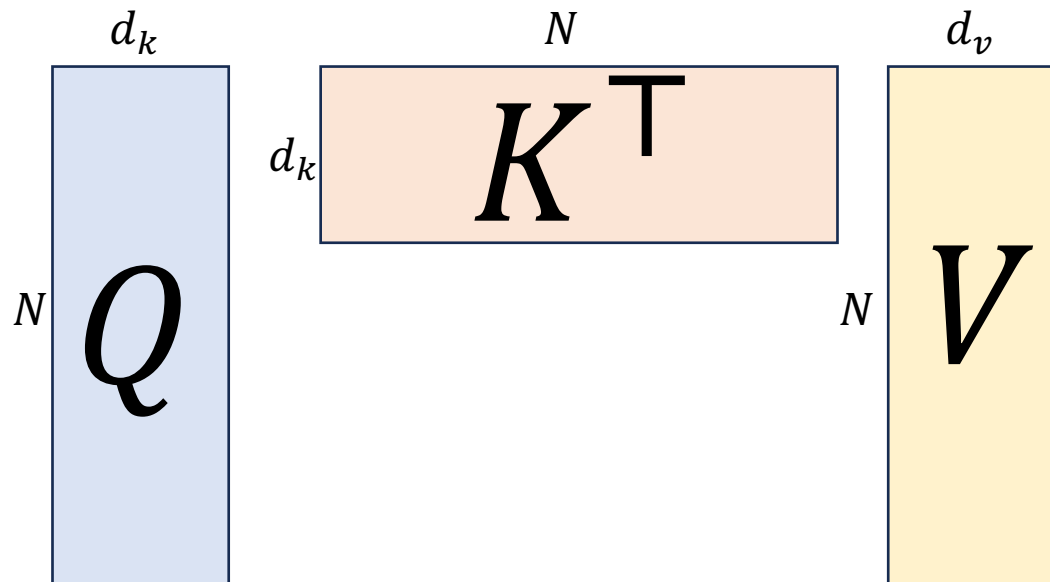
$$O(N d_k N)$$



$$O(N N d_v)$$

Original

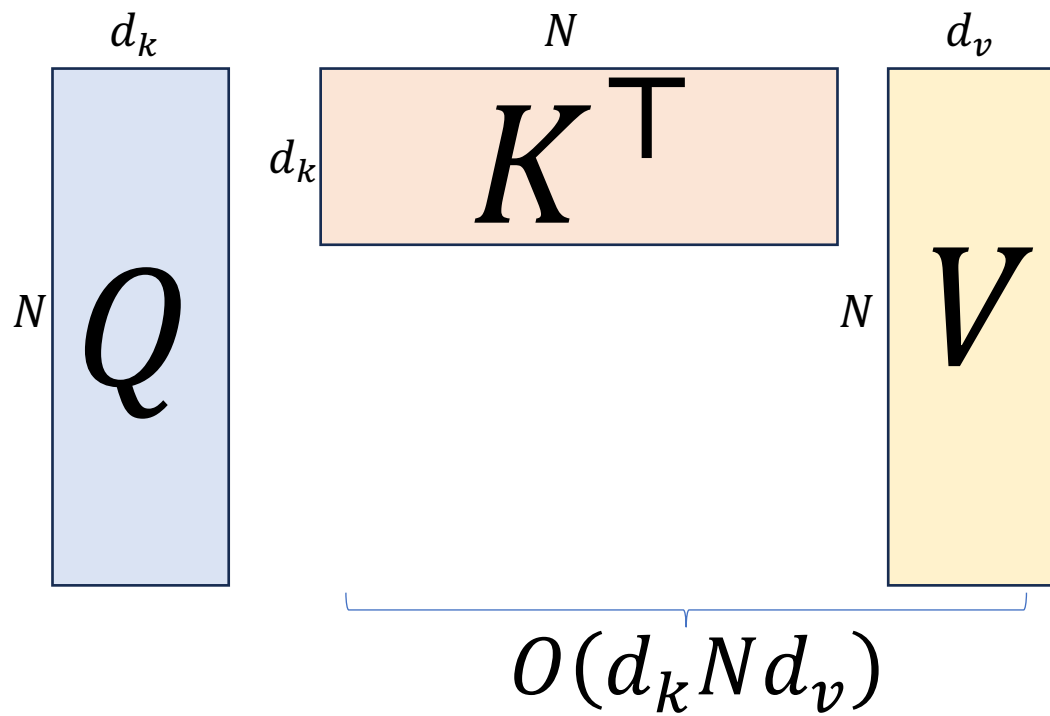
$$O(N^2(d_v + d_k))$$



$$O(N N d_v)$$

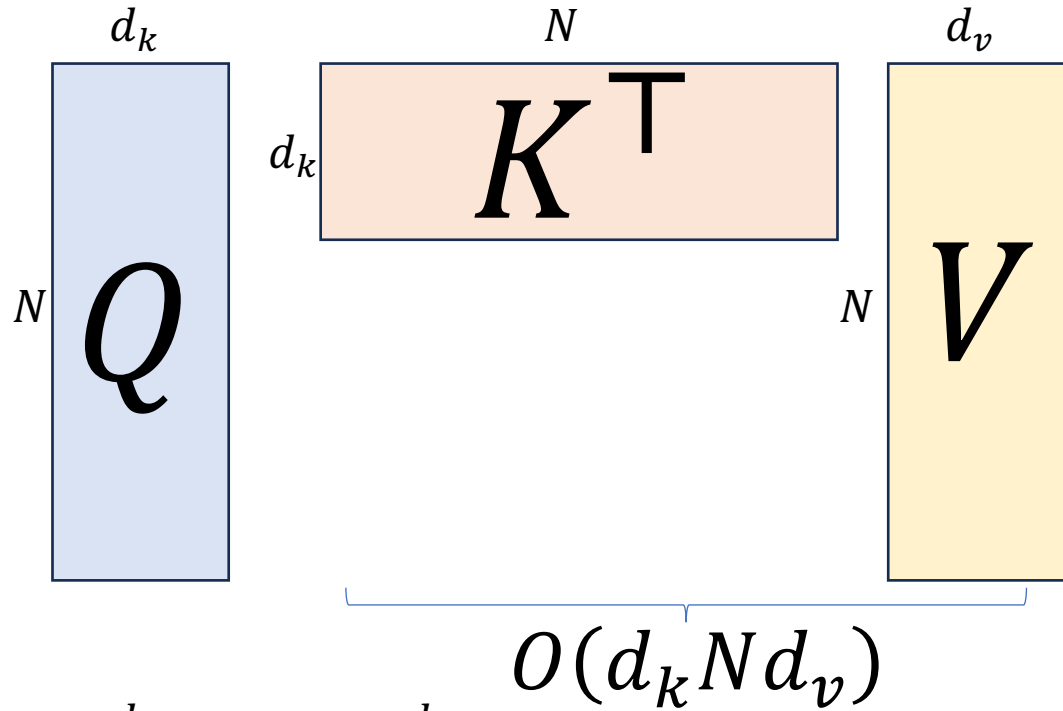
Original

$$O(N^2(d_v + d_k))$$



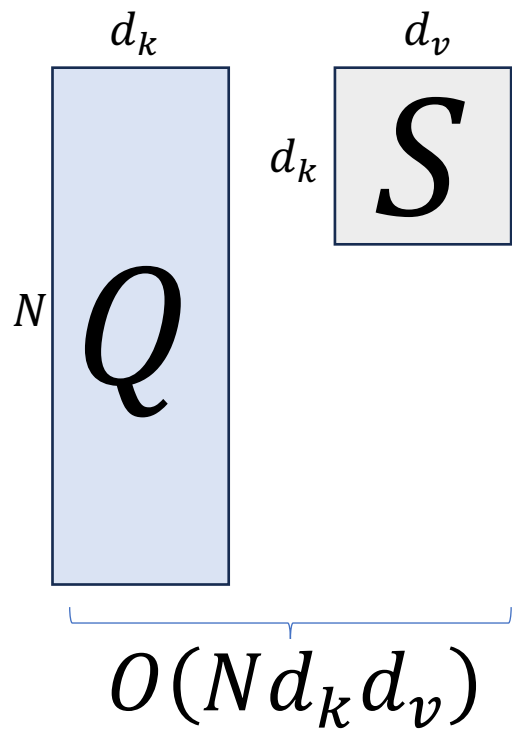
Original

$$O(N^2(d_v + d_k))$$



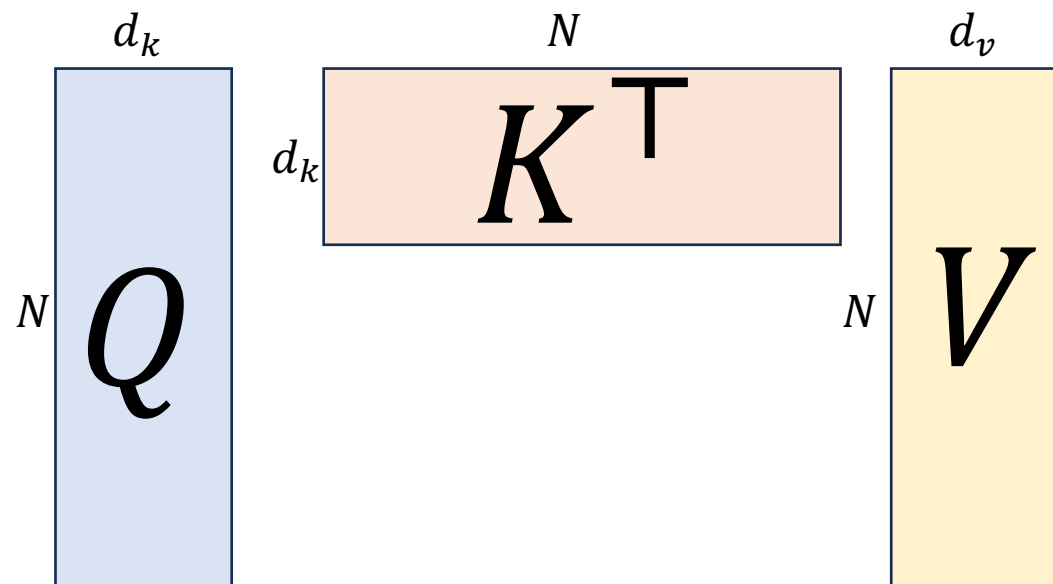
New

$$O(N d_k d_v)$$



Letting $N = 2048$
 $d_k = d_v = 64$

Original

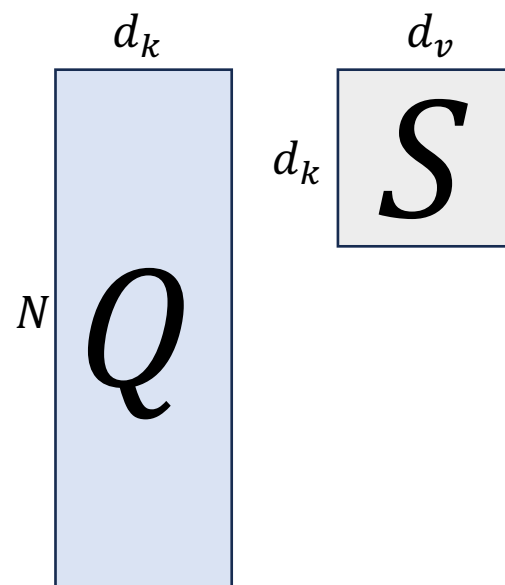


$$O(N^2(d_v + d_k)) = 536,870,912$$

$$O(d_k N d_v)$$



1.56%!



New

$$O(N d_k d_v) = 8,388,608$$

$$O(N d_k d_v)$$

Obstacle

- Softmax prevents this reordered GEMM

$$\text{Softmax} \left(\frac{1}{\sqrt{d_k}} \begin{matrix} d_k \\ N \\ Q \end{matrix} \begin{matrix} N \\ d_k \\ K^T \end{matrix} \right) \begin{matrix} d_v \\ N \\ V \end{matrix}$$

Linear Attention

- Computing Softmax (Katharopoulos et al., 2020)

$$o_1 = \sum_{i=1}^N \alpha_{1,i} v_i = \sum_{i=1}^N \frac{\exp(q_1 \cdot k_i)}{\sum_{j=1}^N \exp(q_1 \cdot k_j)} v_i$$

- Substituting $\exp$ by

$$\exp(q \cdot k) \approx \phi(q) \cdot \phi(k)$$

- there exists

$$o_1 \approx \sum_{i=1}^N \frac{\phi(q_1) \cdot \phi(k_i)}{\sum_{j=1}^N \phi(q_1) \cdot \phi(k_j)} v_i$$

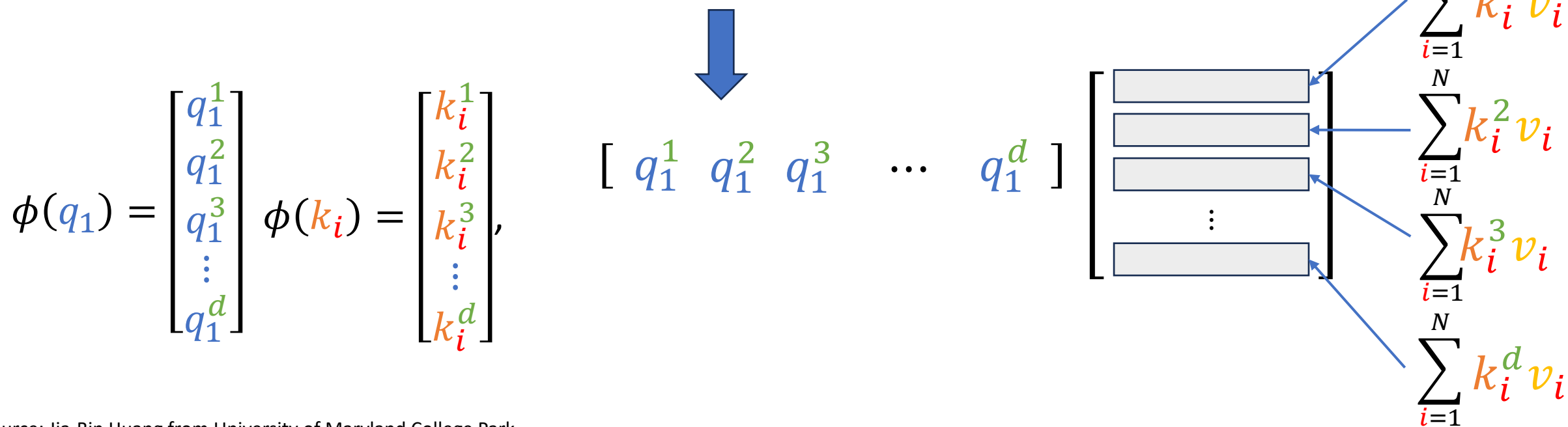
Kernel function

Linear Attention

- Overall Output: complexity order $O(N)$

$$O \approx \phi(Q) \cdot (\phi(K) V), \quad O, Q, K, V \in \mathbb{R}^{N \times d}$$

- An example: denominator of o_1



Linear Attention

- Iterative computation to further reduce complexity

$$o_1 \approx \sum_{i=1}^N \frac{\phi(q_1) \cdot \phi(k_i)}{\sum_{j=1}^N \phi(q_1) \cdot \phi(k_j)} v_i$$

- where

$$\sum_{i=1}^N [\phi(q_1) \cdot \phi(k_i)] v_i = \phi(q_1)^\top \left(\sum_{i=1}^N k_i v_i^\top \right) \Rightarrow$$

$$S_t = \sum_{i=1}^t k_i v_i^\top \in R^{d \times d}$$

$$S_t = S_{t-1} + k_t v_t^\top$$

$$o_t = \phi(q_t)^\top S_t$$

Linear Attention

- Today's Linear Attention
 - Baby linear attention: Linear Transformer, SANA, CHELA, LightningAttention, etc.
 - **Efficient** in computation, I/O, storage
 - Performance loss compared with ***Softmax***
 - More advanced linear attention:
 - Delta rule
 - Gamma forget
 - Gated attention
 - Qwen3-Next and Kimi-linear, and many others

Inference Optimization: Outline

- Overview
- Attention Computation Optimization
 - Sparse Attention
 - Linear Attention
 - **Flash Attention**
 - Continuous Batching
- KV Cache Optimization
- Speculative Decoding
- Distributed Serving

Flash Attention

- Prior works
 - Reducing # of scaled dot-product, potentially sacrificing attention performance
 - Compute is fast while reading K and V , and writing S back the results are slow
- Challenges
 - I/O cost of loading/storing Q , K and V are non-trivial

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^\top$, write $\mathbf{S}$ to HBM.
- 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
- 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
- 4: Return $\mathbf{O}$.

Memory Load/Read

Memory Write/Store

Flash Attention

- From Softmax to Safe Softmax

Standard Softmax

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$



Safe Softmax

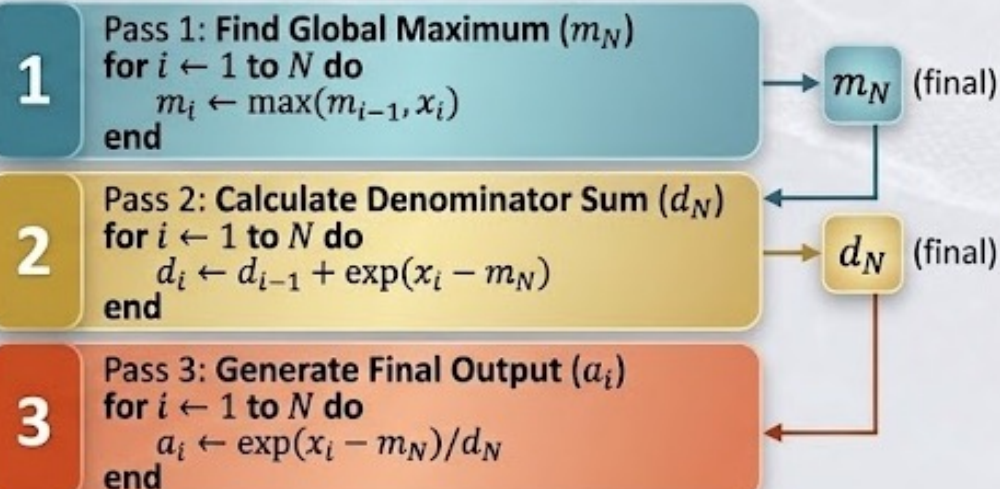
$$\text{softmax}(x_i) = \frac{e^{x_i - \max(x)}}{\sum_{j=1}^n e^{x_j - \max(x)}}$$

- **Online Safe Softmax:** not enough space to store entire x

- Alg1: three-pass algorithm

Definitions & States

Variable	Meaning	Explanation
m_i	$\max_{j=1}^i \{x_j\}$	Running max, $m_0 = -\infty$
d_i	$\sum_{j=1}^i e^{x_j - m_N}$	Accumulated Denominator, uses final m_N , is precise)
a_i	is the final Softmax output	Softmax value for x_i



Flash Attention

- Online Safe Softmax
 - d_i can be computed iteratively

Iterative Denominator Derivation

$$\begin{aligned}d'_i &= \sum_{j=1}^i e^{x_j - m_i} \\&= \left(\sum_{j=1}^{i-1} e^{x_j - m_i} \right) + e^{x_i - m_i} \\&= \left(\sum_{j=1}^{i-1} e^{x_j - m_{i-1}} \right) e^{m_{i-1} - m_i} + e^{x_i - m_i} \\&= d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}\end{aligned}$$



ALGORITHM: 2-PASS ONLINE SOFTMAX

Pass 1: Max & Denominator Sum Update

for $i \leftarrow 1$ **to** N **do**

$m_i \leftarrow \max(m_{i-1}, x_i)$

$d'_i \leftarrow d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$

end for

Pass 2: Compute Final Softmax Output

for $i \leftarrow 1$ **to** N **do**

$a_i \leftarrow \frac{e^{x_i - m_N}}{d'_N}$

end for

Flash Attention

ONLINE SAFE SOFTMAX

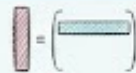
CONCEPT & TARGET

- One-pass safe softmax algorithm does not exist
- Safe softmax is only our intermediate result:

$$O = \text{Softmax}(QK^T)V$$

Target Computation

NOTATIONS & SETUP

$Q[k, :]$	the k -th row vector of the Q matrix.	
$K^T[:, i]$	the i -th column vector of the K^T matrix.	
$O[k, :]$	the k -th row of the output O matrix.	
$V[i, :]$	the i -th row of the V matrix.	
$\mathbf{o}_i$	storing partial aggregation result $A[k, :i] \times V[:, i]$	
$\mathbf{o}_i = \sum_{j=1}^i a_j V[j, :]$		

ALGORITHMIC BODY

```
for i from 1 to N do
   $x_i \leftarrow Q[k, :] K^T[:, i]$  # Dot-product score
   $m_i \leftarrow \max(m_{i-1}, x_i)$  # Running maximum
   $d'_i \leftarrow d'_{i-1} \exp(m_{i-1} - m_i) + \exp(x_i - m_i)$  # Running denominator update
end

for i from 1 to N do
   $a_i \leftarrow \exp(x_i - m_N) / d'_N$  # Normalized score
   $\mathbf{o}_i \leftarrow \mathbf{o}_{i-1} + a_i V[i, :]$  # Running output vector
end

 $O[k, :] \leftarrow \mathbf{o}_N$  # Final assignment
```

Flash Attention

- **Online Safe Softmax**

- One-pass safe softmax algorithm does not exist
- Safe softmax is only our intermediate result:

$$O = \text{Softmax}(QK^T)V$$

NOTATIONS

$Q[k, :]$: the k -th row vector of Q matrix.

$K^T[:, i]$: the i -th column vector of K^T matrix.

$O[k, :]$: the k -th row of output O matrix.

$V[i, :]$: the i -th row of V matrix.

$\{\mathbf{o}_i\}$: $\sum_{j=1}^i a_j V[j, :]$, a row vector storing partial aggregation result $A[k, :i] \times V[:i, :]$

BODY

for $i \leftarrow 1, N$ **do**

$$x_i \leftarrow Q[k, :] K^T[:, i]$$

$$m_i \leftarrow \max(m_{i-1}, x_i)$$

$$d'_i \leftarrow d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

end

for $i \leftarrow 1, N$ **do**

$$a_i \leftarrow \frac{e^{x_i - m_N}}{d_N^f}$$

$$\mathbf{o}_i \leftarrow \mathbf{o}_{i-1} + a_i V[i, :]$$

end

$$O[k, :] \leftarrow \mathbf{o}_N$$

Flash Attention

One-pass Attention

- Define an “intermediate” output

$$o'_i := \left(\sum_{j=1}^i \frac{e^{x_j - m_i}}{d'_i} V[j, :] \right) \rightarrow$$

for $i \rightarrow 1, N$

$$x_i \leftarrow Q[k, :] K^T[:, i]$$

$$m_i \leftarrow \max(m_{i-1}, x_i)$$

$$d'_i \leftarrow d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$o'_i = o'_{i-1} \frac{d'_{i-1} e^{m_{i-1} - m_i}}{d'_i} + \frac{e^{x_i - m_i}}{d'_i} V[i, :]$$

end

$$O[k, :] \leftarrow o'_N$$

Flash Attention

- Compute it iteratively

$$\begin{aligned} o'_i &= \sum_{j=1}^i \frac{e^{x_j - m_i}}{d'_i} V[j, :] \\ &= \left(\sum_{j=1}^{i-1} \frac{e^{x_j - m_i}}{d'_i} V[j, :] \right) + \frac{e^{x_i - m_i}}{d'_i} V[i, :] \\ &= \left(\sum_{j=1}^{i-1} \frac{e^{x_j - m_{i-1}}}{d'_{i-1}} \frac{e^{x_j - m_i}}{e^{x_j - m_{i-1}}} \frac{d'_{i-1}}{d'_i} V[j, :] \right) + \frac{e^{x_i - m_i}}{d'_i} V[i, :] \\ &= o'_{i-1} \dots V[j, :] + \frac{d'_{i-1}}{d'_i} e^{m_{i-1} - m_i} + \frac{e^{x_i - m_i}}{d'_i} V[i, :] \\ &= o'_{i-1} \frac{d'_{i-1} e^{m_{i-1} - m_i}}{d'_i} + \frac{e^{x_i - m_i}}{d'_i} V[i, :] \end{aligned}$$

Flash Attention

Tiling: loading and computing at the block granularity

Pseudocode of Flash Attention

Algorithm 1 FLASHATTENTION

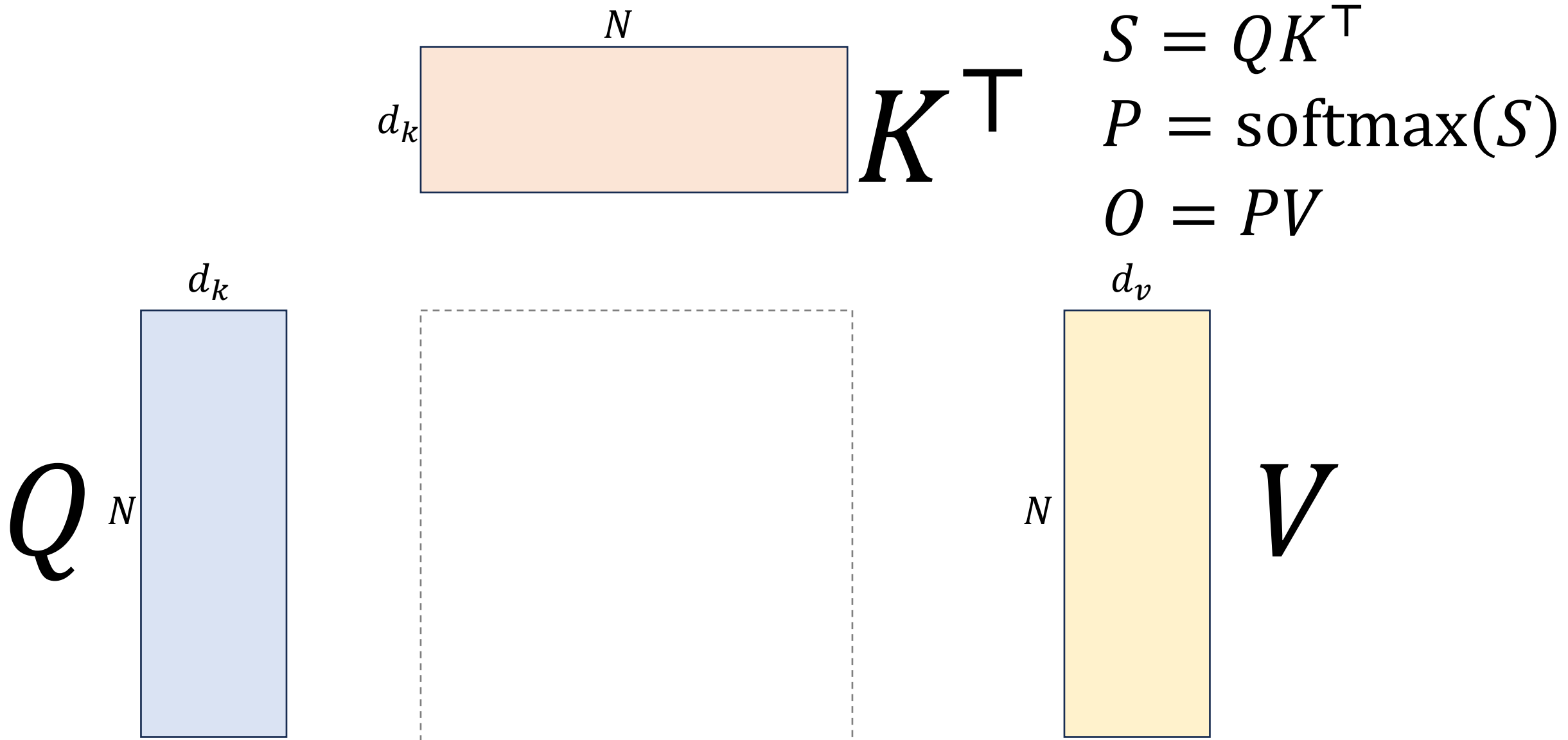
Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

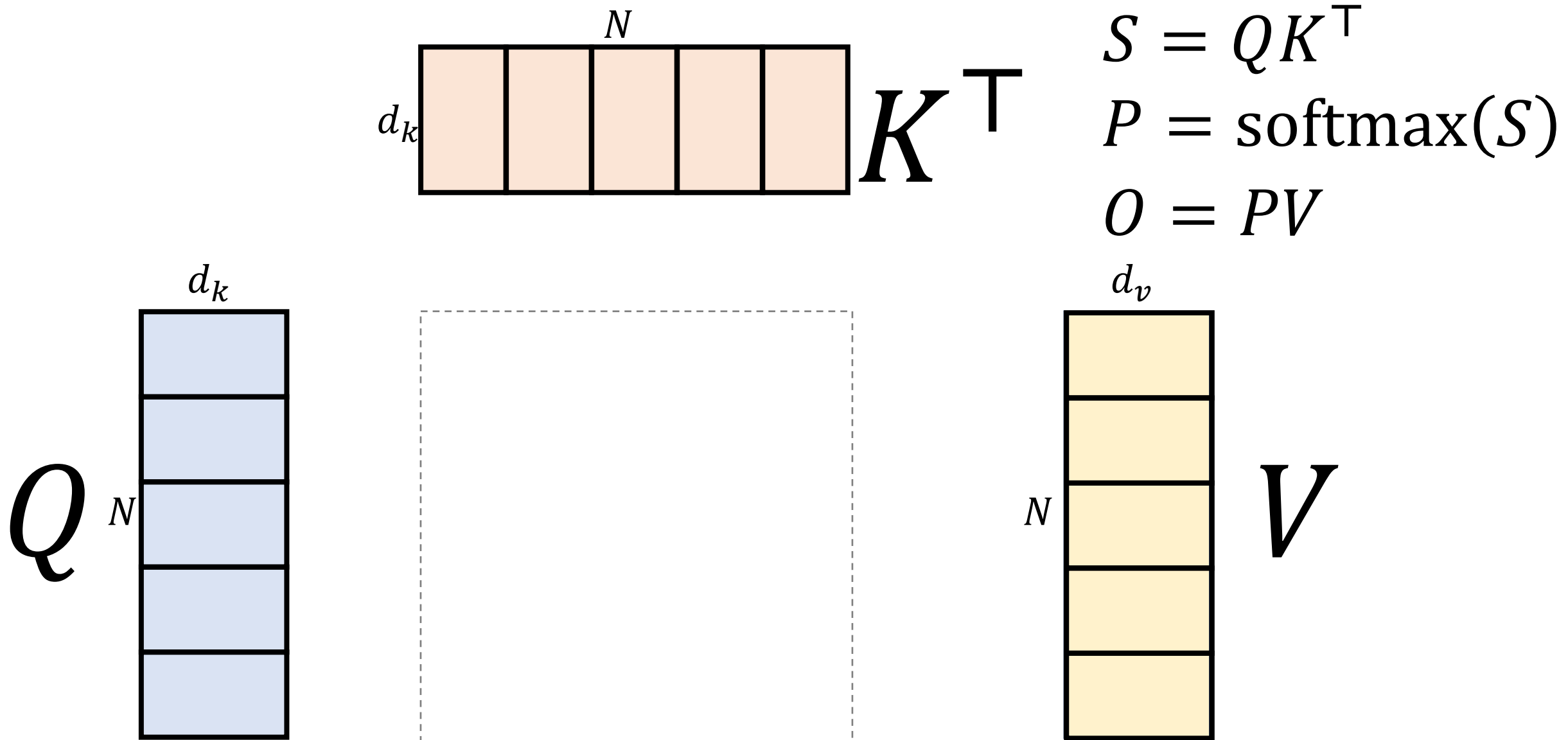
- 1: Set block sizes $B_c = \lfloor \frac{M}{4d} \rfloor$, $B_r = \min(\lfloor \frac{M}{4d} \rfloor, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lfloor \frac{N}{B_r} \rfloor$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lfloor \frac{N}{B_c} \rfloor$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_r$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

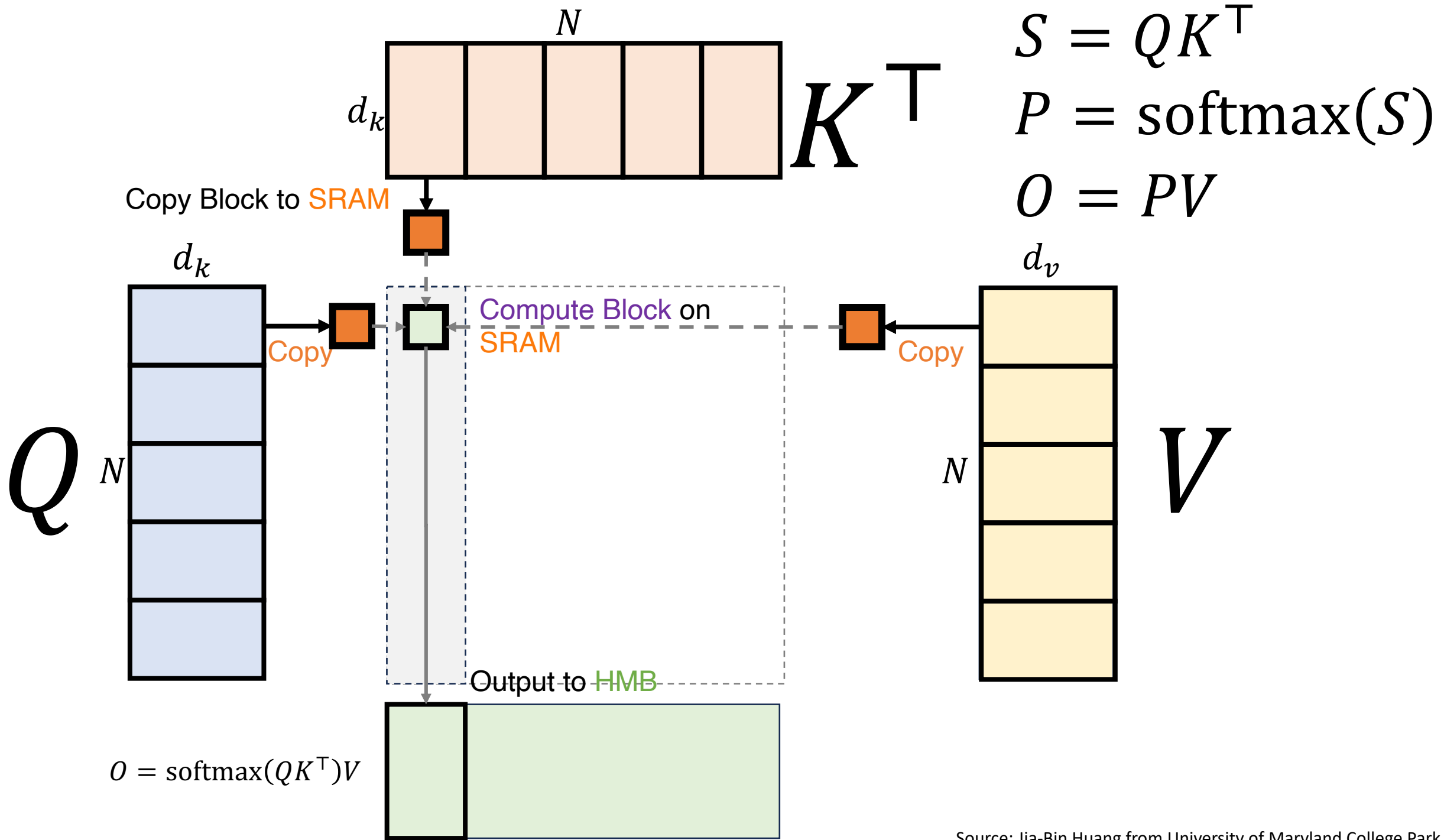
Outer Loop on Key/Value
(over index j)

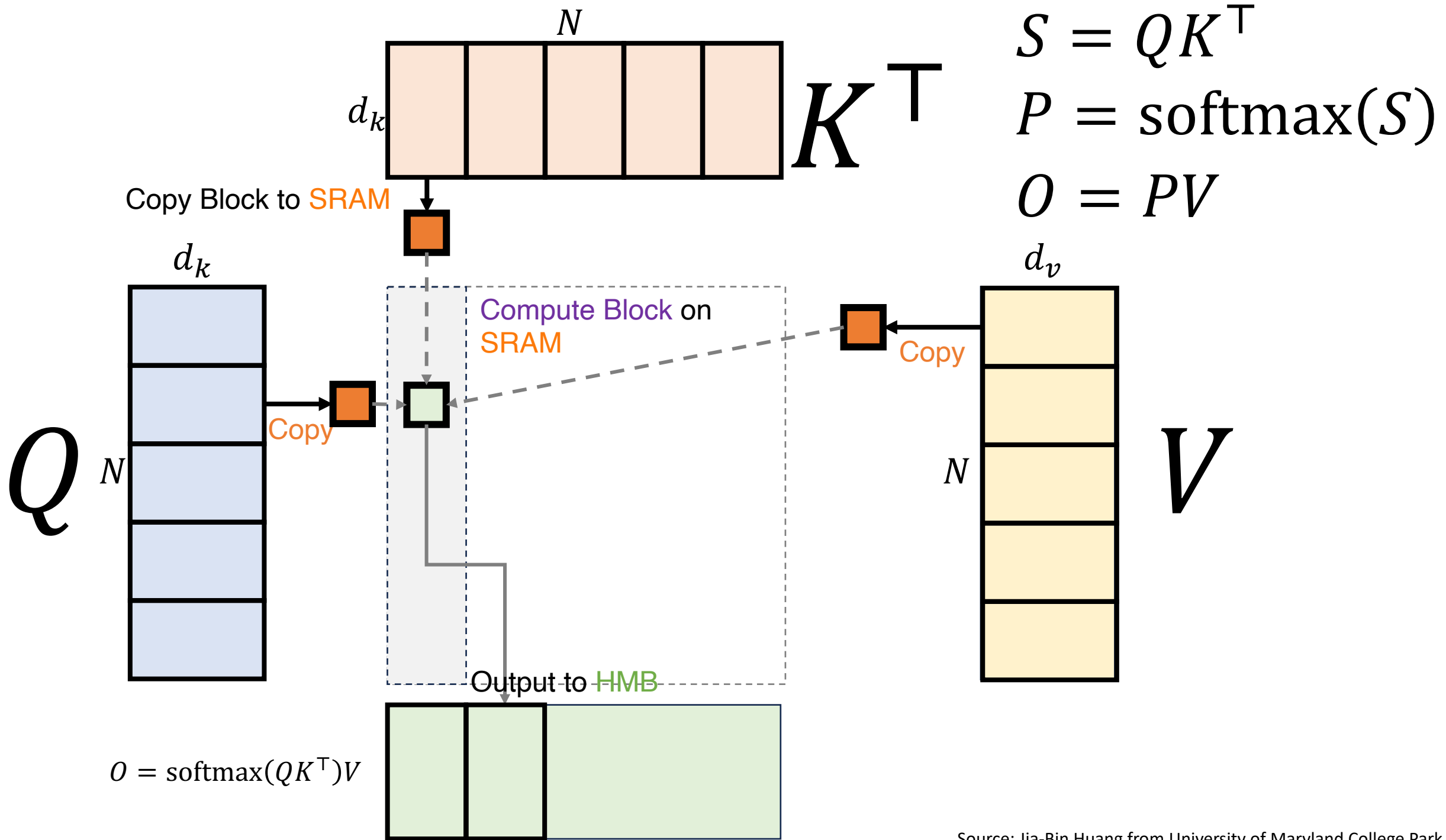
Inner Loop on Query/Output
(over index i)

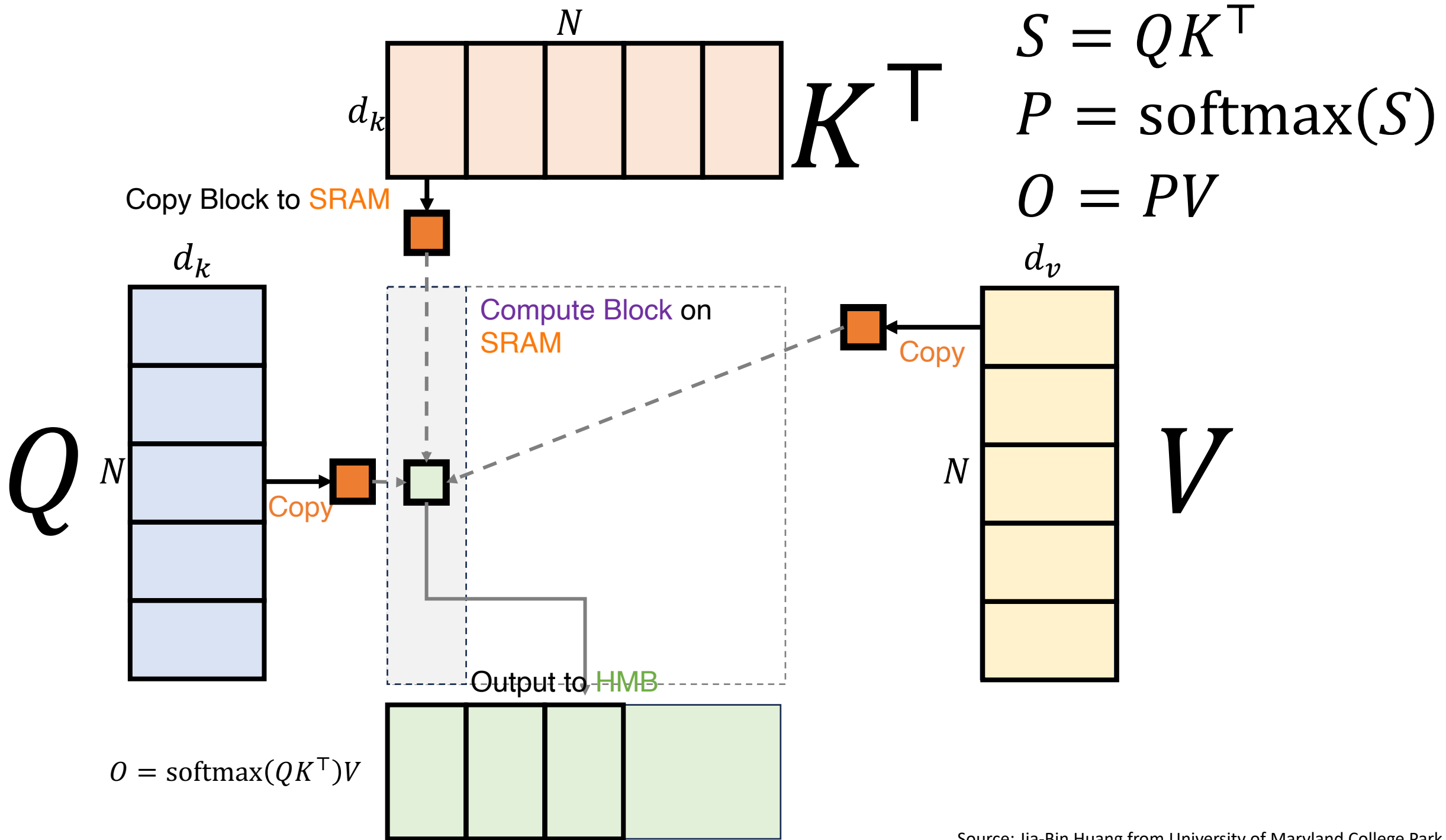
Write $\mathbf{O}_i$ to HBM
Write ℓ_i to HBM

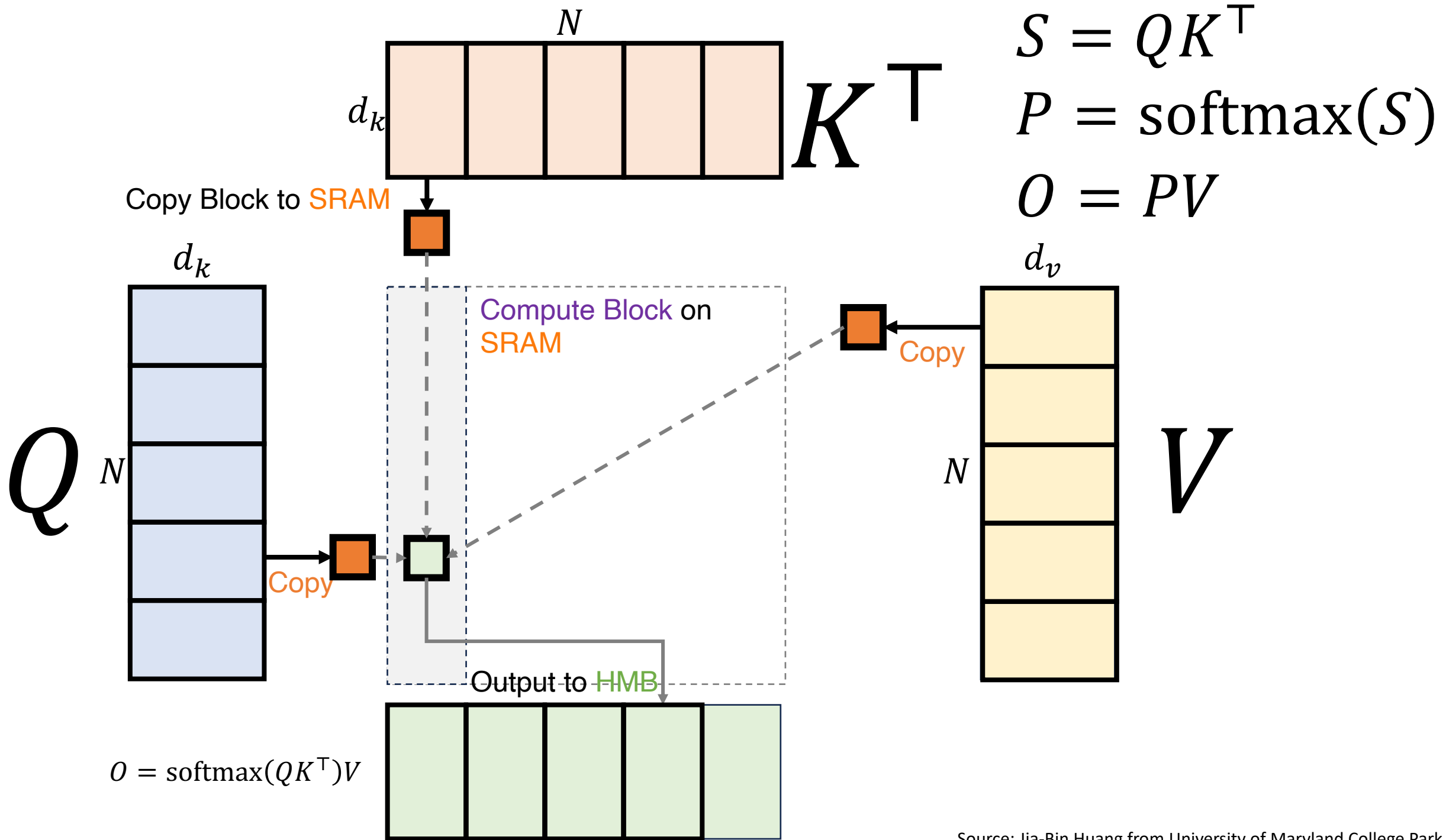


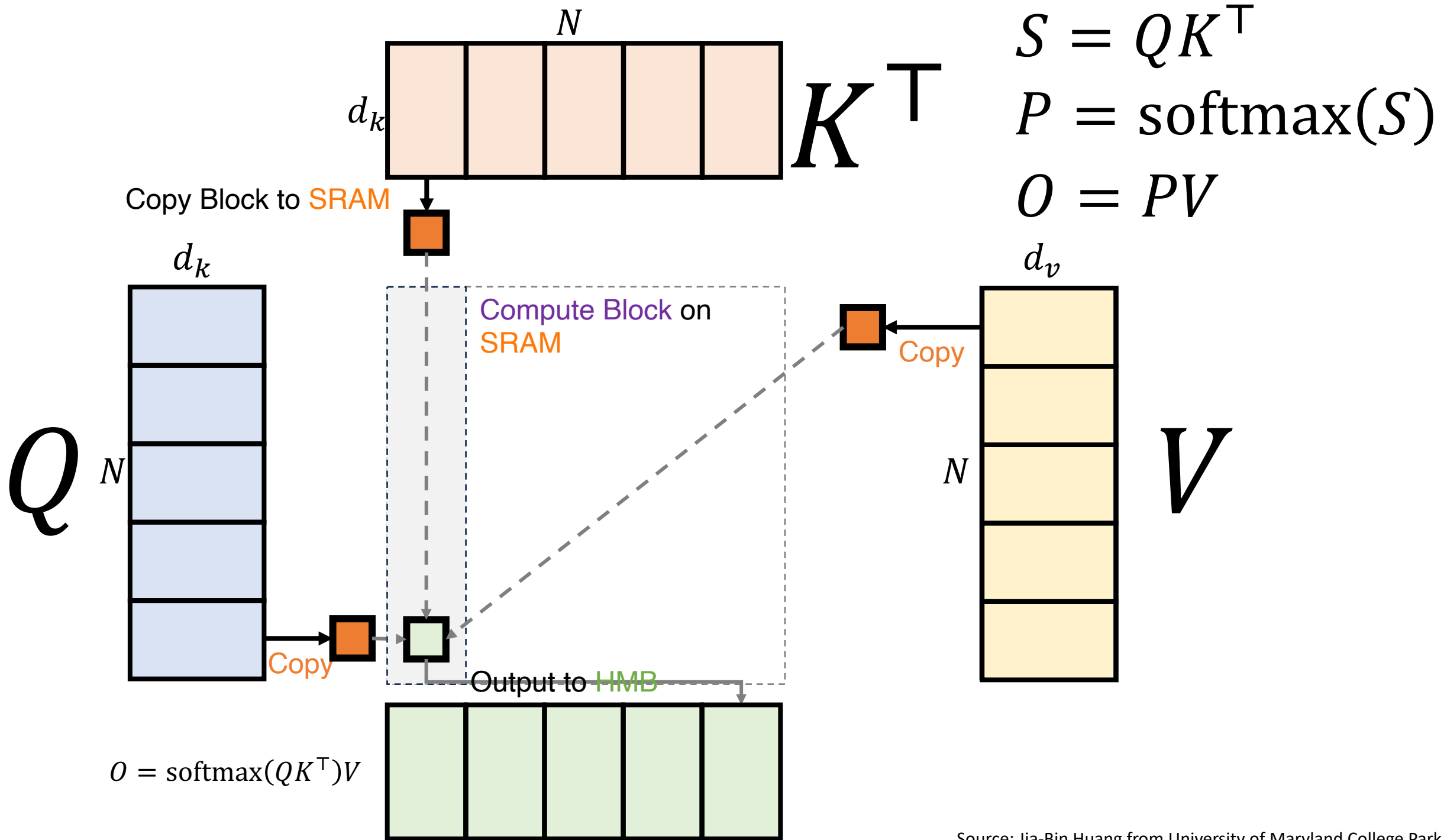


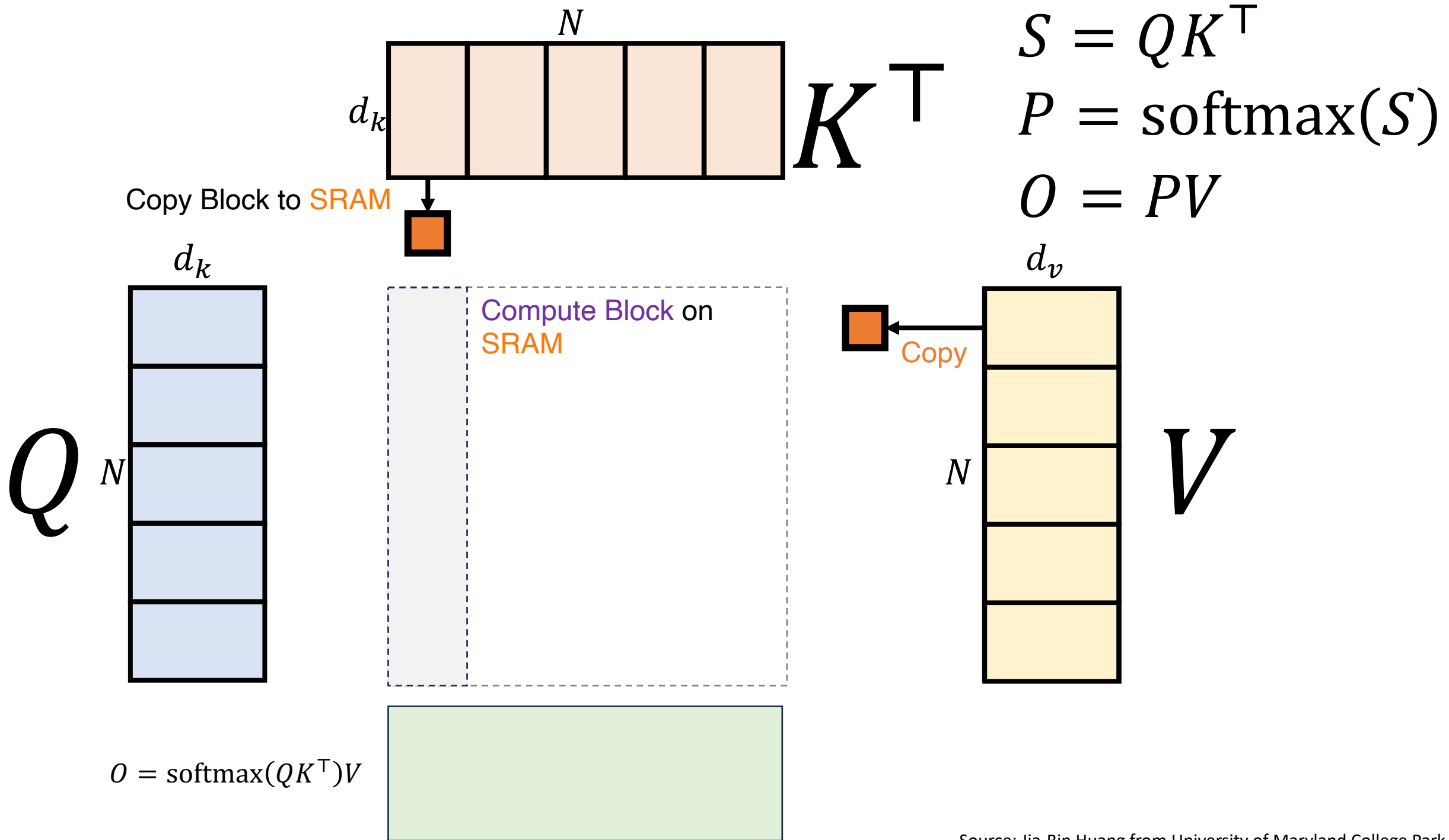


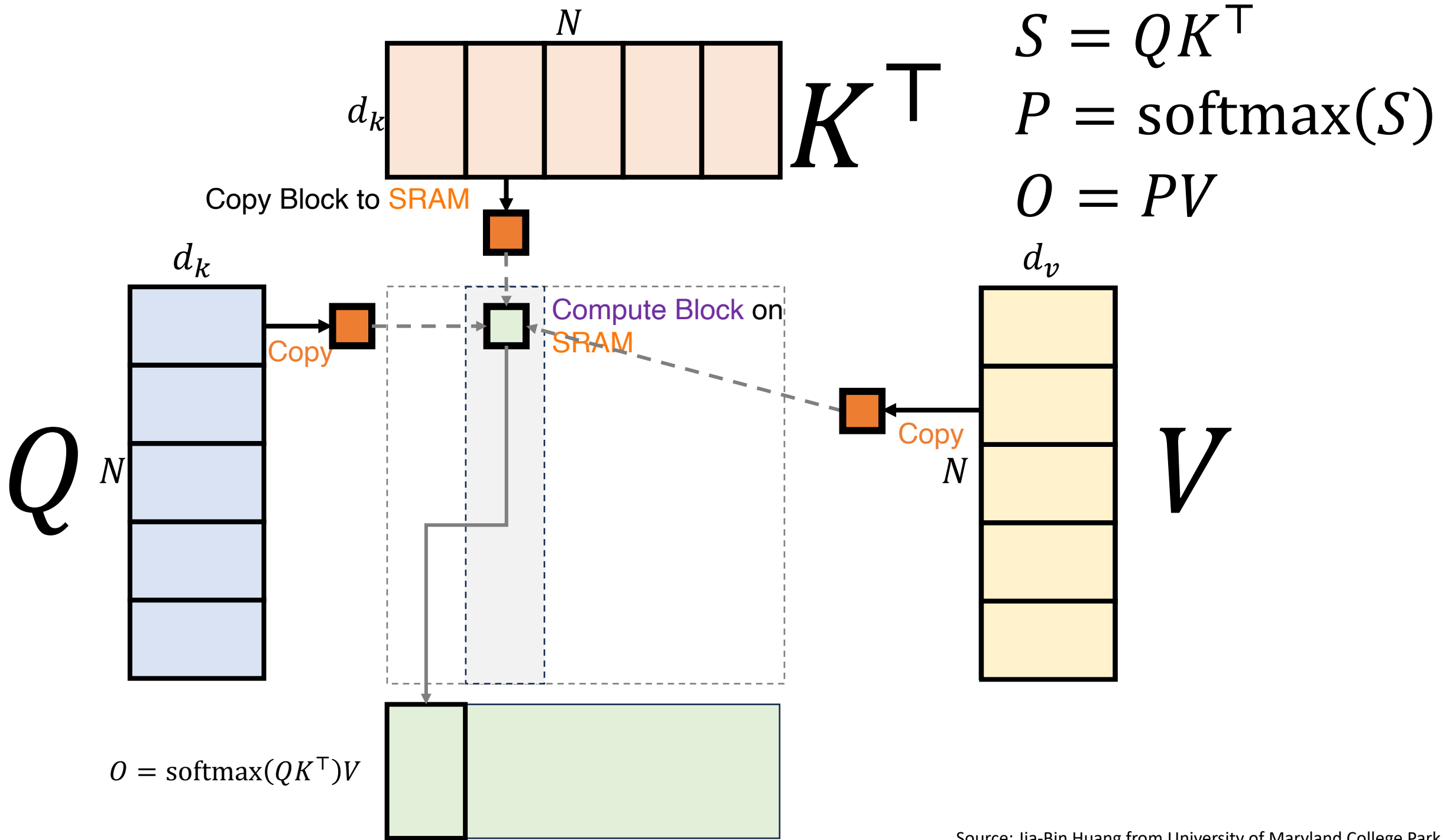


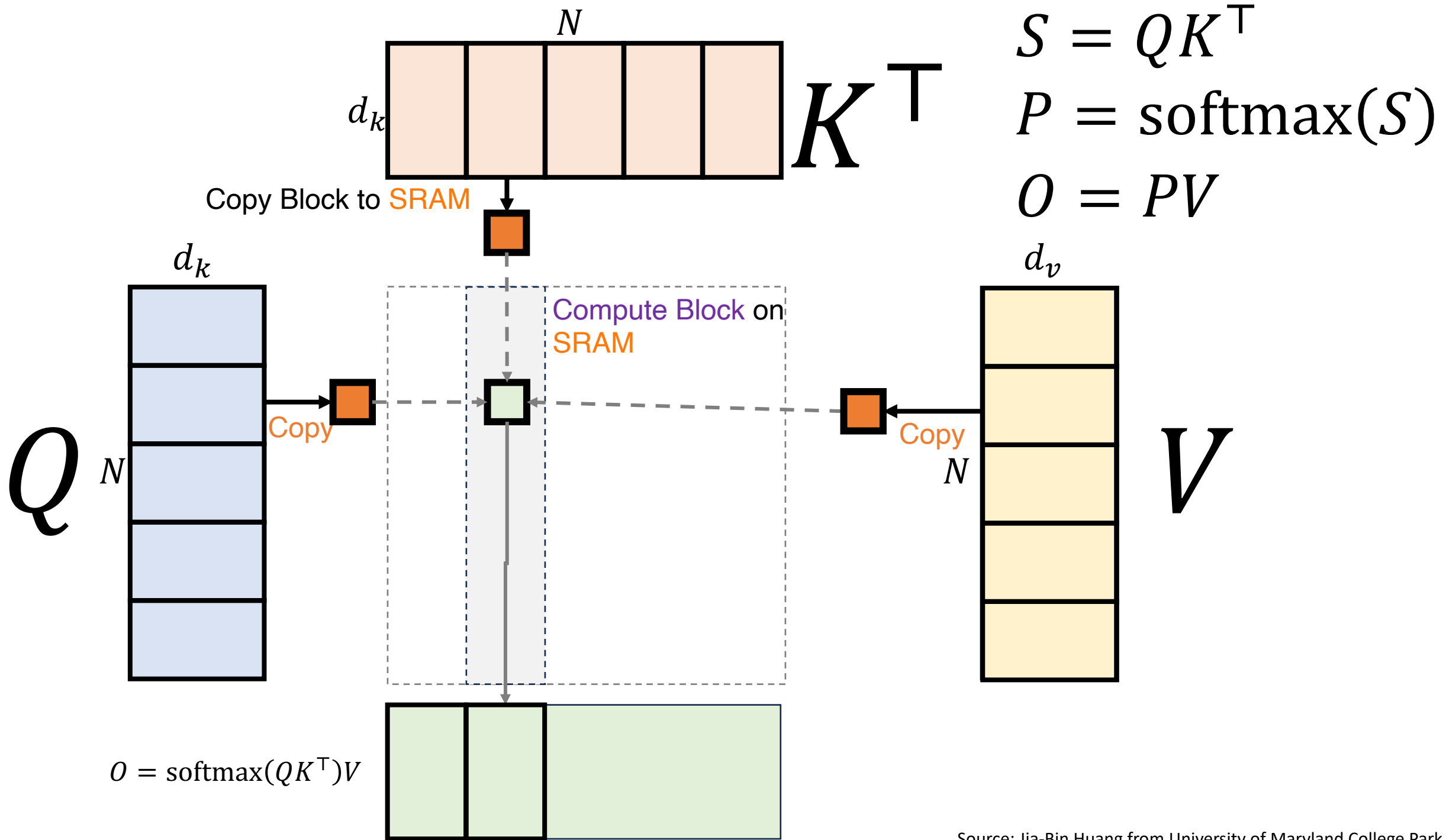


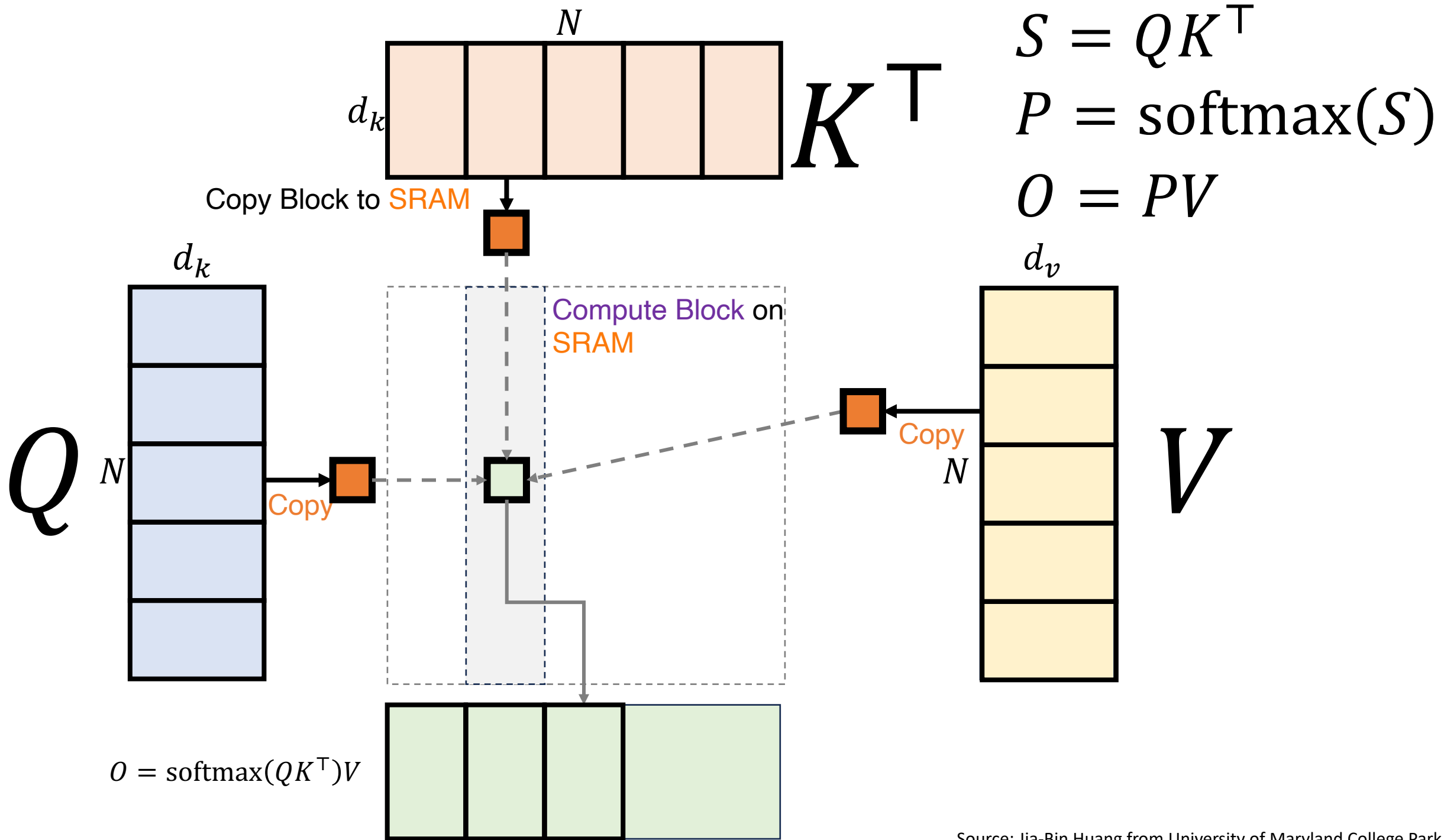


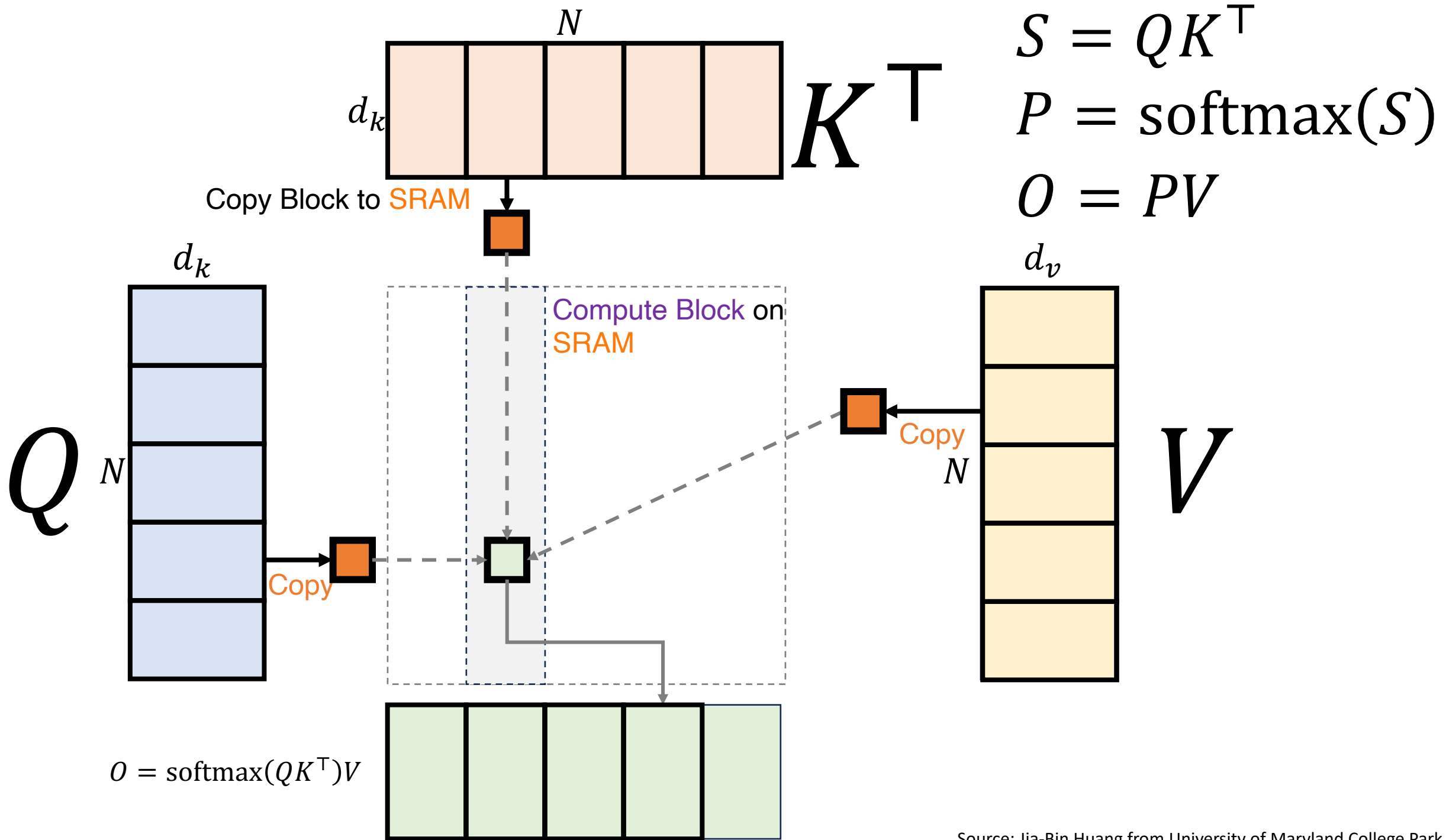


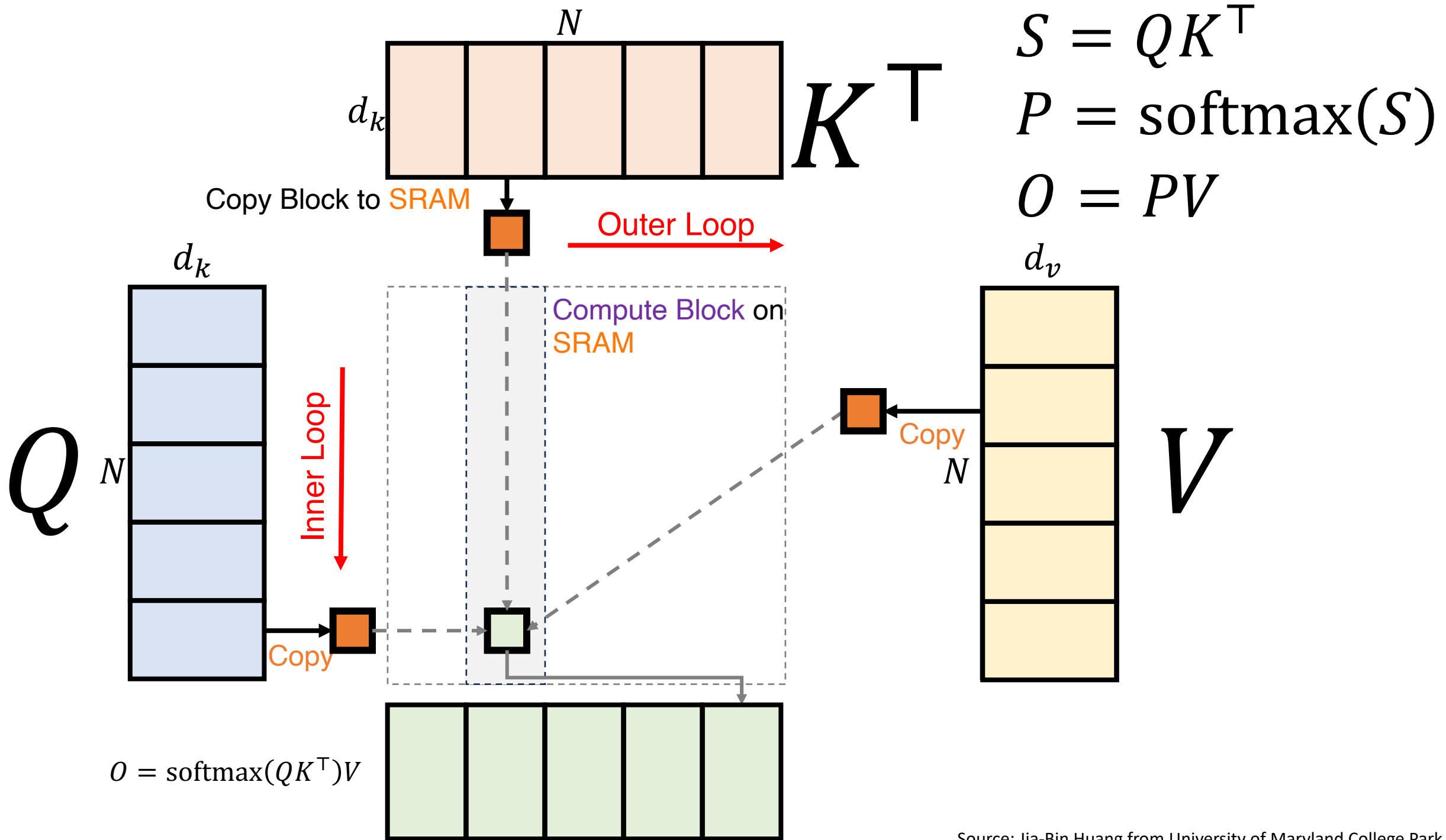






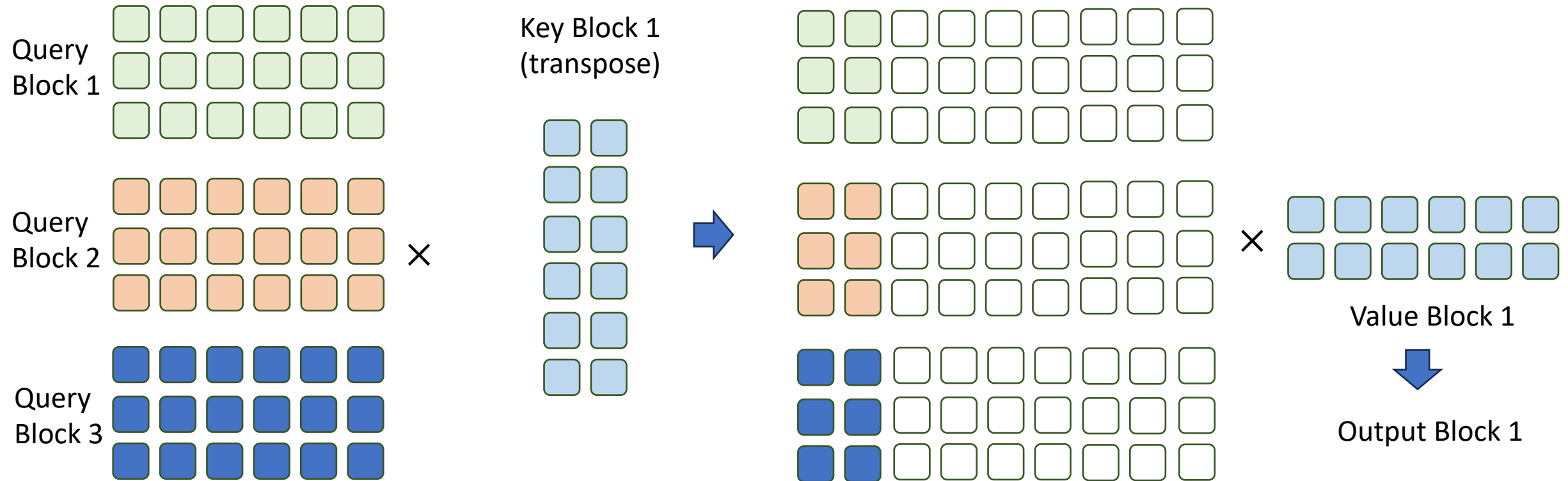






Flash Attention

- Flash Attention (Tiling)

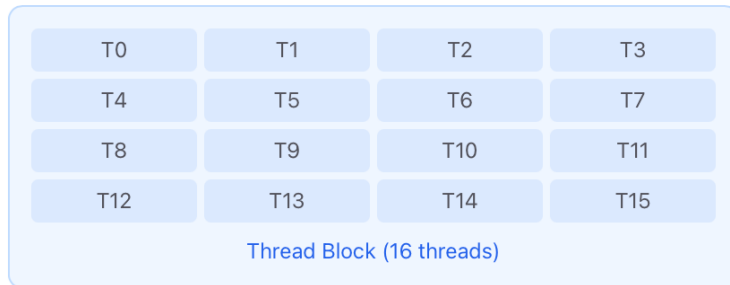


Flash Attention

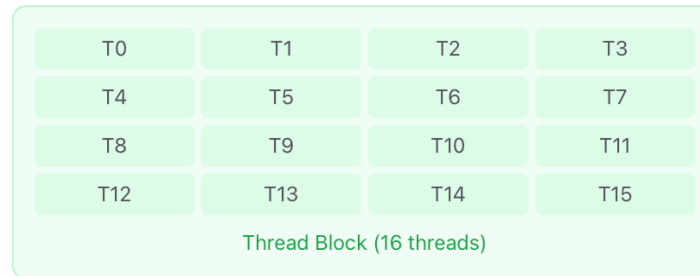
- Kernel Fusion

- Multiple separate operations or kernels (small, independent programs that run on hardware like GPUs) are combined or "fused" into a single, more efficient kernel.

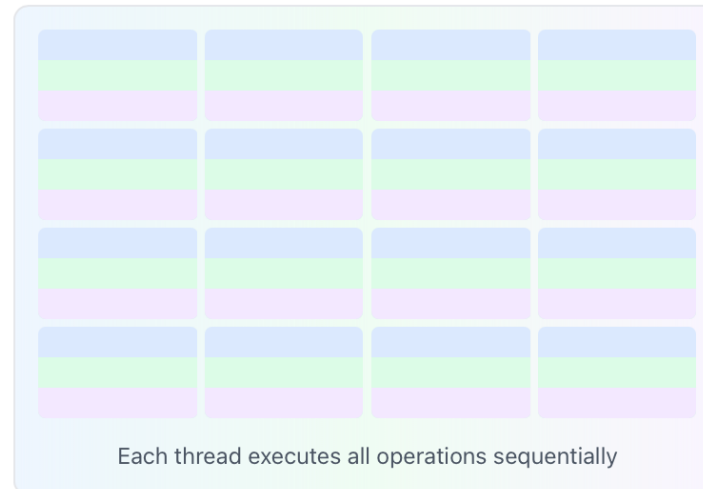
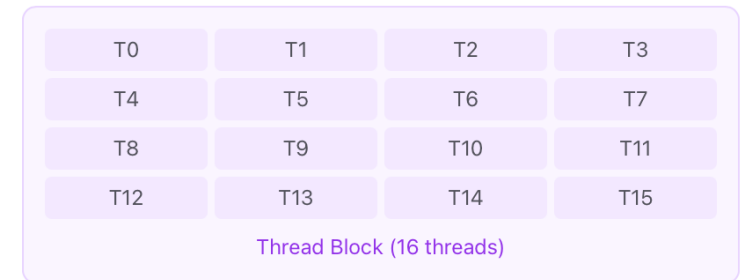
Kernel 1: Scale



Kernel 2: ReLU



Kernel 3: Add

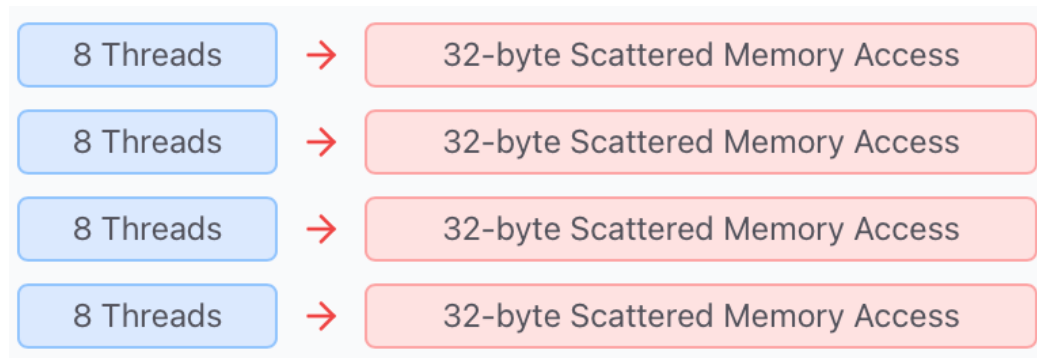


Pay the launch overhead **once**
instead of many times

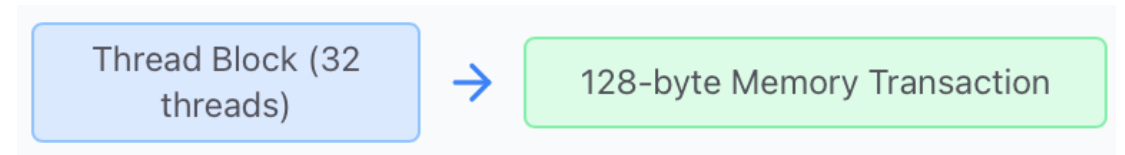
Flash Attention

- Kernel Fusion

- Minimizing data movement between slow global memory and faster on-chip memory (like SRAM or registers), avoids storing intermediate results



Multiple smaller memory transactions

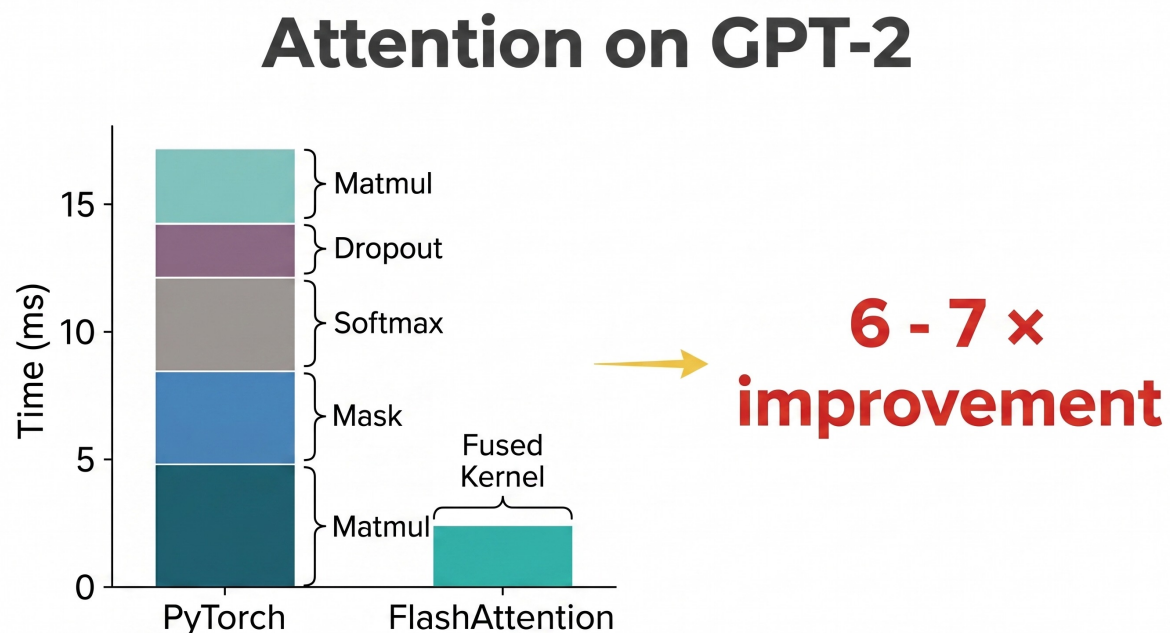


Single memory transaction for 32 consecutive elements

Flash Attention

- Kernel Fusion

- FlashAttention kernel fusion combining multiple (block-wise) steps of the attention computation—such as scaling, masking, softmax normalization, and matrix multiplications—into custom fused CUDA kernels



Flash Attention

- Complexity Analysis

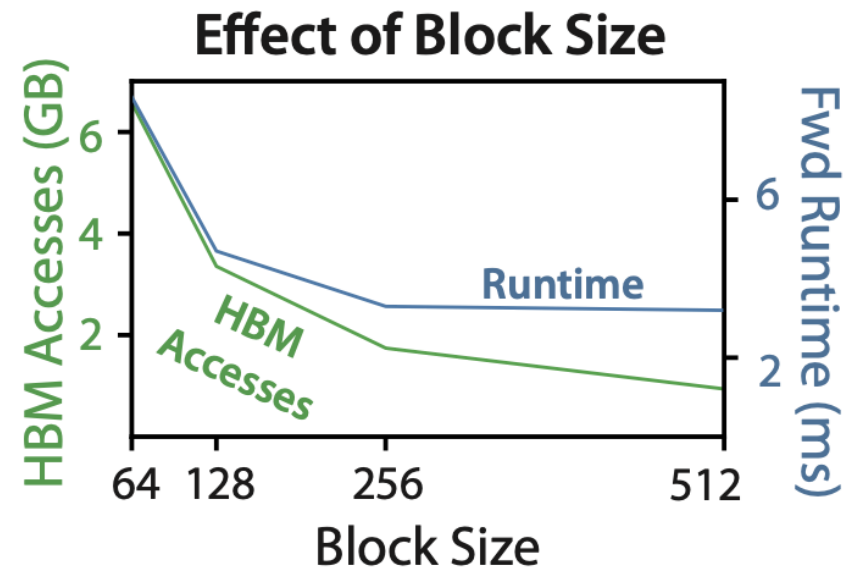
- GPT2 Medium (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100

Attention	Standard	FLASHATTENTION
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3

- Computational complexity: Flash Attention is slightly higher
- Memory complexity: Flash Attention is significantly better $O(Nd)$
- I/O complexity
 - Standard Attention $O(N^2d)$ versus Flash Attention $O(N^2d^2M^{-1})$
 - Outer loop runs $T_c = N/B_c = 4Nd/M$ where M is SRAM size of a SM, and reads K and V once
 - Inner loop reads Q , O , and write O back to HBM
 - Total I/O complexity: $O(Nd) \times T_c = O(N^2d^2M^{-1})$

Flash Attention

- Flash Attention in practice
 - Impact of hyper parameters:
Larger block size reduces the number of data loading, thus improving the forward runtime

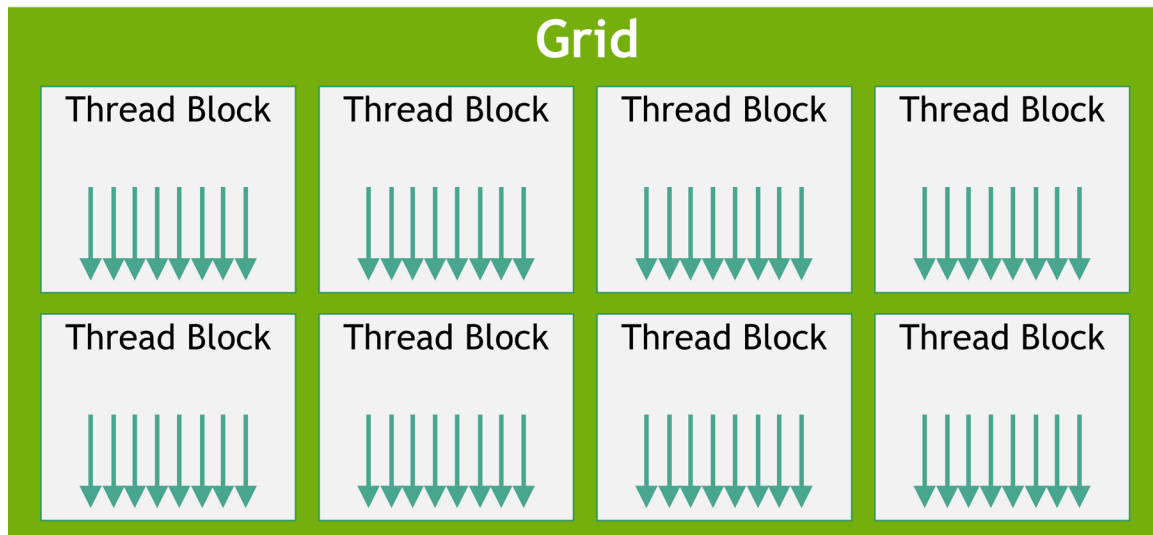


Forward runtime of FlashAttention

- As hidden dimension becomes large, the block size shrinks
 - $O(N^2 d^2 M^{-1})$ is quadratic to the hidden dimension d , while M is constrained by the hardware

Flash Attention

- Flash Attention
 - Underutilized streaming multi-processors for **small batch size** but **long sequences**

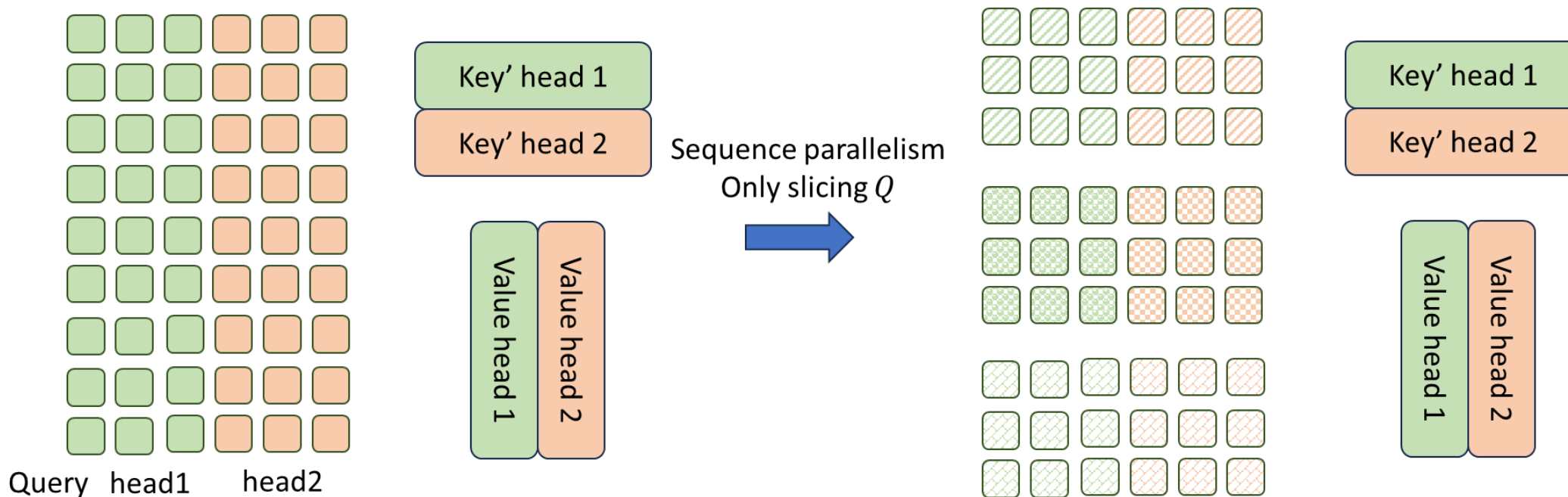


Grid of Thread Blocks

- GPU grid
 - Flash Attention enables Parallel computing on **batches and heads**
 - The grid consists of $batch_size \times head_number$ thread blocks
 - Batch_size = 2 and head_number = 16, meaning that only 16 out of 108 SMs are used on A100

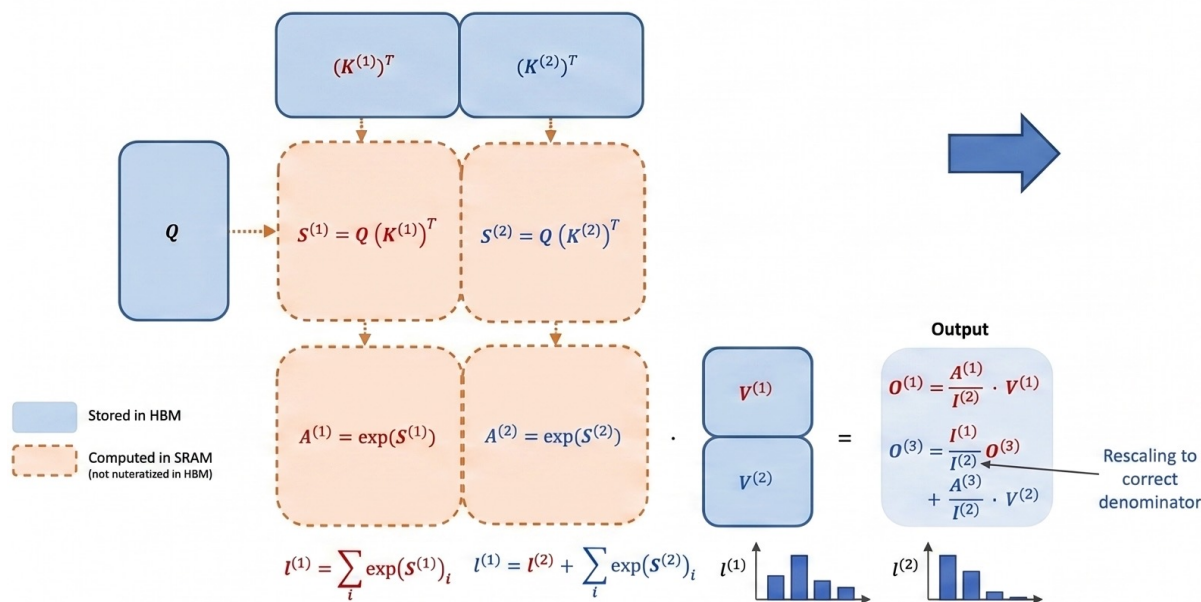
Flash Attention

- Flash Attention v2
 - Making FlashAttention **sequence parallel**
 - The shape of a grid: `grid(num_m_block, batch_size, num_heads)`
 - Introducing the idea of *sequence parallelism*



Flash Attention

- Flash Attention v2: **Loop Reordering**



FlashAttention forward pass: key K is partitioned into two blocks and value V is also partitioned into two blocks

$$m^{(1)} = \text{rowmax}(S^{(1)}) \in \mathbb{R}^{B_r}$$

$$\ell^{(1)} = \text{rowsum}(e^{S^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r}$$

$$\tilde{O}^{(1)} = e^{S^{(1)} - m^{(1)}} V^{(1)} \in \mathbb{R}^{B_r \times d}$$

$$m^{(2)} = \max(m^{(1)}, \text{rowmax}(S^{(2)})) = m$$

$$\ell^{(2)} = e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{S^{(2)} - m^{(2)}}) = \text{rowsum}(e^{S^{(1)} - m}) + \text{rowsum}(e^{S^{(2)} - m}) = \ell$$

$$\tilde{P}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{S^{(2)} - m^{(2)}}$$

$$\tilde{O}^{(2)} = \text{diag}(e^{m^{(1)} - m^{(2)}})^{-1} \tilde{O}^{(1)} + e^{S^{(2)} - m^{(2)}} V^{(2)} = e^{S^{(1)} - m} V^{(1)} + e^{S^{(2)} - m} V^{(2)}$$

$$O^{(2)} = \text{diag}(\ell^{(2)})^{-1} \tilde{O}^{(2)} = O.$$

Check how the final output o_i is computed!

Flash Attention

- FlashAttention v2
 - Algorithm: fewer *non-matmul* FLOPs
 - GPU is highly optimized on *matmul* (e.g. Tensor Cores)
 - NVIDIA A100 GPU: 312 TFLOPs/s of FP16/BF16 *matmul*, only 19.5 TFLOPs/s of *non-matmul* FP32

COMPARISON OF ONLINE SOFTMAX SCALING: FLASHATTENTION v1 vs v2

FlashAttention v1 (Iterative Scaling)

$$\begin{aligned} S_{ij} &\leftarrow Q_i K_j^T \\ m_{ij} &\leftarrow \text{rowmax}(S_{ij}) \\ m_i^{\text{new}} &\leftarrow \max(m_i, m_{ij}) \\ P_{ij} &\leftarrow \exp(S_{ij} - m_i^{\text{new}}) \\ l_{ij} &\leftarrow \text{rowsum}(P_{ij}) \\ l_i^{\text{new}} &\leftarrow \exp(m_i - m_i^{\text{new}}) l_i + l_{ij} \\ O_i^{\text{new}} &\leftarrow (l_i^{\text{new}})^{-1} \cdot (l_i \exp(m_i - m_i^{\text{new}}) O_i + P_{ij} V_j) \end{aligned}$$

 Scales Previous Results for Next Step

Intermediate Output Rescaling (Per Iteration)



Moved all rescaling to the end of the process

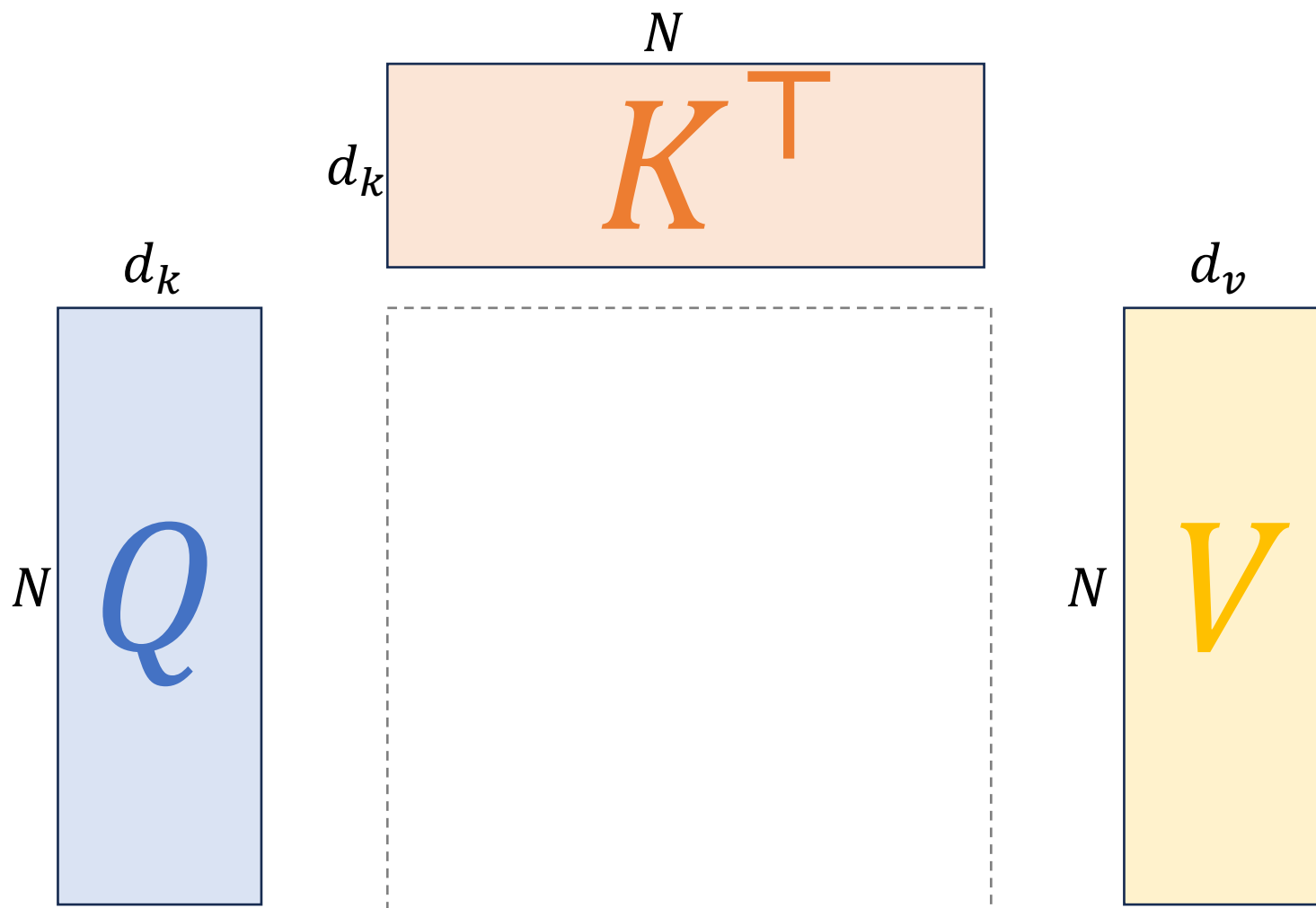
FlashAttention v2 (Optimized Post-Processing Scaling)

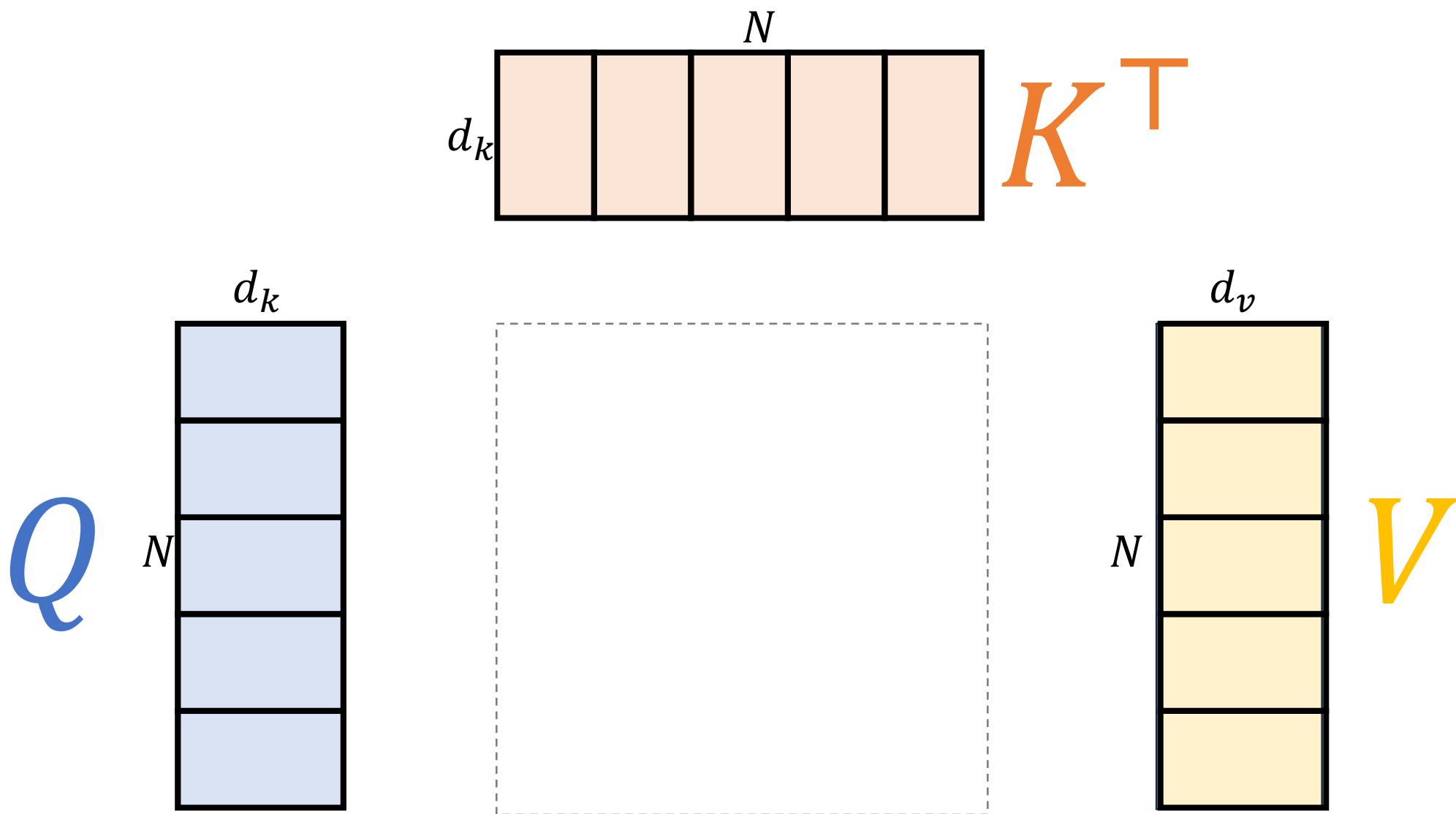
$$\begin{aligned} \tilde{O}_i^{\text{new}} &\leftarrow \exp(m_i - m_i^{\text{new}}) \tilde{O}_i + P_{ij} V_j \\ O_i^{\text{last}} &\leftarrow (l_i^{\text{last}})^{-1} \cdot \tilde{O}_i^{\text{last}} \end{aligned}$$

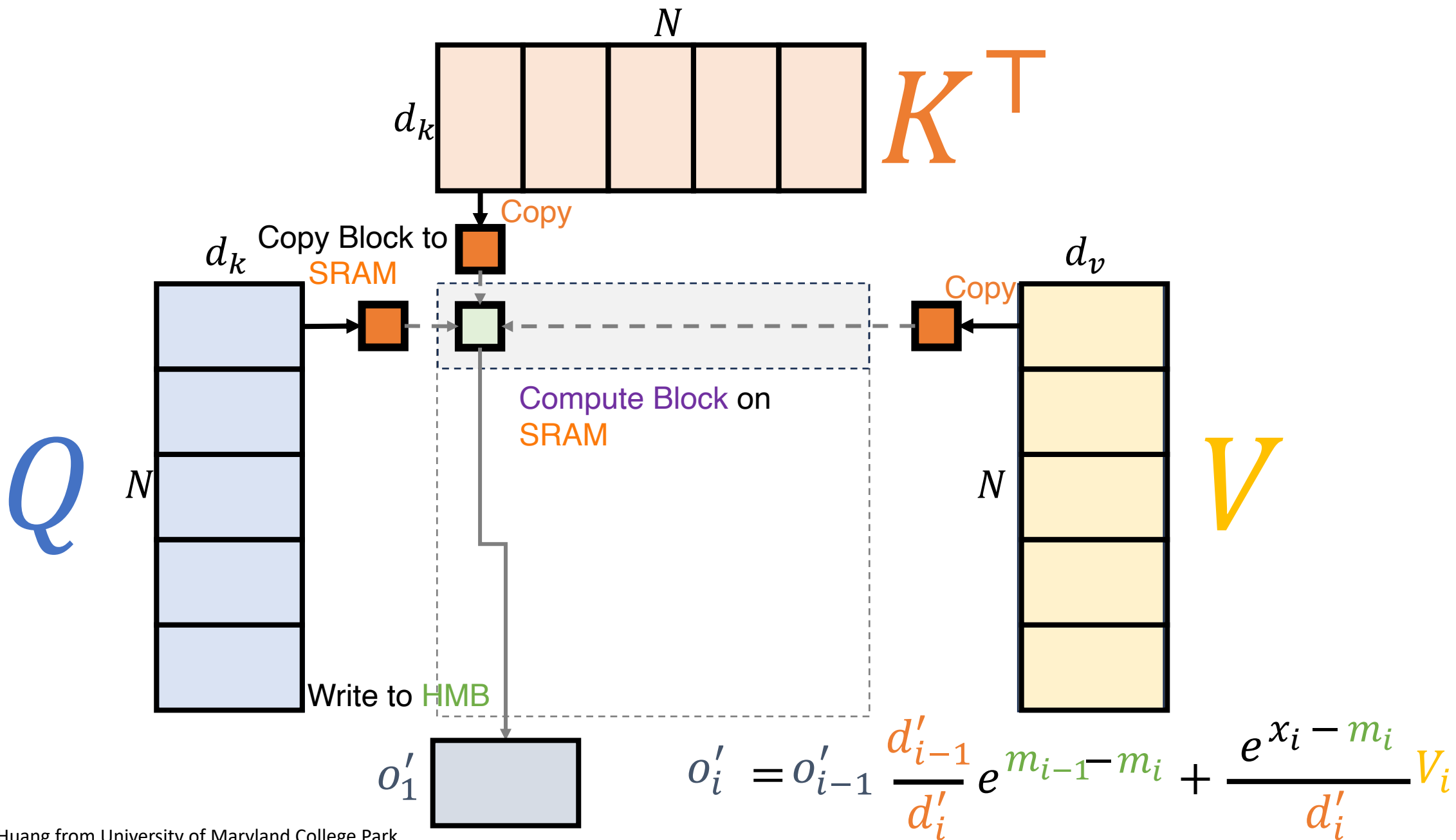


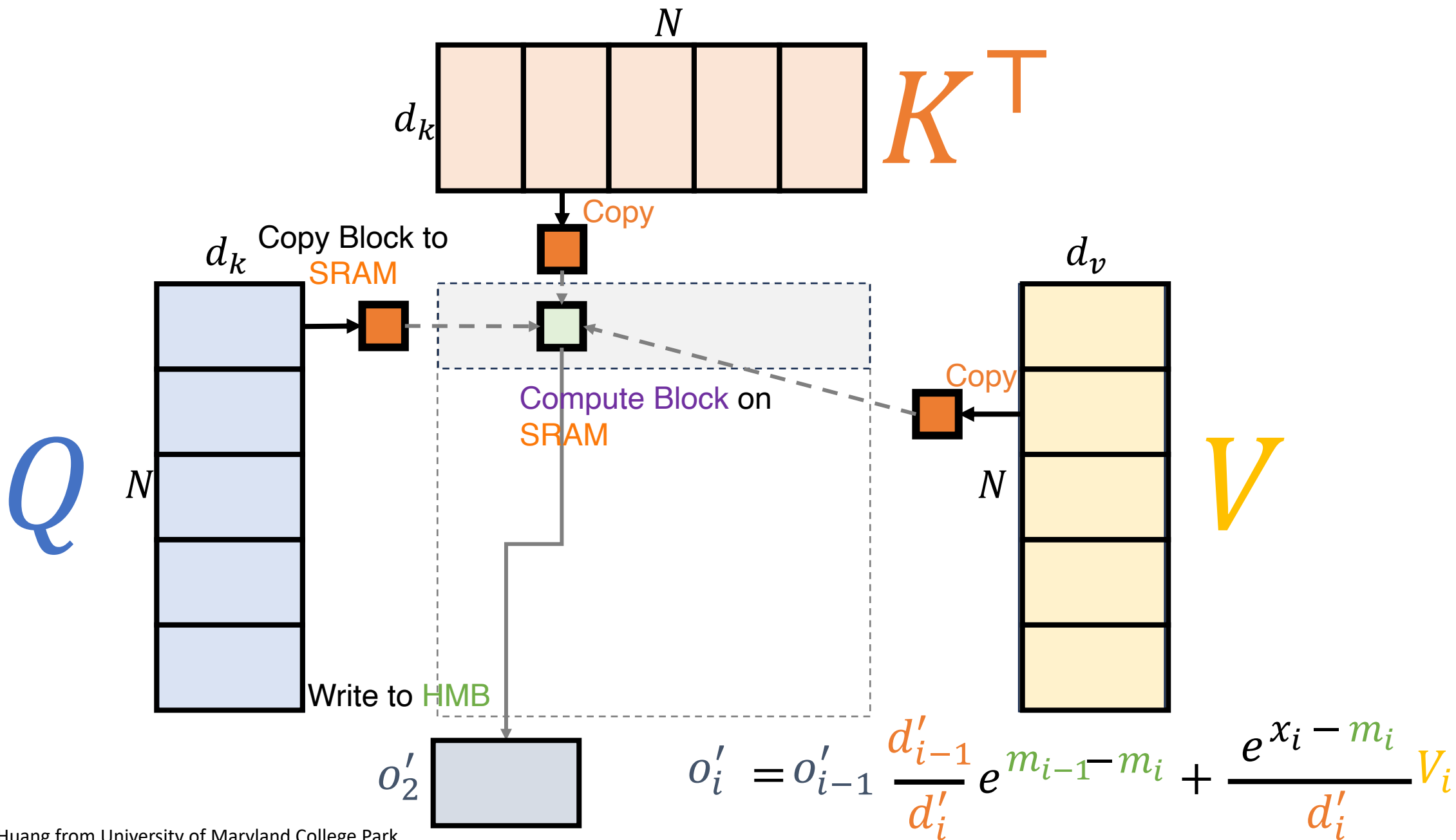
Single Final Result Scaling (Single Point)

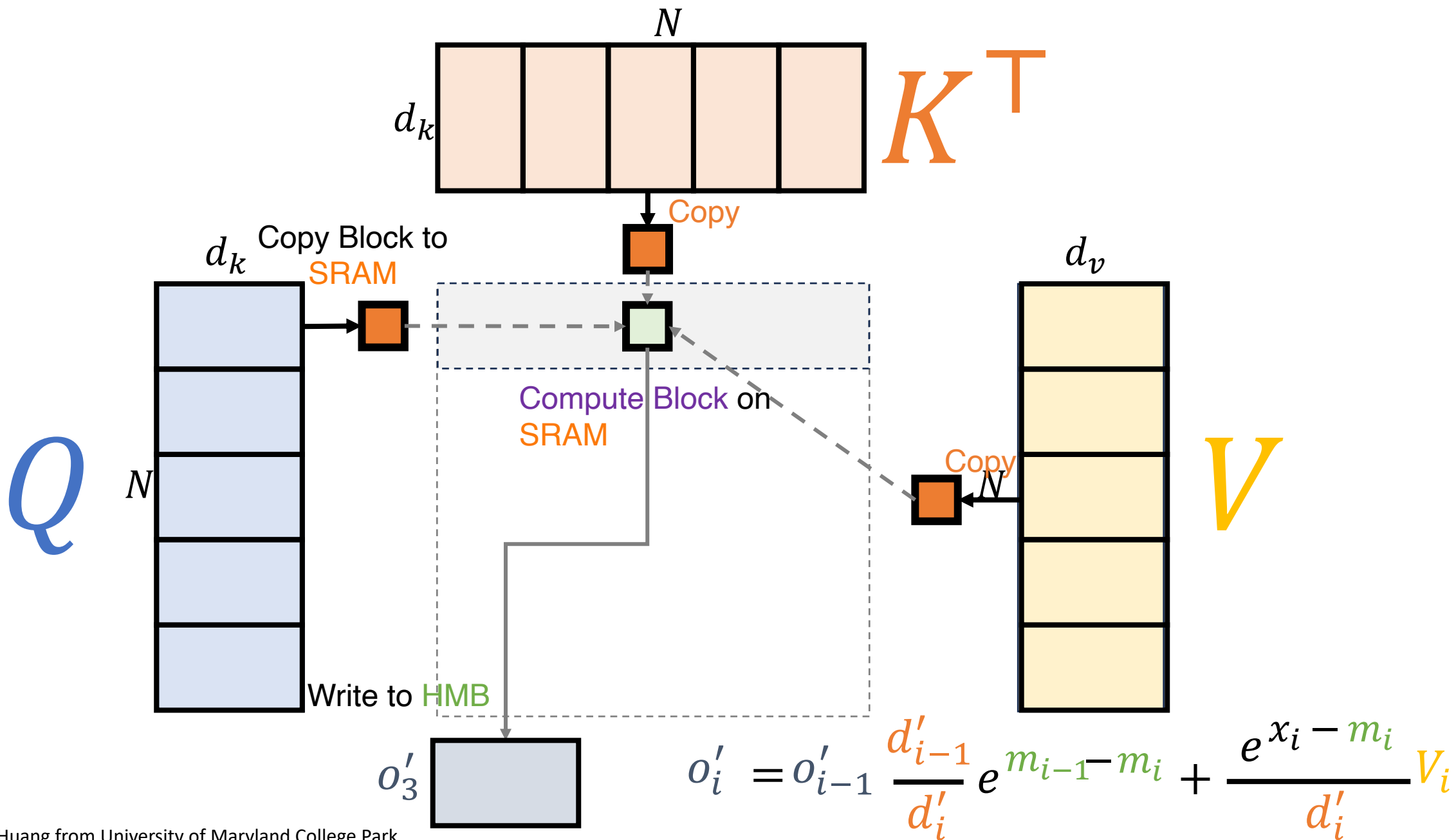
FlashAttention v2 Optimizations: ~ Drastically reduce the number of rescaling ops, as well as bound-checking and causal masking operations.

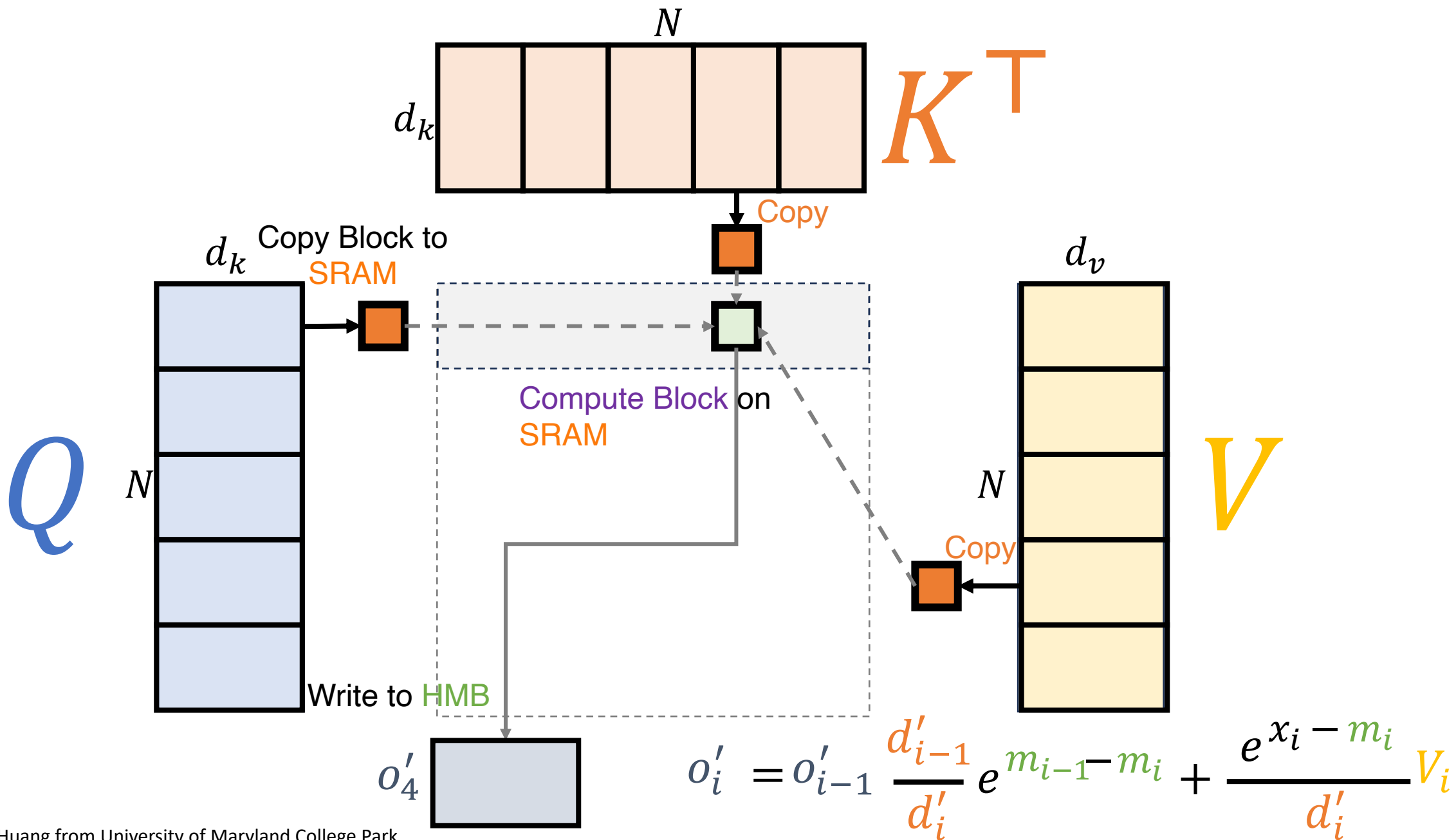


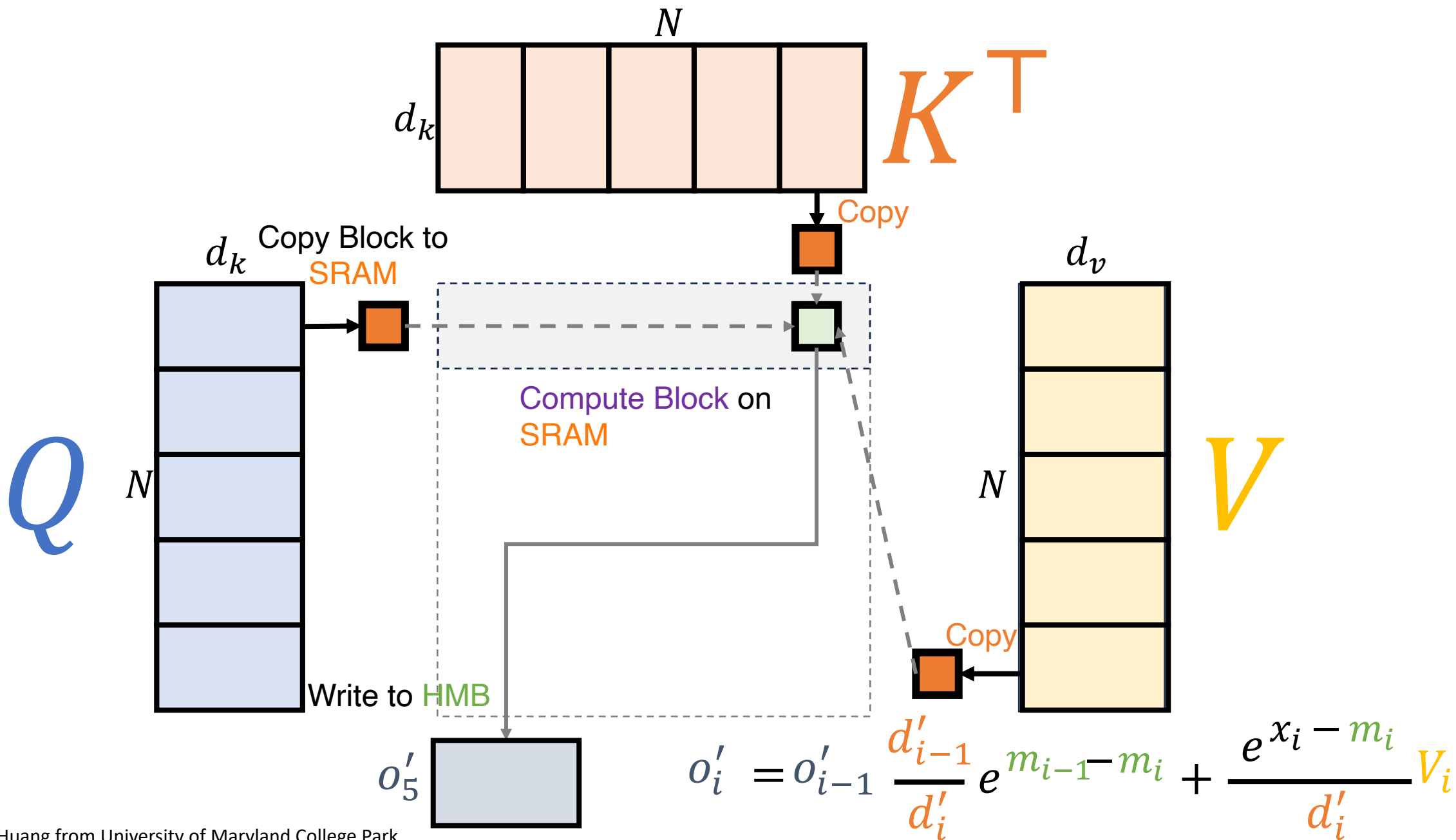


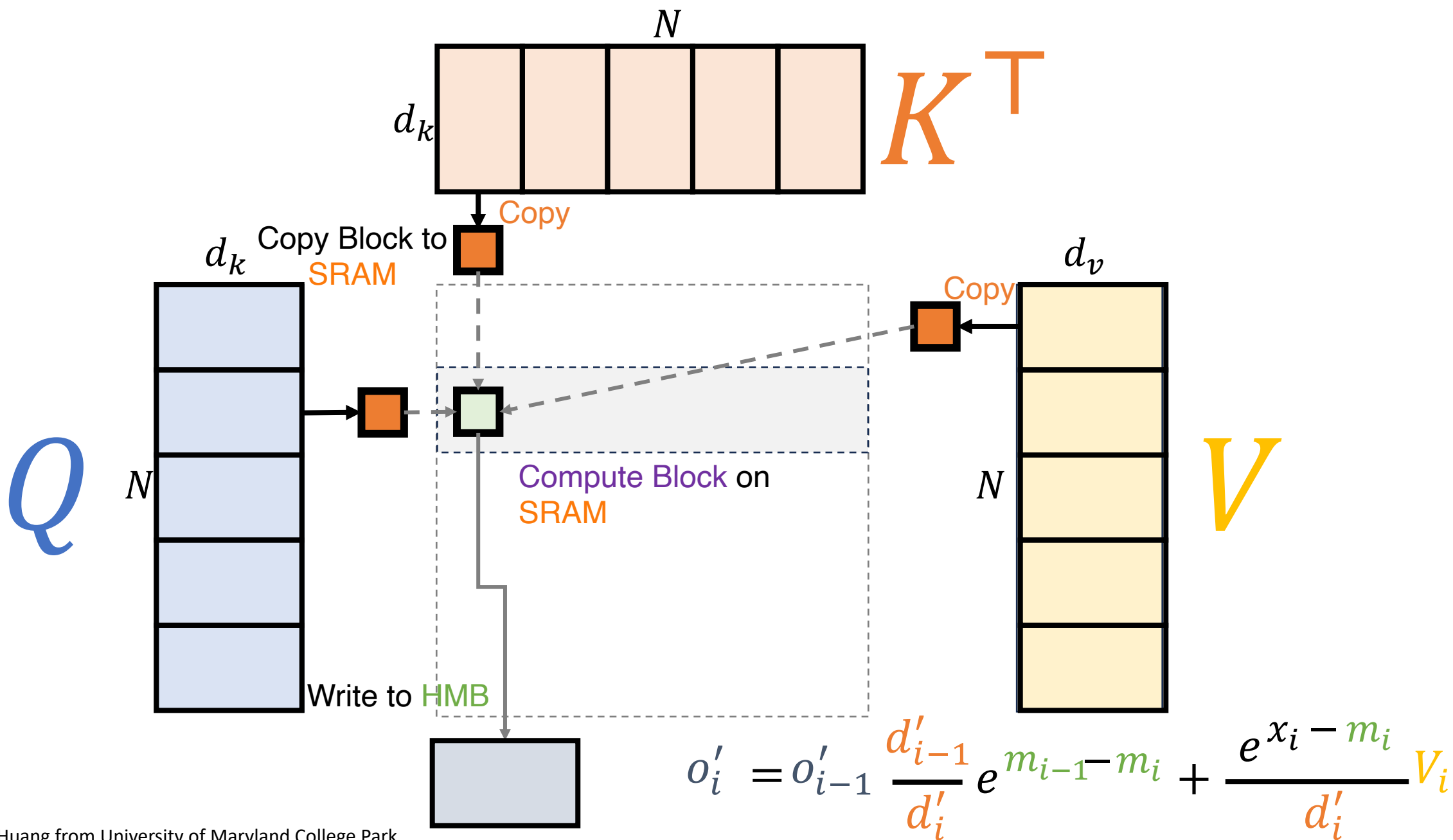


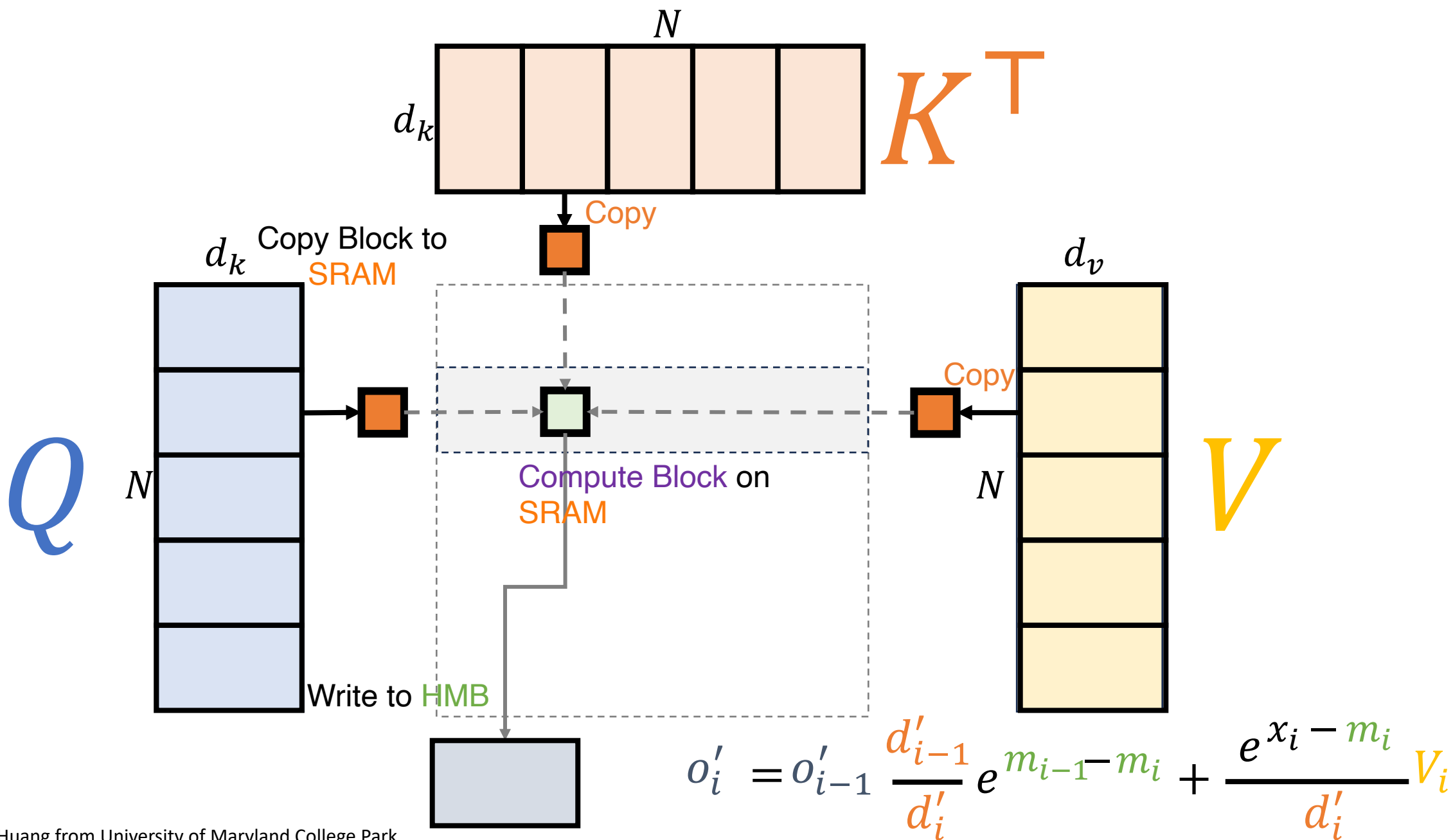


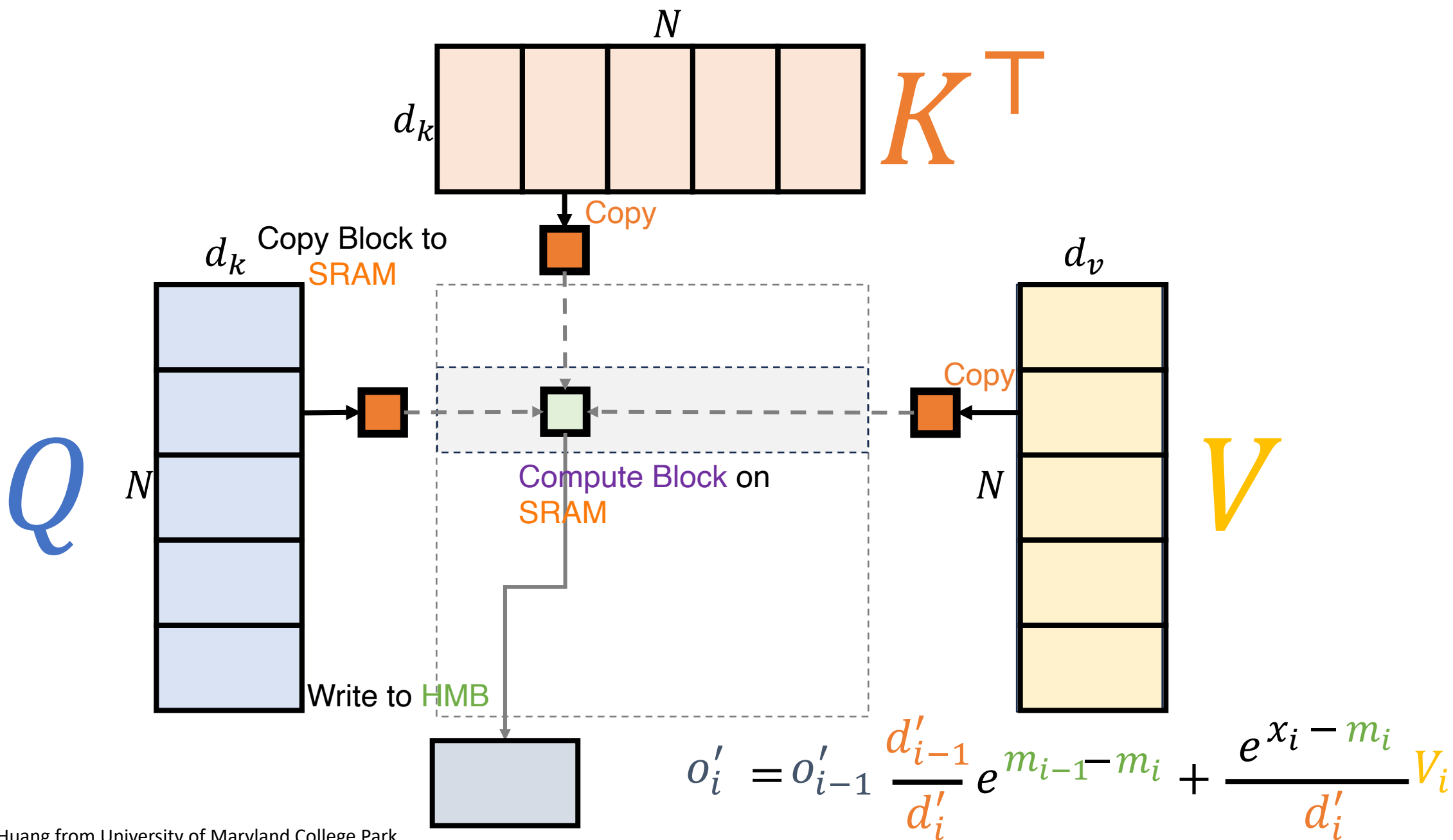


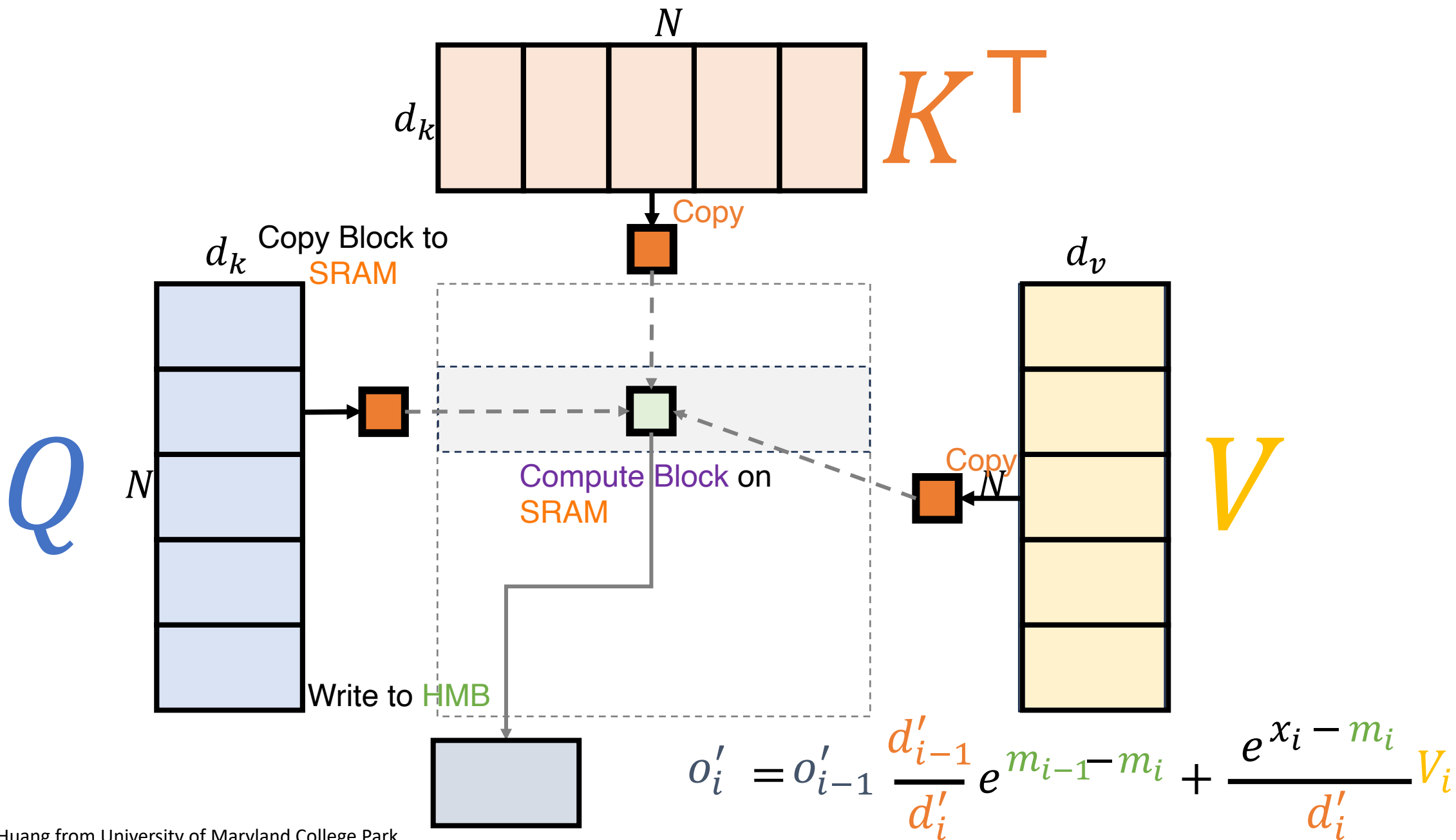


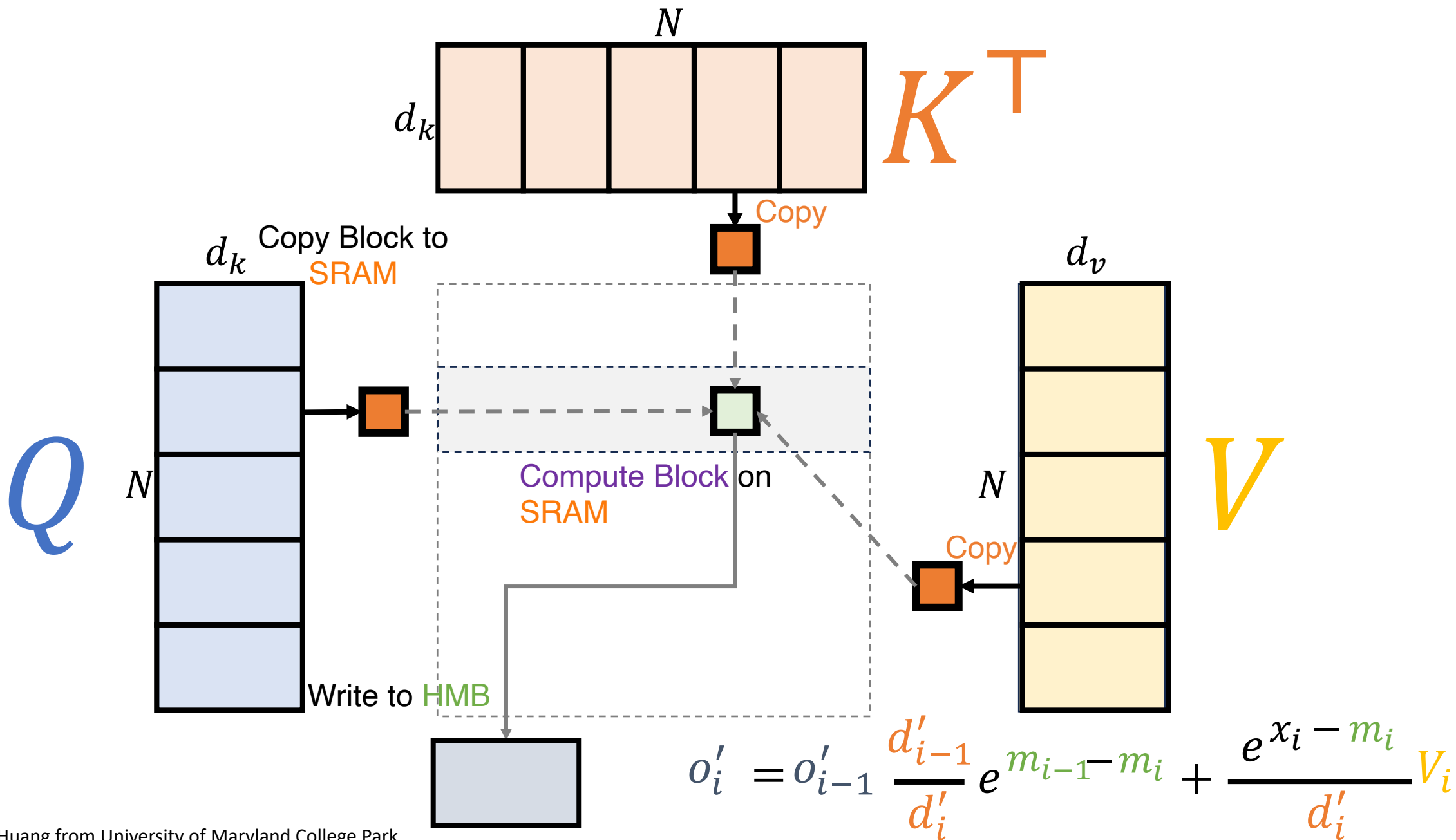


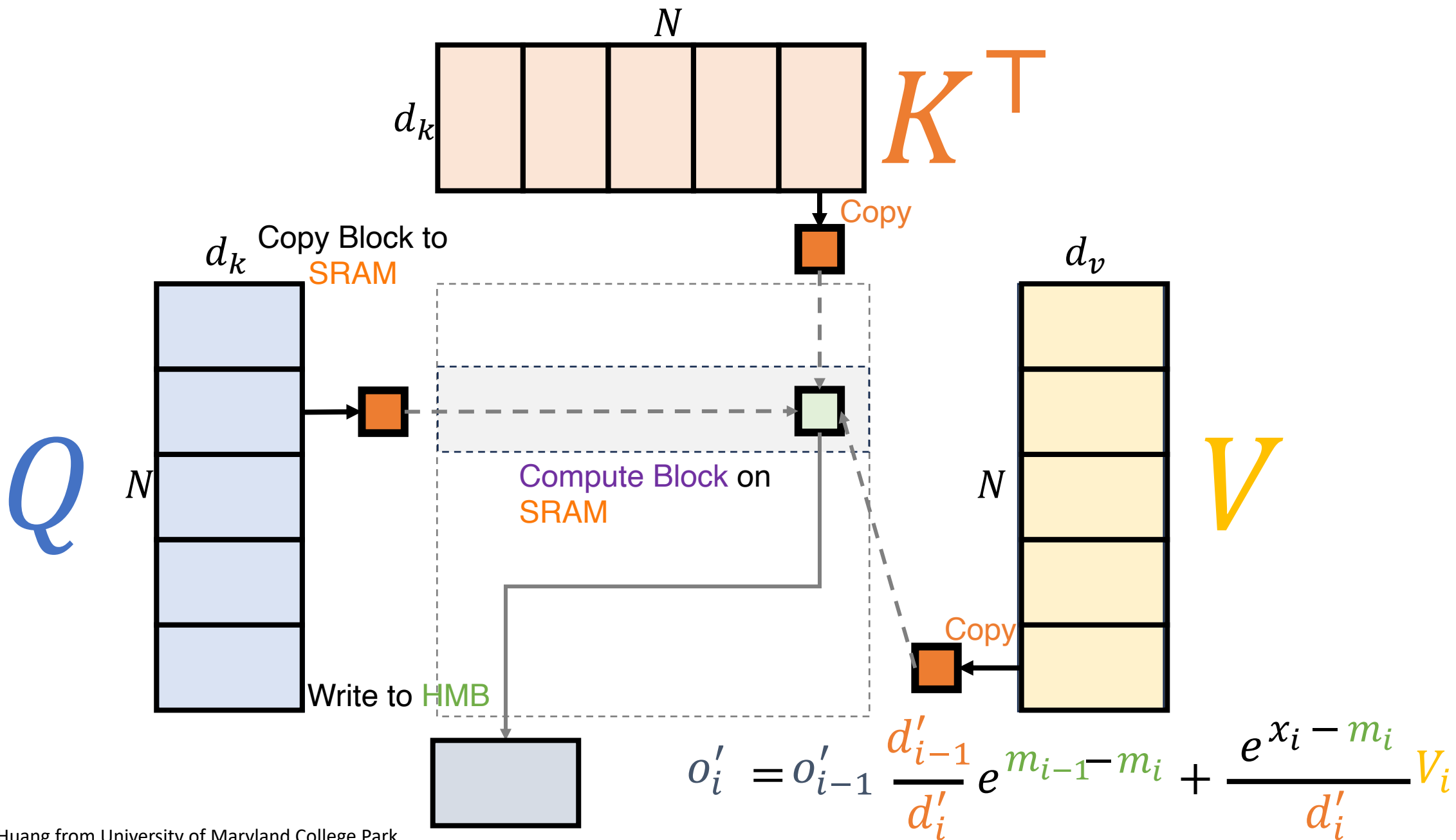












Flash Attention

Algorithm 1 FLASHATTENTION-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
- 3: **for** $1 \leq i \leq T_r$ **do** ← **Outer Loop Execution**
 $\mathbf{Q}$ traversing $\mathbf{Q}$ submatrices first;
- 4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
- 5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}$, $\ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}$, $m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
- 6: **for** $1 \leq j \leq T_c$ **do** ← **Inner Loop Execution**
 Innerloop: traversing $\mathbf{K}$ and $\mathbf{V}$ submatrices
- 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
- 10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
- 11: **end for**
- 12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
- 13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
- 14: Write $\mathbf{O}_i$ to HBM as the i -th block of $\mathbf{O}$.
- 15: Write L_i to HBM as the i -th block of L .
- 16: **end for**
- 17: Return the output $\mathbf{O}$ and the logsumexp L .

HBM Writeback Optimization

Write back to HBM after the inner-loop:
reducing HBM read/write compared with **v1**

Flash Attention v2

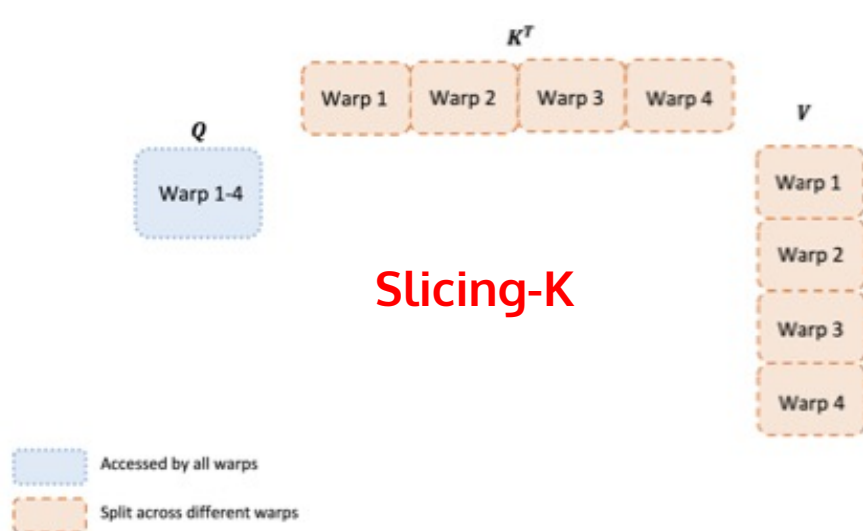
- The same order of I/O complexity
- Reduced read/write of $\mathbf{O}_i$: write $\mathbf{O}_i$ to HBM after traversing all $\mathbf{K}/\mathbf{V}$ blocks at the inner loop

Flash Attention

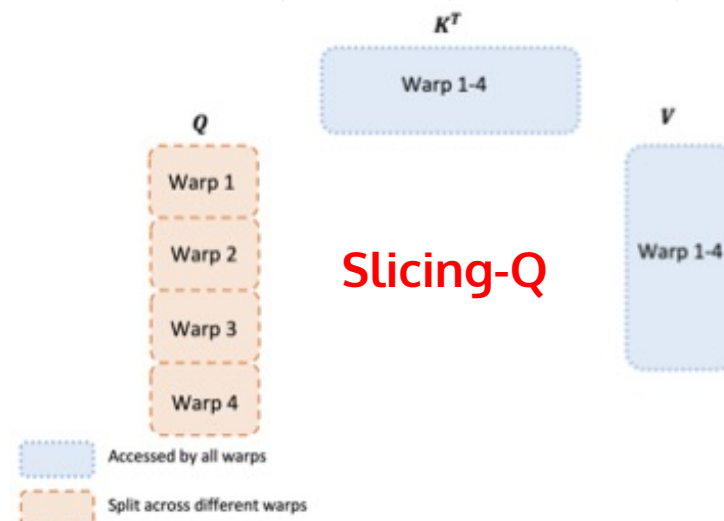
- FlashAttention v2

- Better Work Partitioning

- A thread block is partitioned into multiple warps (32 threads per warp, 4~8 warps per block)



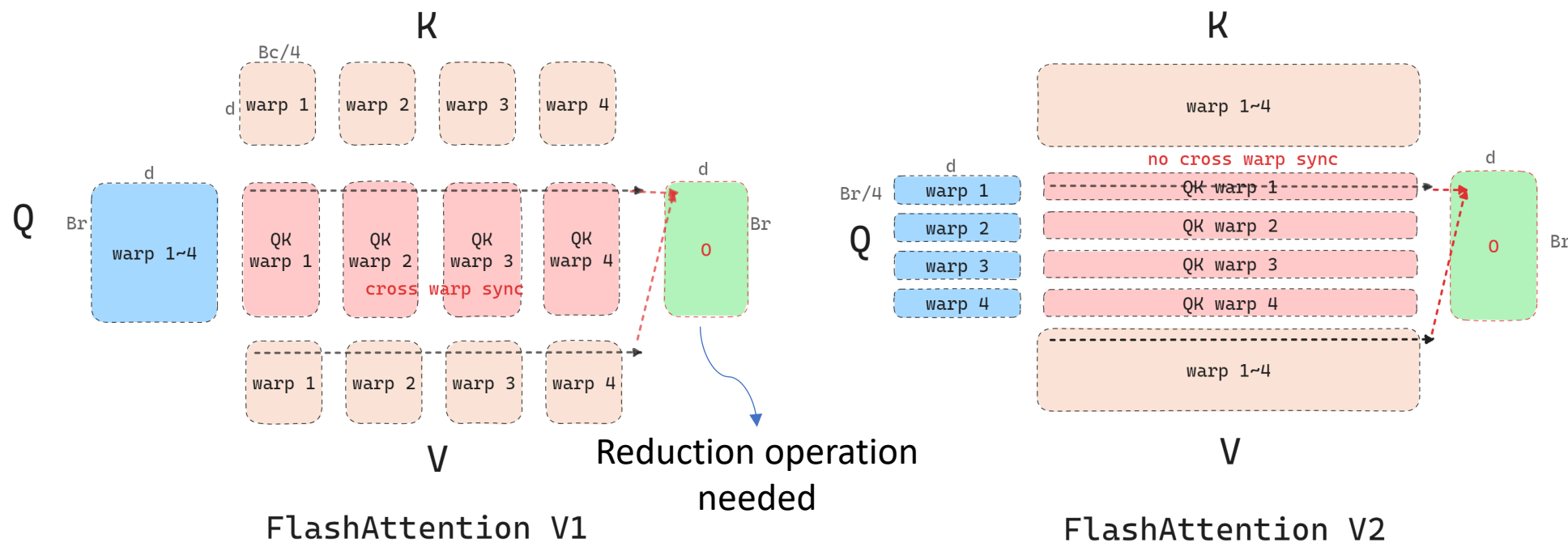
- Slicing K/V at each Warp while maintaining complete Q
 - QK^T has four partitions, and $(QK^T)V$ needs REDUCTION (write to shared memory, synchronize, and add-up intermediate results)



- Slicing Q while keeping K/V accessible by all Warps
 - Each warp performs matrix multiply to obtain a slice of QK^T , and no intra-block communication

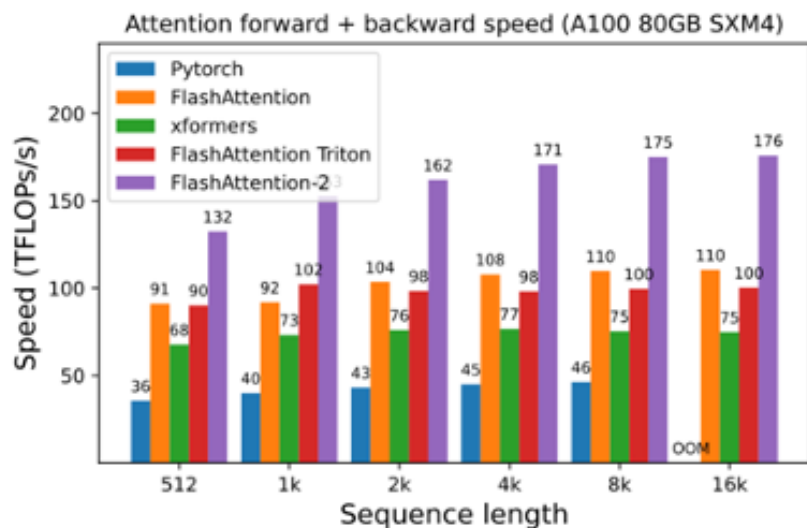
Flash Attention

- FlashAttention v2
 - Better Work Partitioning
 - A thread block is partitioned into multiple warps (32 threads per warp, 4~8 warps per block)

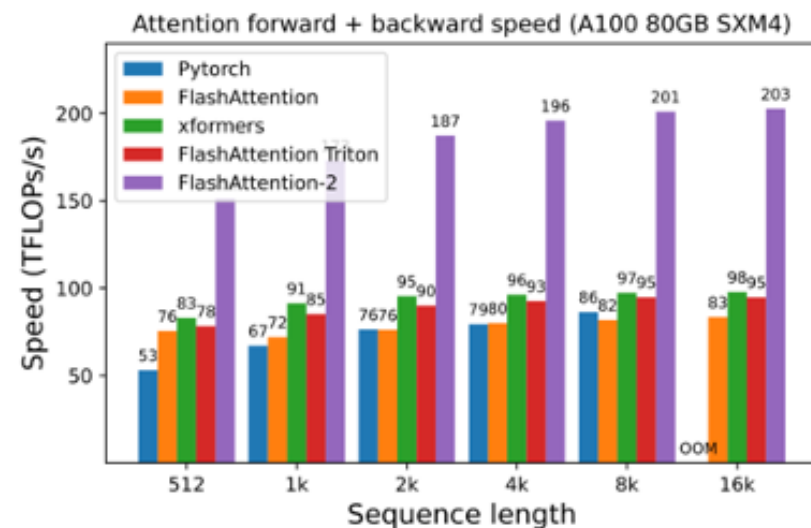


Flash Attention

- FlashAttention v2
 - Performance speedup



(a) Without causal mask, head dimension 64

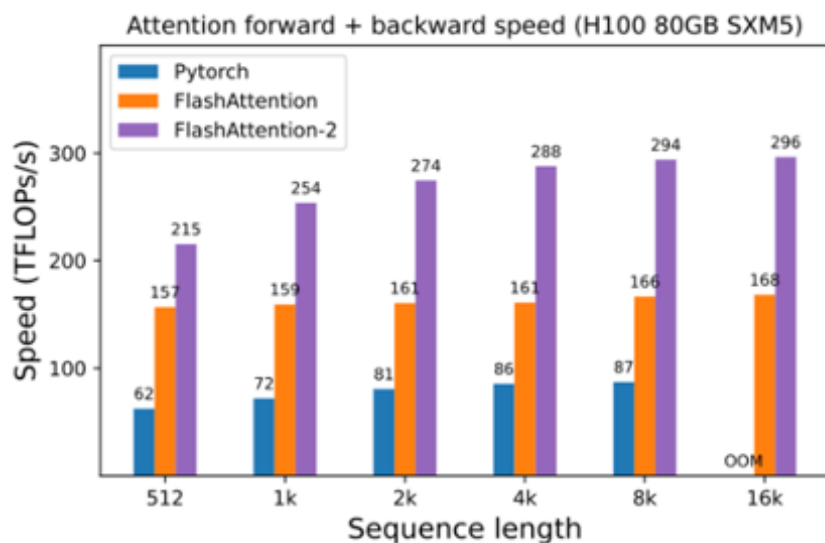


(b) Without causal mask, head dimension 128

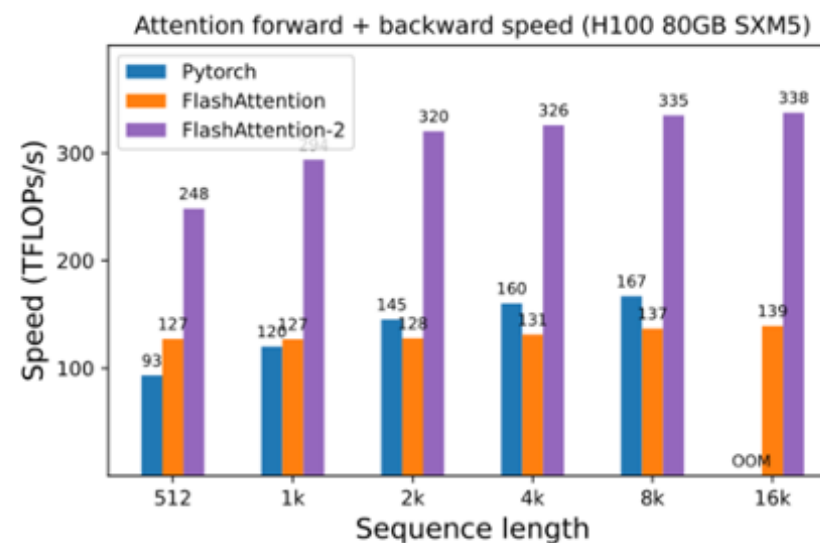
Attention forward + backward speed on A100 GPU

Flash Attention

- FlashAttention v2
 - Performance speedup



(a) Without causal mask, head dimension 64

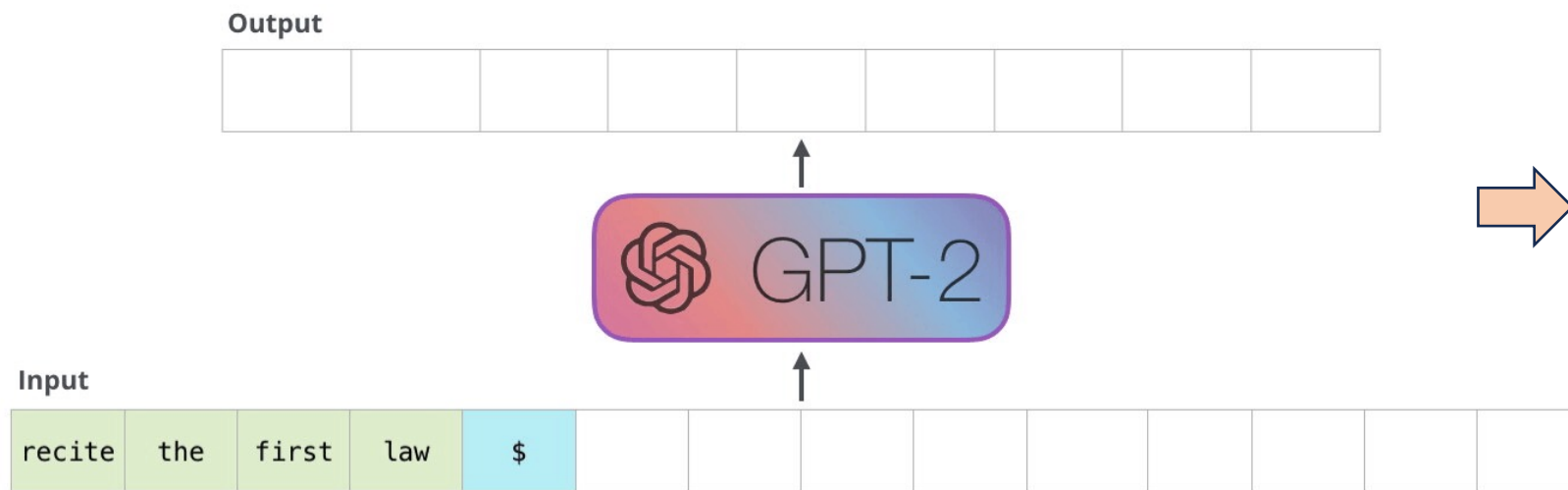


(b) Without causal mask, head dimension 128

Attention forward + backward speed on H100 GPU

Flash Attention

- Flash Decoding
 - New challenges in autoregressive decoding

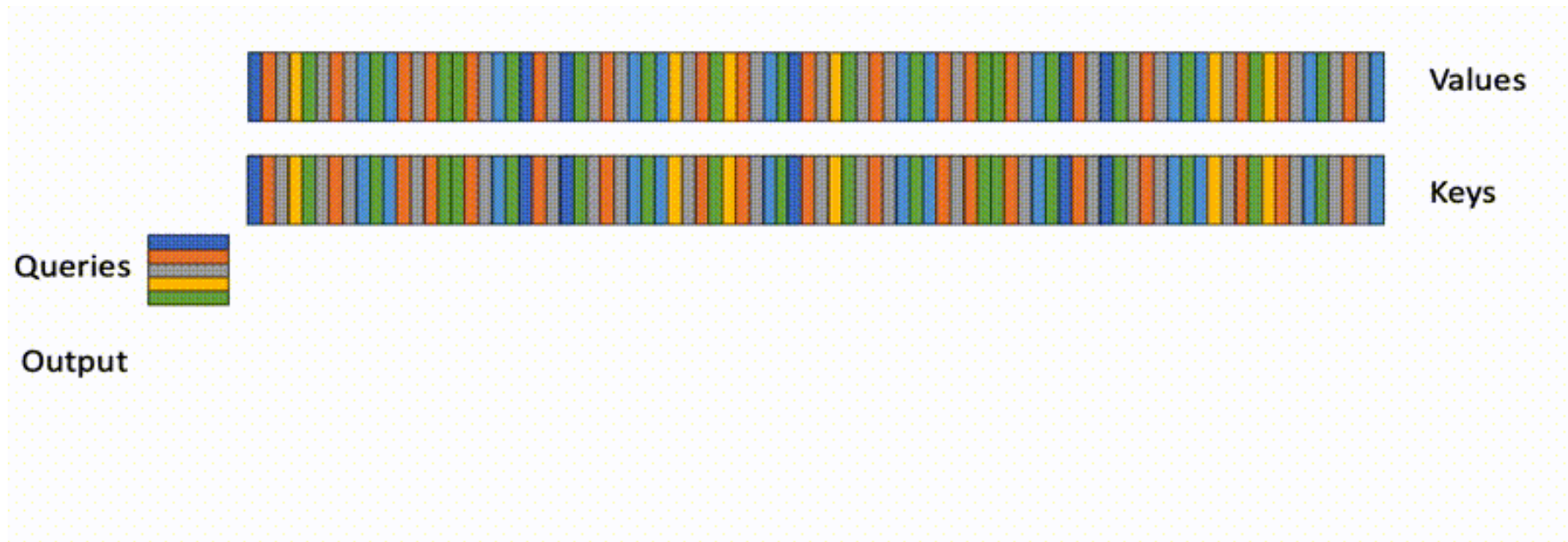


Query (Q) length is always 1, i.e. the newly generated token

- Key (K) and Value (V) keep growing to be extraordinarily large
 - Out of Memory error
 - Swapping to CPU memory which is very slow
 - Truncating the input causes poor performance

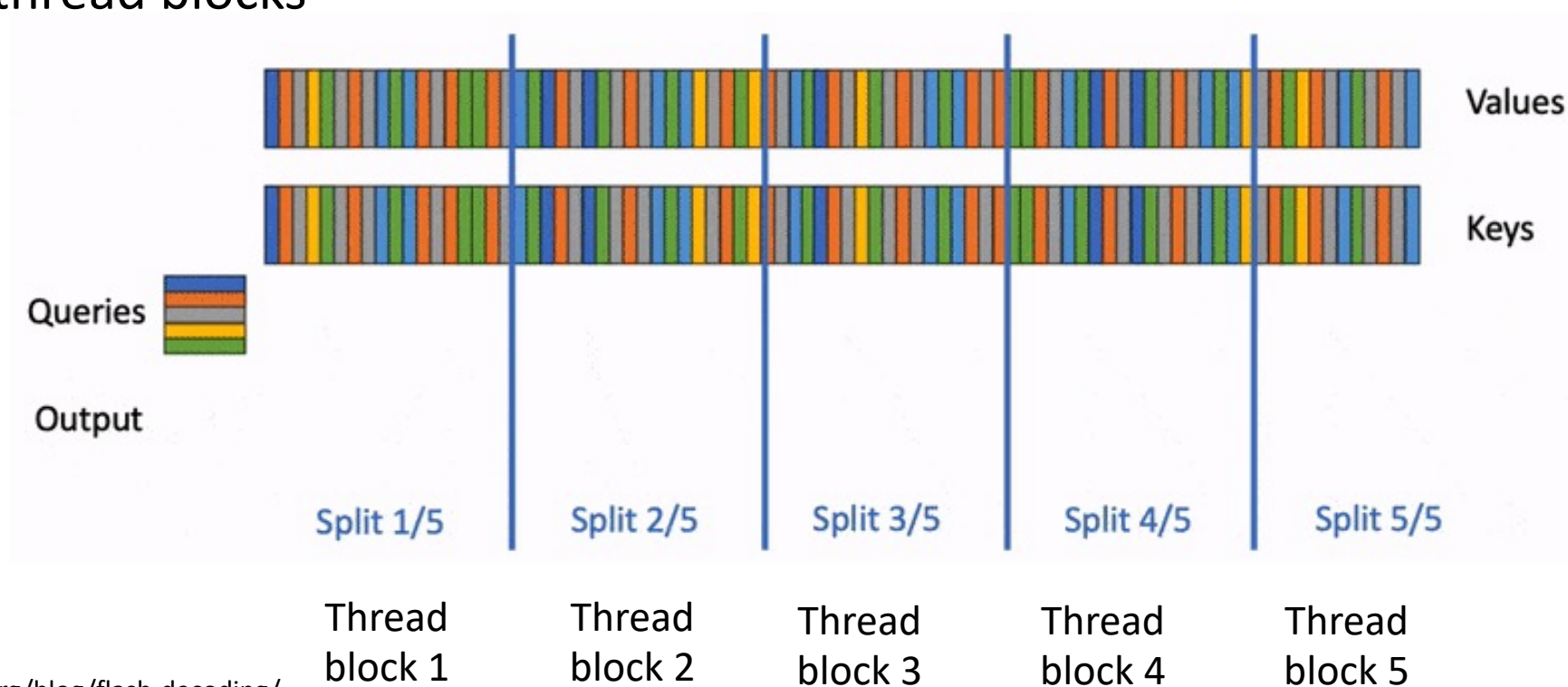
Flash Attention

- Flash Decoding
 - Low efficiency due to small Q (i.e. sequence length of 1 in decoding)



Flash Attention

- Flash Decoding
 - Sharding Key and Value matrices (e.g. 5 pieces) in order to create many more thread blocks

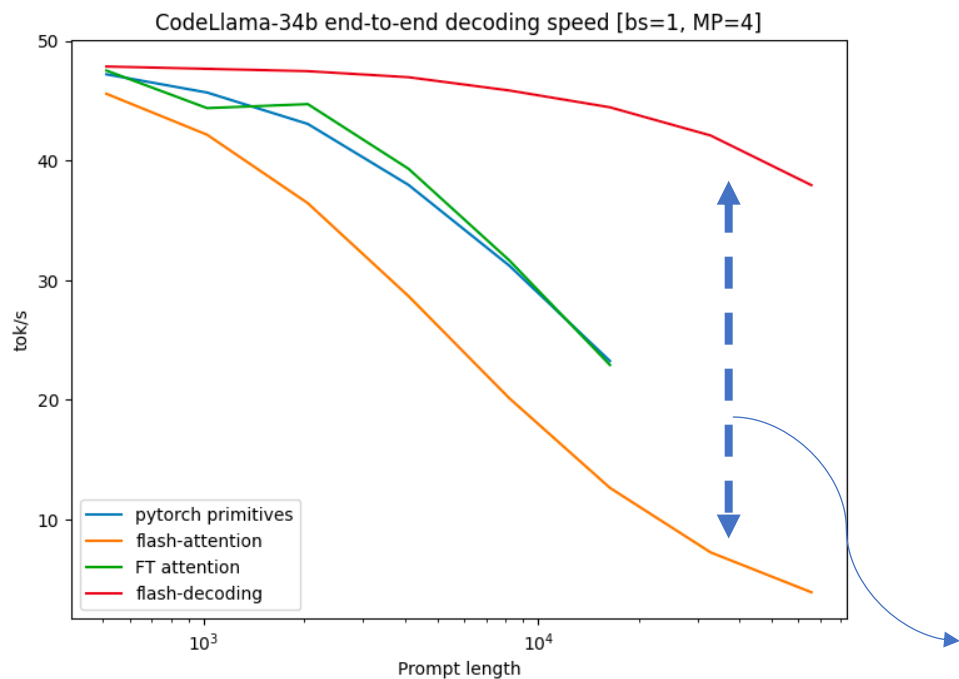


Flash Attention

- Flash Decoding
 - Computing attention scores with K/V splits using FlashAttention v1/v2
 - Only obtaining local ***max*** and local ***sum***
 - Output O not properly scaled
 - Launching an independent ***REDUCE*** kernel
 - Obtaining global ***max*** and global ***sum***
 - Rescaling local O to obtain the final output

Flash Attention

- Flash Decoding



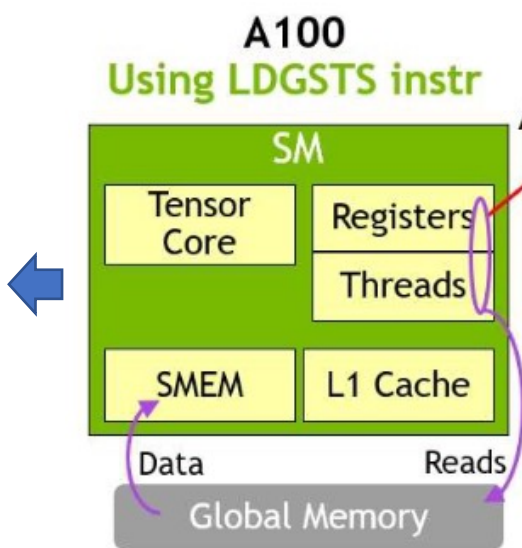
Setting \ Algorithm	PyTorch Eager	Flash-Attention v2.0.9	Flash-Decoding
B=256, seqlen=256	3058.6	390.5	63.4
B=128, seqlen=512	3151.4	366.3	67.7
B=64, seqlen=1024	3160.4	364.8	77.7
B=32, seqlen=2048	3158.3	352	58.5
B=16, seqlen=4096	3157	401.7	57
B=8, seqlen=8192	3173.1	529.2	56.4
B=4, seqlen=16384	3223	582.7	58.2
B=2, seqlen=32768	3224.1	1156.1	60.3
B=1, seqlen=65536	1335.6	2300.6	64.4
B=1, seqlen=131072	2664	4592.2	106.6

Flash Decoding versus Flash Attention

Flash Attention v3

- **TMA (Tensor Memory Accelerator):** Transferring large blocks of data very efficiently between global memory and shared memory, and supporting asynchronous copies between Thread Blocks in a Cluster.

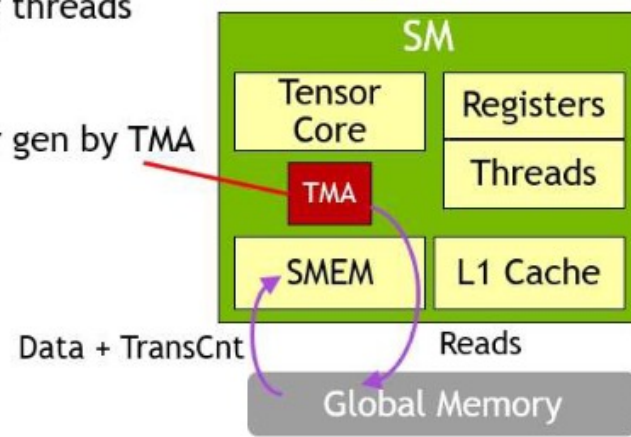
On A100, in the left part, asynchronous memory copies were executed using a special `LoadGlobalStoreShared` instruction, so the threads were responsible for generating all addresses and looping across the whole copy region.



Addr gen by threads

Addr gen by TMA

H100
Using TMA Unit

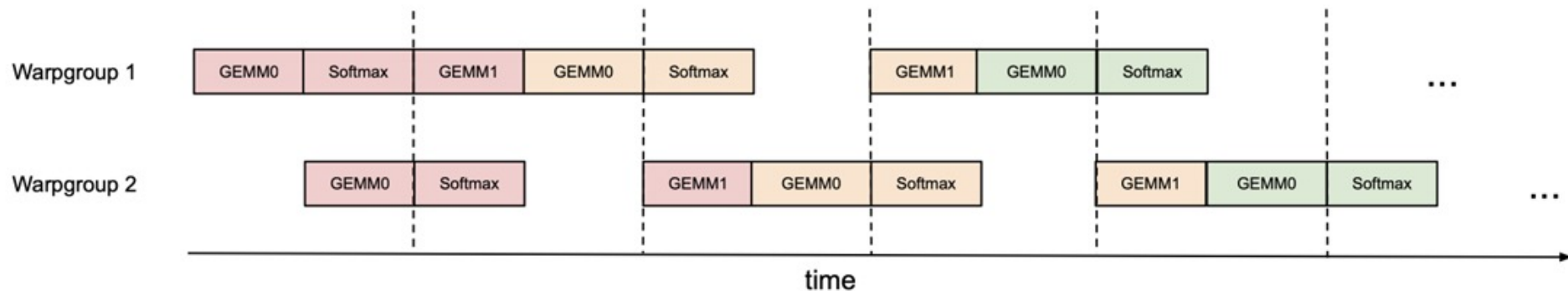


On Hopper, TMA takes care of everything. A single thread creates a copy descriptor before launching the TMA, and from then on address generation and data movement are handled in hardware.

A key advantage of TMA is it frees the threads to execute other independent work

Flash Attention v3

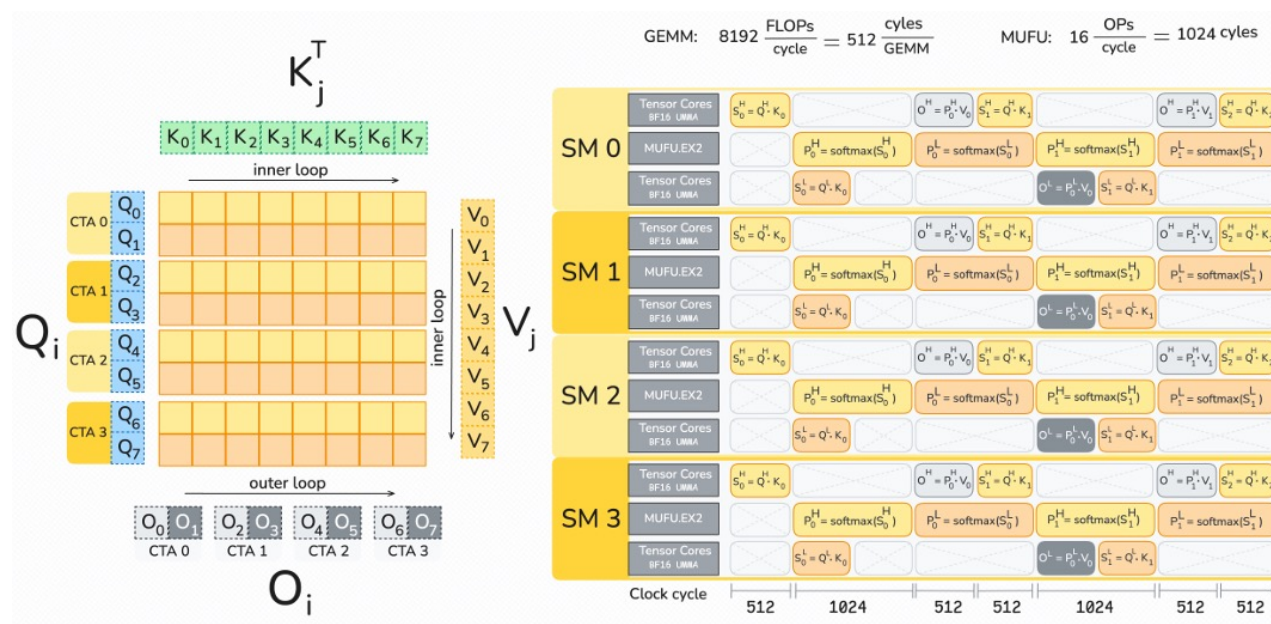
- TMA Asynchrony
 - Producer-Consumer asynchrony
 - Warp-specialization: some warps for data movement and some for tensor core computes
 - Ping-pong scheduling
 - *matmul* is more efficient than *softmax* in terms of actual FLOPs utilization



Using two warp groups to interleave the execution of GEMM and Softmax

Flash Attention v4

- TMA Asynchrony (for reference only)
 - Greatly increment on tensor core computing power, but non-matmul operations become the bottleneck



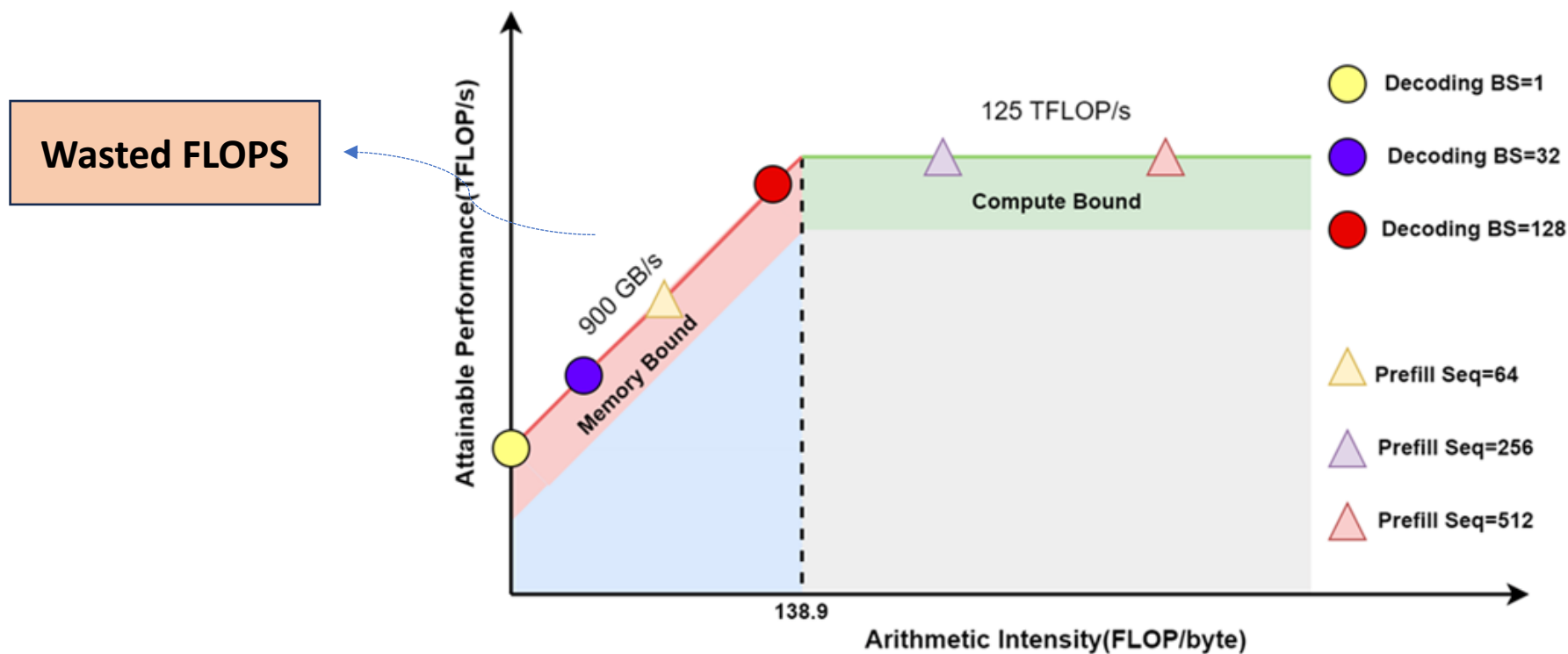
Optimizations on asynchronous matrix multiply-accumulate, and shared memory read/write

Inference Optimization: Outline

- Overview
- Attention Computation Optimization
 - Sparse Attention
 - Linear Attention
 - Flash Attention
- **Continuous Batching**
- KV Cache Optimization
- Speculative Decoding
- Distributed Serving

Batching to Meet Compute-Bound

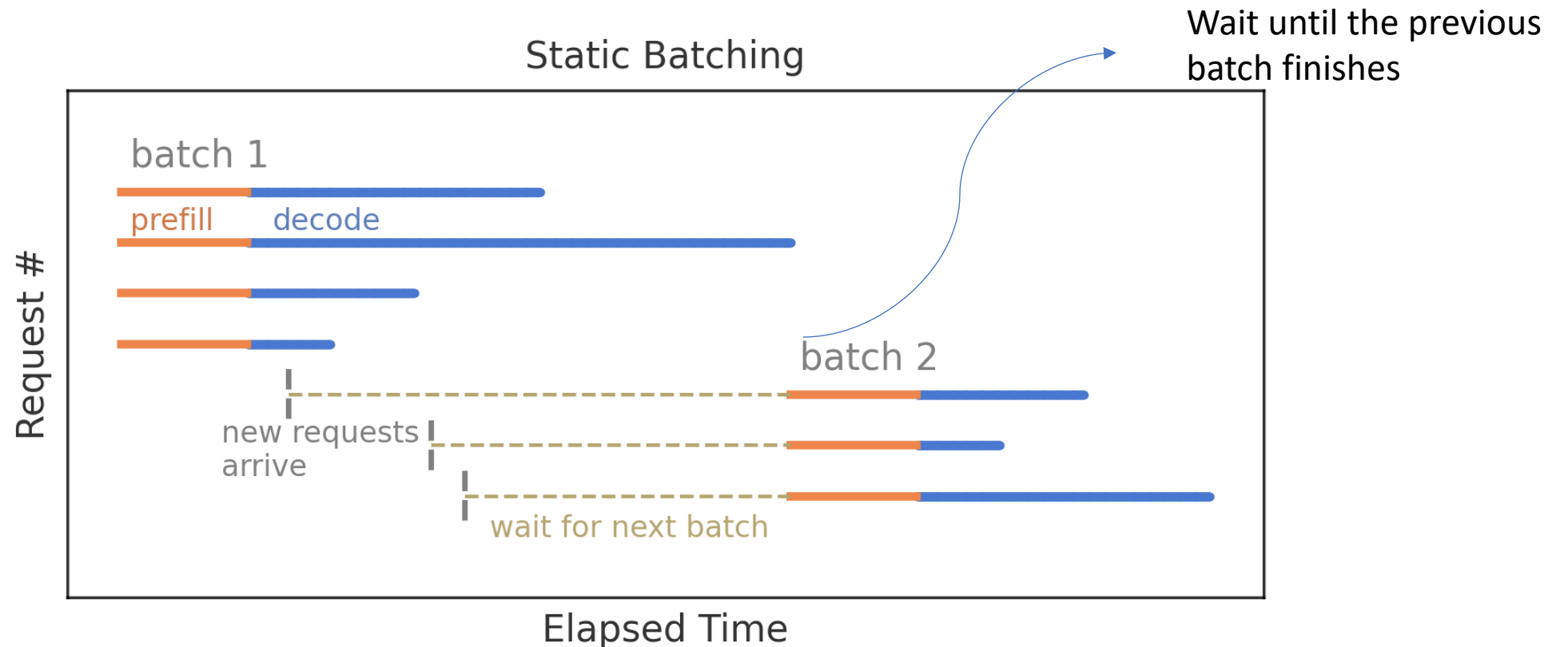
- Why batching multiple requests?
 - Generating tokens for a large number of prompts to match the compute bound



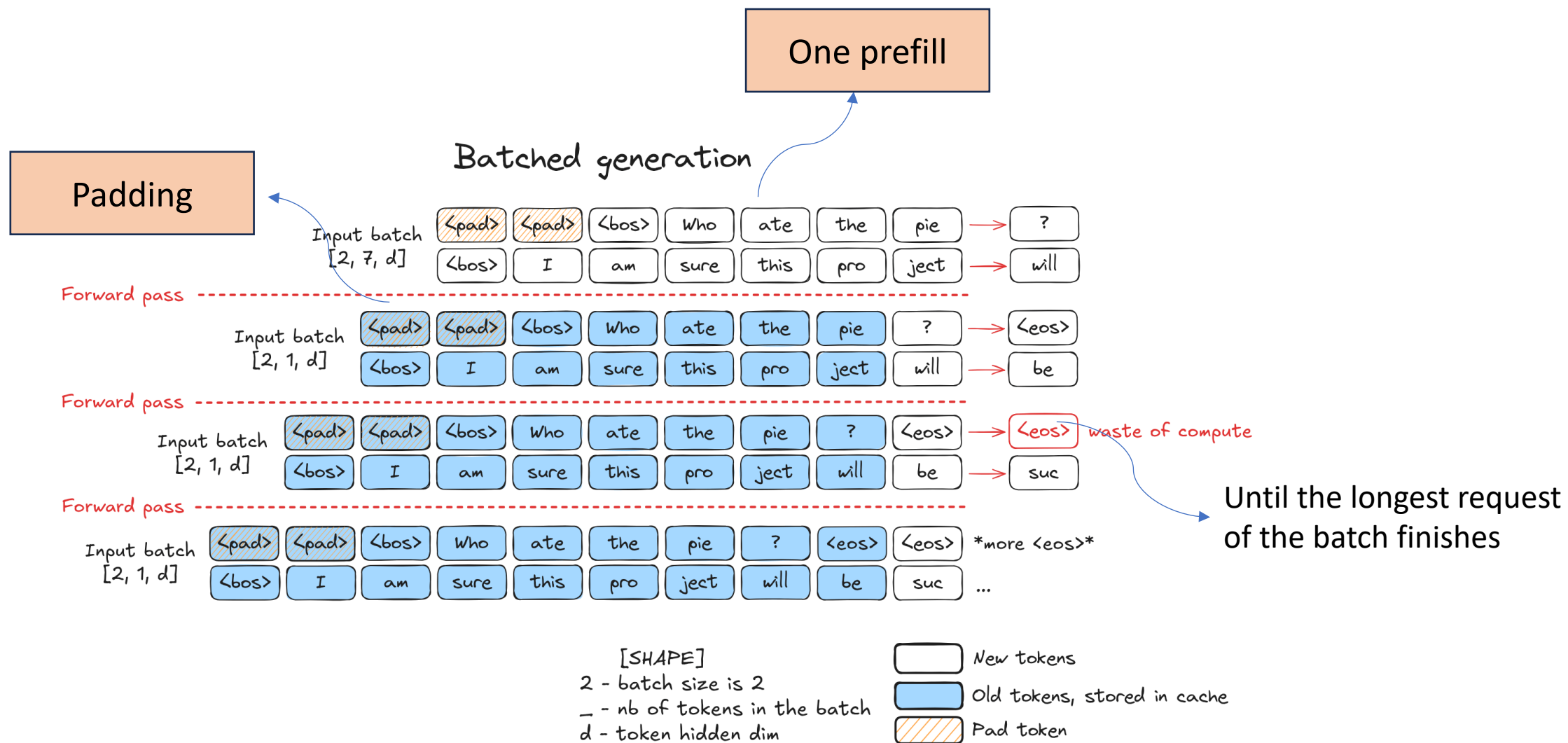
Roofline model of NVIDIA V100 GPU

Static Batching

- Naively serving multiple requests simultaneously
 - Unequal input sequence lengths, unknown output sequence lengths

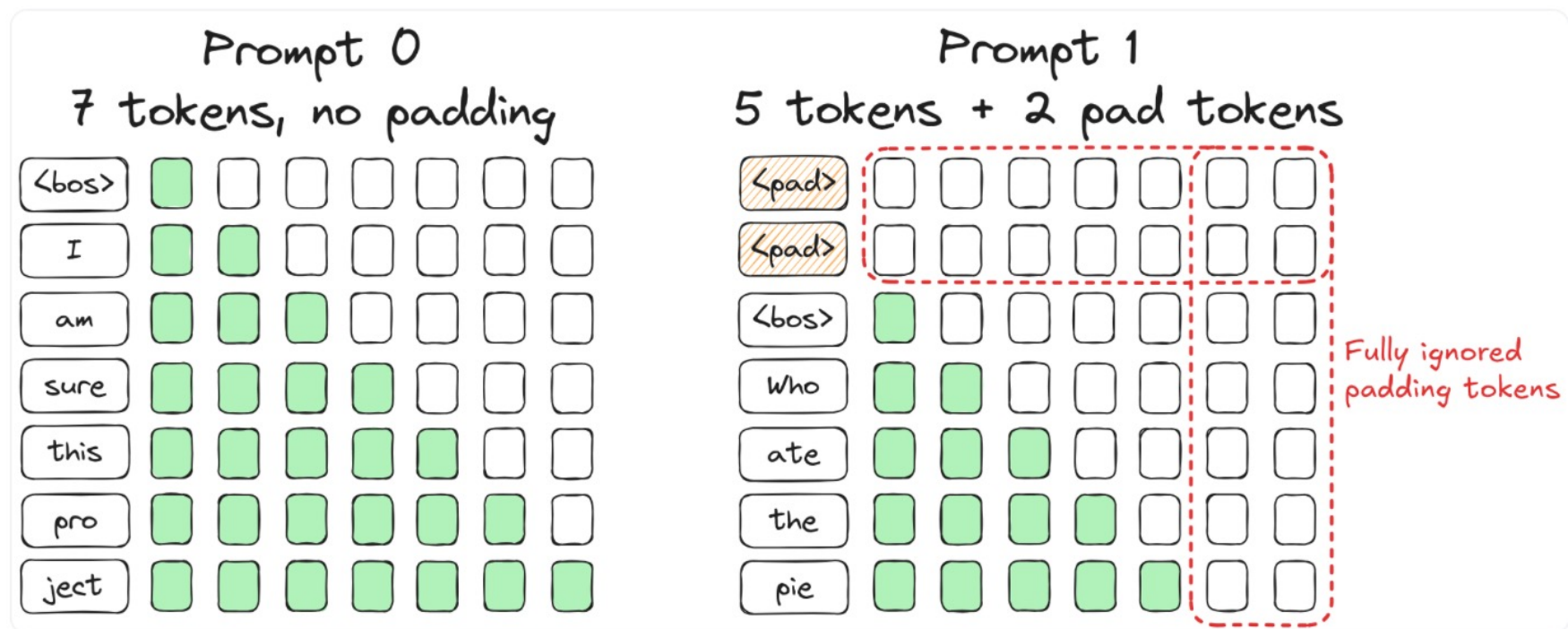


Static Batching



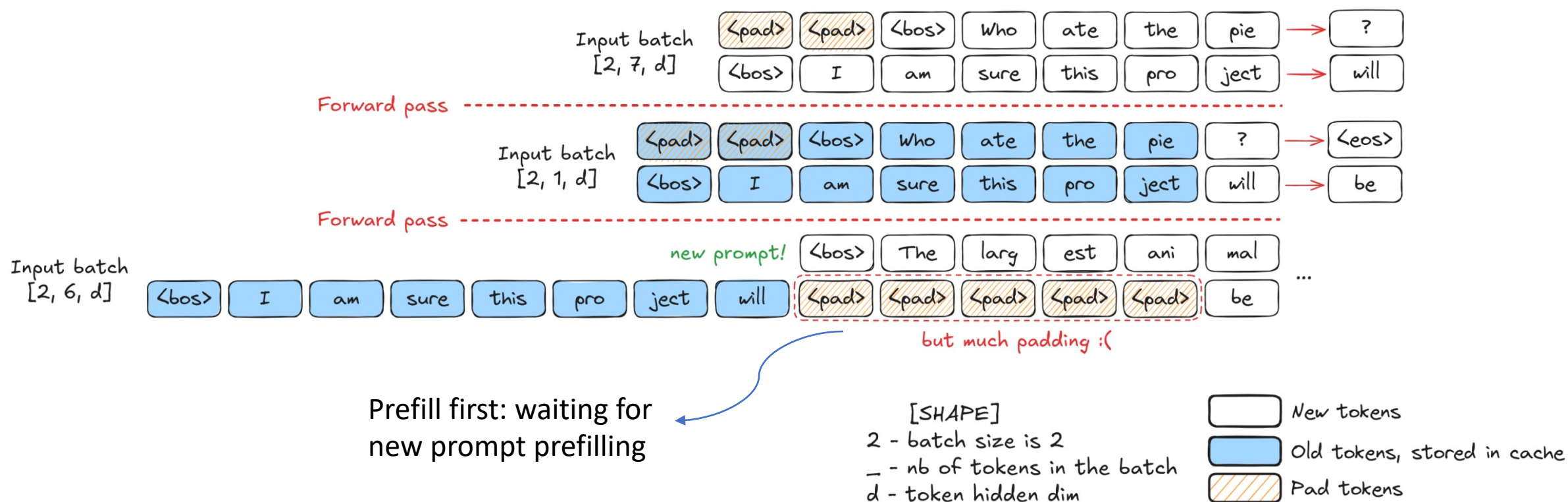
Static Batching

- How to implement it?
 - Avoiding the computations of certain diagonal lines via attention masking



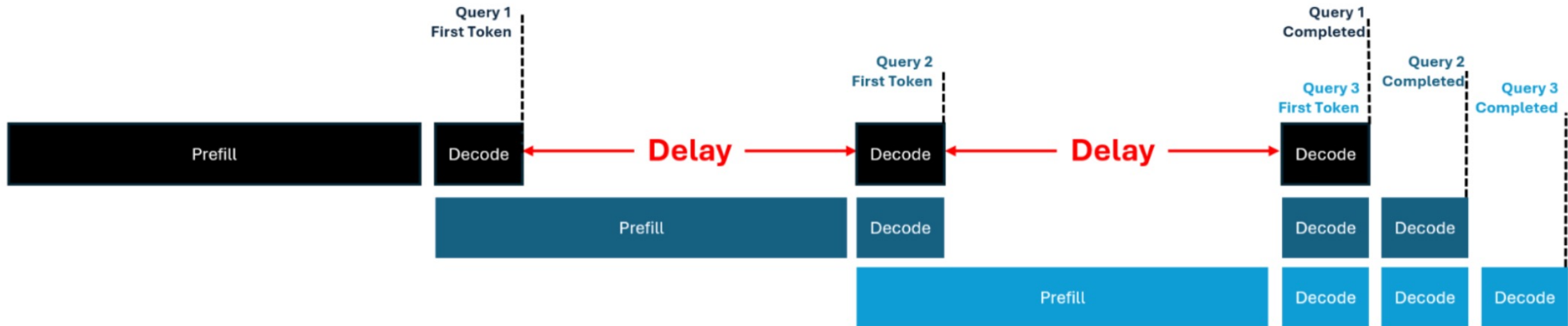
Dynamic Scheduling

- Insert an inference request in the batch when an old request finishes



Continuous Batching

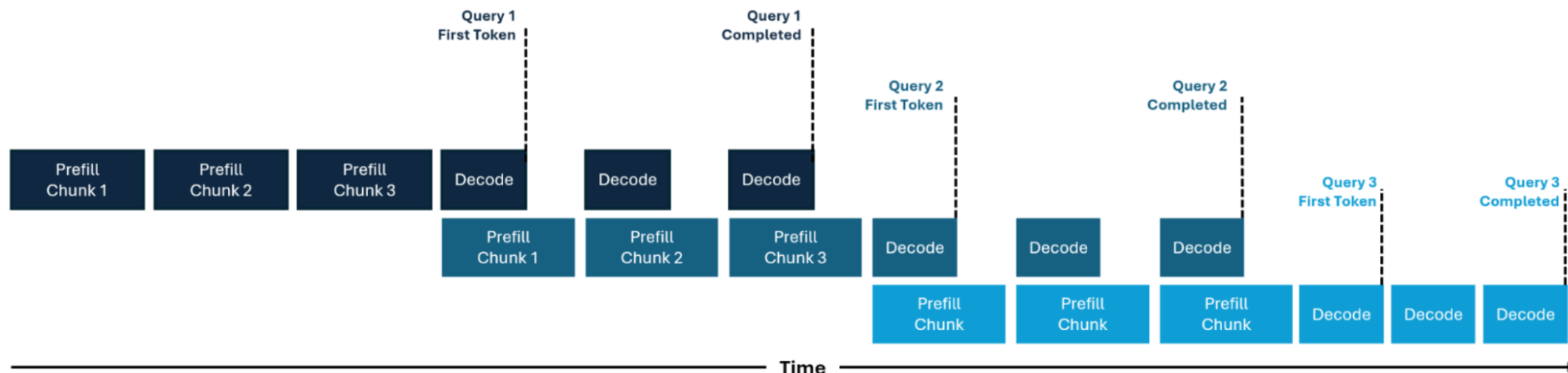
- Question remains unsolved: how to make **prefill** of new requests and **decode** of existing requests operate *in parallel*?



Continuous batching without chunked prefill

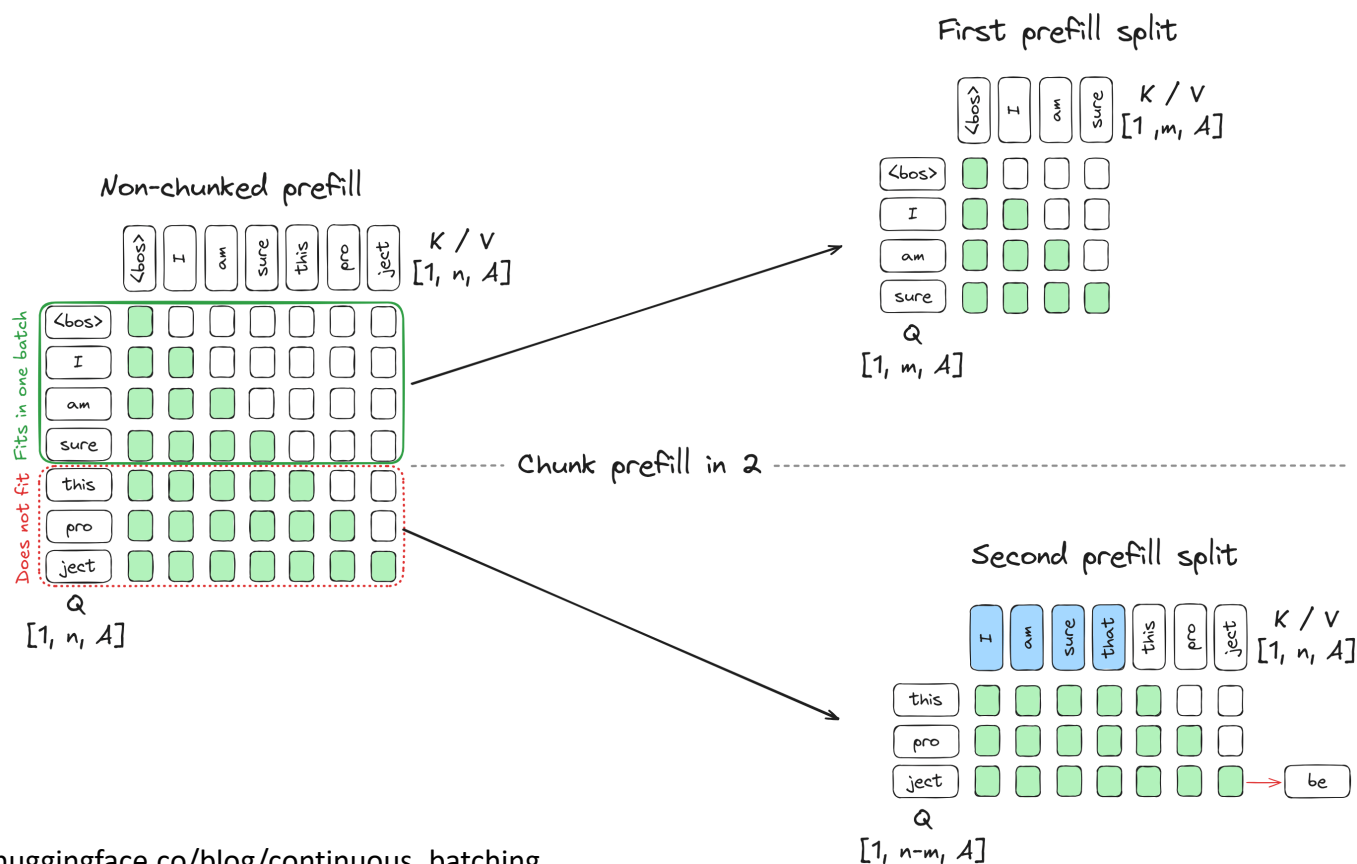
Chunked Prefill

- **Chunked Prefill**
 - **Elastic Scheduling:** split the entire prompt into smaller chunks: more scheduling flexibility and enables mixing prefill and decode.
 - **Decode-Maximal Batching:** combine a single prefill chunk with multiple decode requests in the same batch, allowing decode operations to "piggyback" on the more compute-intensive prefill operations



Chunked Prefill

- How to implement it?
 - Combined together to acquire a global view of complete prefill



	k0	k1	k2	k3
q0	1	-	-	-
q1	1	1	-	-
q2	1	1	1	-
q3	1	1	1	1

attention mask during first chunk prefill

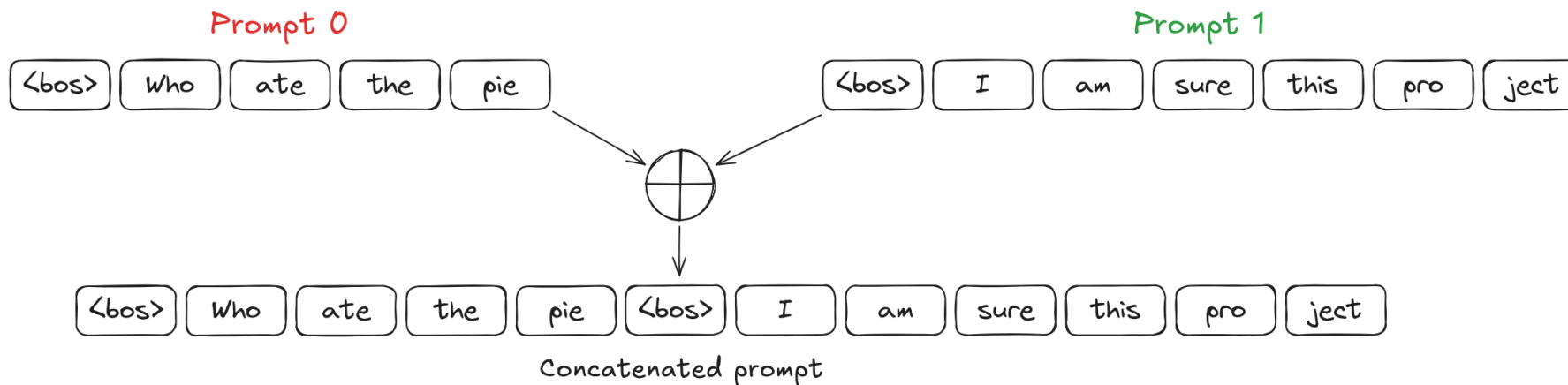
	k0	k1	k2	k3	k4	k5	k6	k7
q4	1	1	1	1	1	-	-	-
q5	1	1	1	1	1	1	-	-
q6	1	1	1	1	1	1	1	-
q7	1	1	1	1	1	1	1	1

attention mask during second chunk prefill

Setting the correct attention mask

Idea Batching

- Concatenating prompts
 - **Decode** is equivalent to **prefill** with sequence length of 1
 - **Decode** and **Prefill** of different prompts operate simultaneously without bubbles



- Question: how to concatenate them?

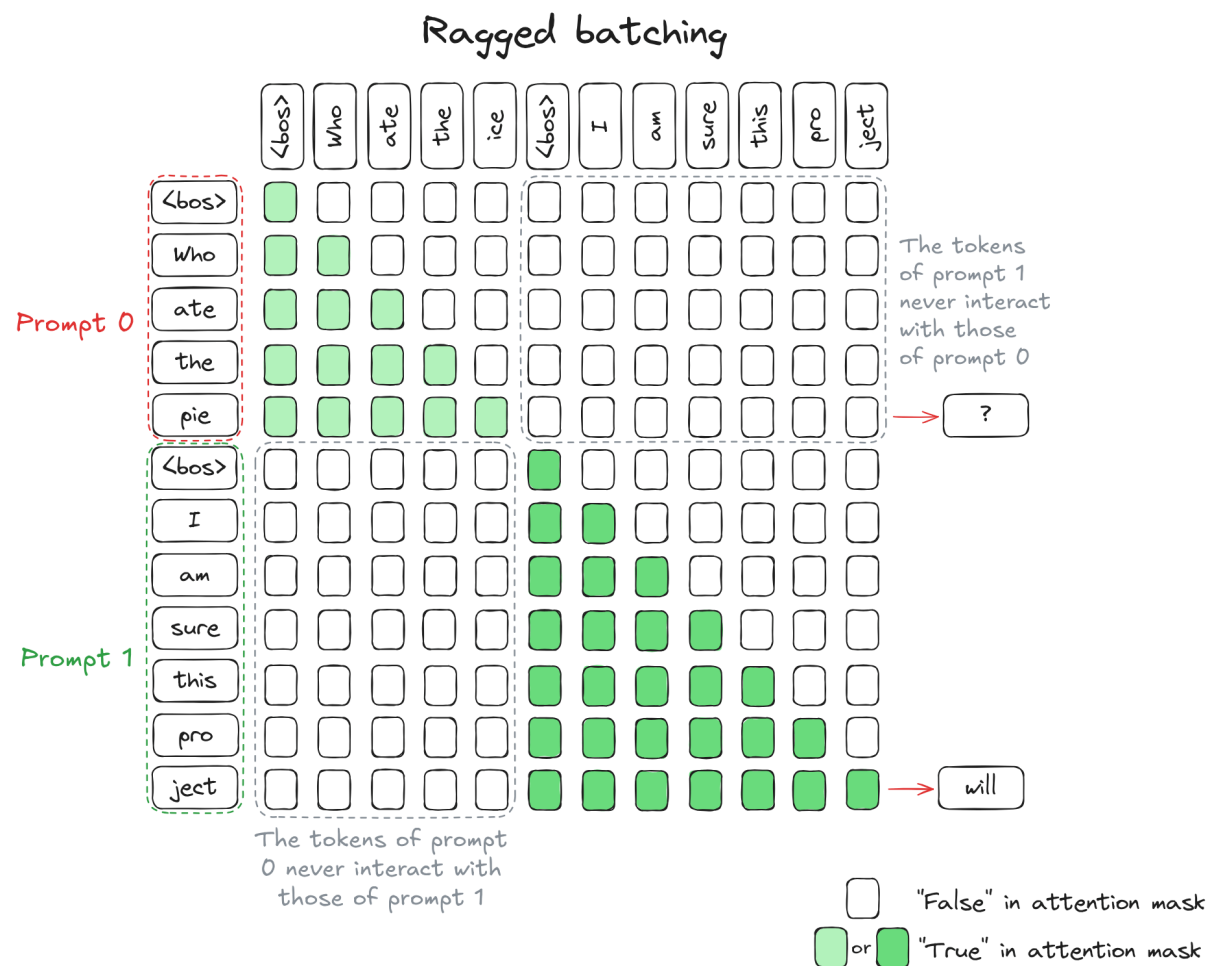
Continuous Batching

- Continuous batching = ragged batching + dynamic scheduling

- Attention scores of tokens belonging to

prompt 0 and *prompt 1* should not be computed together

- Ragged batching because sequence lengths are 'ragged' or uneven, no need for padding tokens

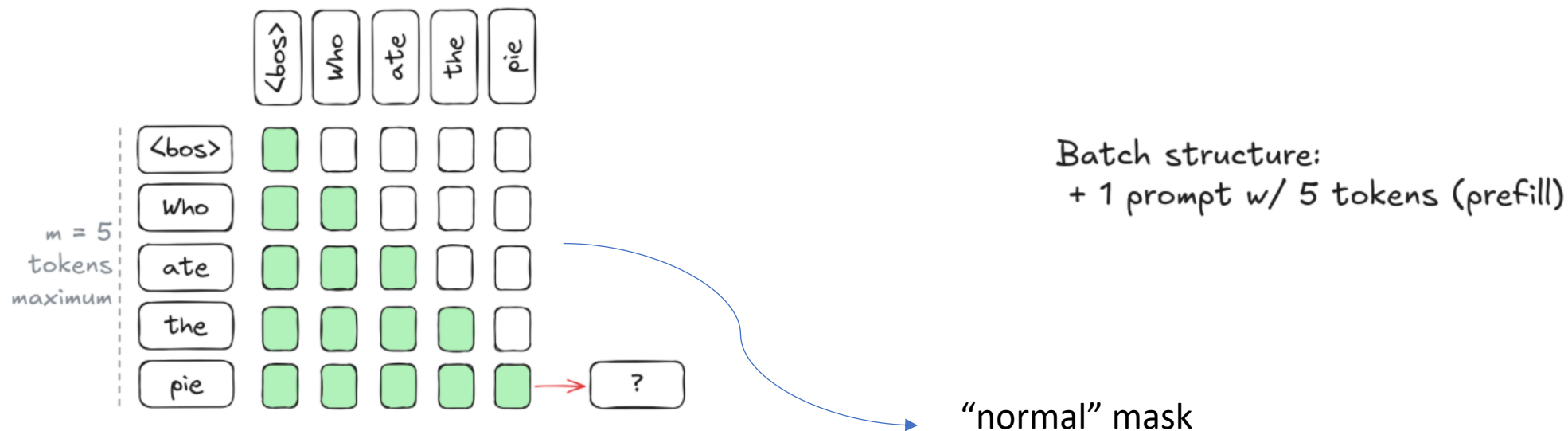


Continuous Batching

- Continuous batching = ragged batching + dynamic batching

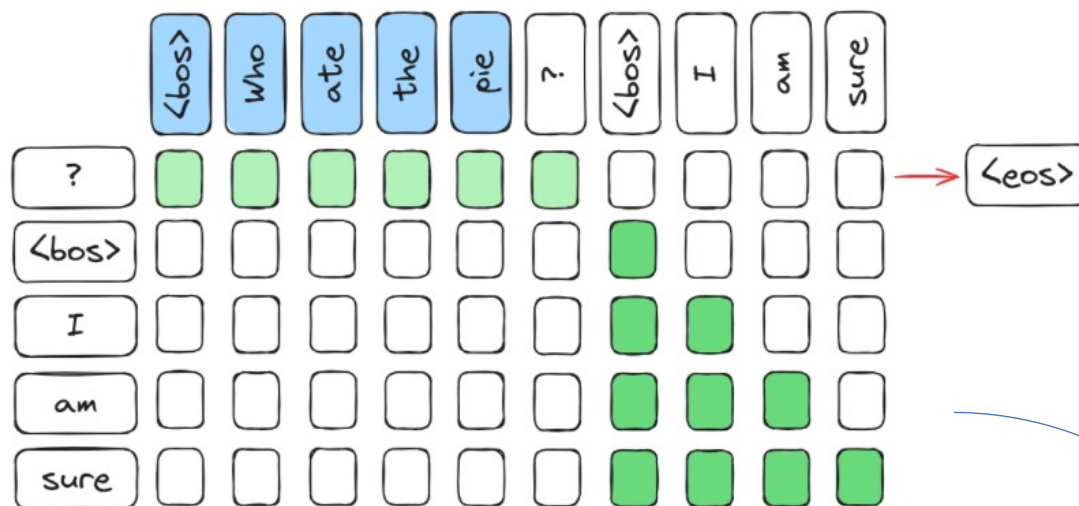
Continuous batching

Initial prompts: ["Who ate the pie", "I am sure this project", "The largest animal"]



Continuous Batching

- Continuous batching = ragged batching + dynamic batching
 - After the completion of the first request, the prefill of the second prompt is scheduled
 - Only the **four tokens** in the second prompt is processed concurrently via chunked prefill



Batch structure:
+ 1 prompt w/ 1 tokens (decode)
+ 1 prompt w/ 4 tokens (chunked prefill)

Forward pass -----

“ragged” mask

Continuous Batching

- Continuous batching = ragged batching + dynamic batching
 - 'this', 'pro', 'ject' are scheduled via chunked prefill and attend to all previous tokens in this prompt
 - Another prompt can also be prefilled



Batch structure:
+ 1 prompt w/ 3 tokens (end of chunked prefill)
+ 1 prompt w/ 2 tokens (chunked prefill)

“ragged” mask

Thanks!

